

Бизнес-логика программных систем

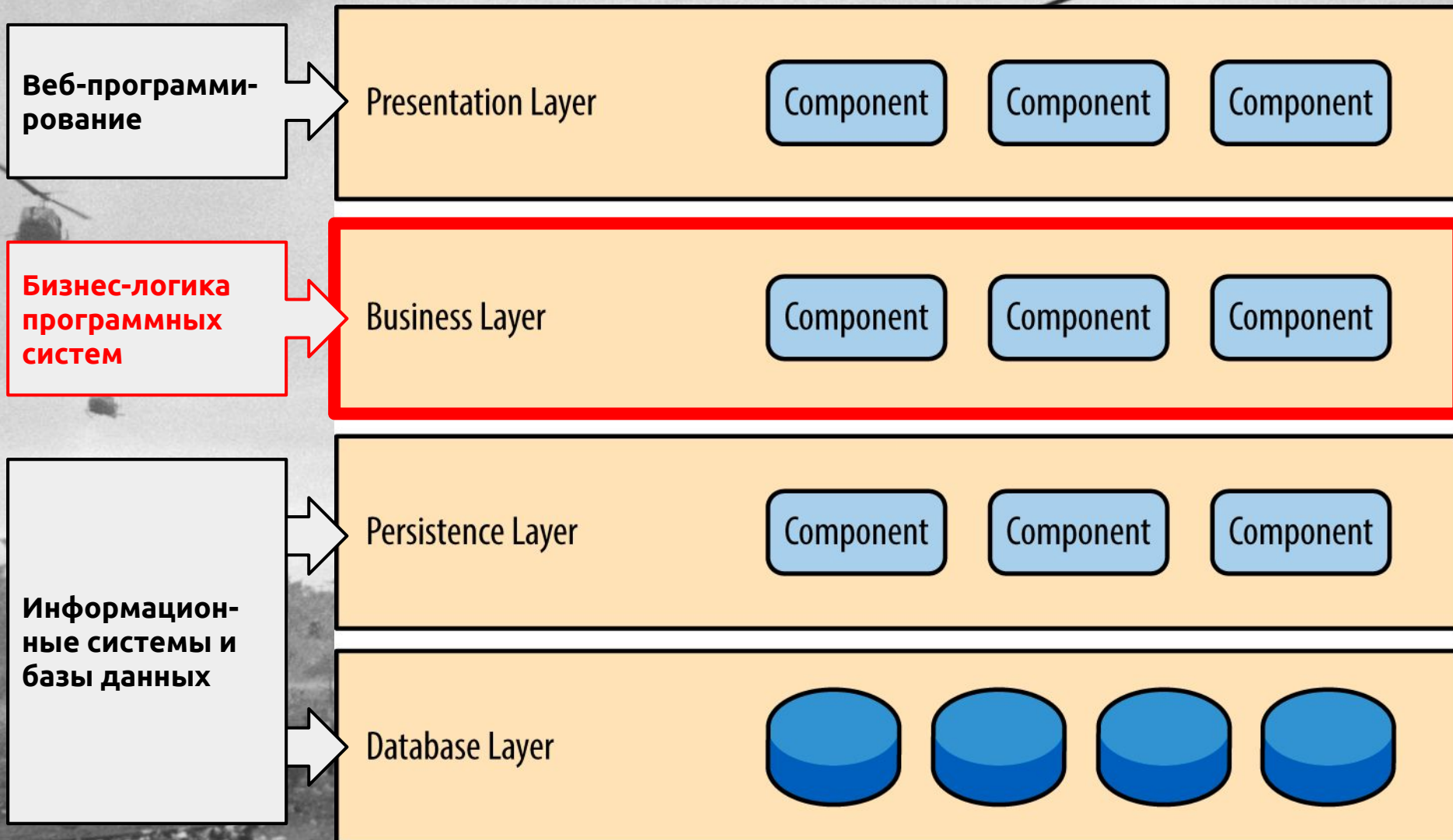
лектор – Цопа Е.А.
2021/22 уч. год



1. Введение

- Часть ИС, отвечающая за реализацию предметной области.
- В многоуровневых ИС выделяется в отдельный уровень, изолированный от остальных интерфейсами.
- Уровень БЛ находится между инфраструктурным уровнем (например, уровнем доступа к БД) и сервисным уровнем (или даже уровнем представления напрямую).
- Формирует основную вычислительную нагрузку на “железо”.
- Сложна в проектировании, обычно многопоточная, часто – распределённая.
- При разработке часто используется компонентный подход.

Бизнес-логика в многоуровневой архитектуре ИС



- CDI & IoC.
- Управление транзакциями.
- Политики безопасности.
- Распределённая обработка.
- Асинхронная обработка и обмен сообщениями.
- Планировщики задач.
- BPM & BPMN.

- Универсальные архитектурные паттерны – используются не только в БЛ.
- Удобны для “нарезки” бизнес-логики на компоненты и их изоляции друг от друга.
- Удобны для управления зависимостями.
- Обычно реализуются внешним по отношению к компонентам БЛ фреймворком или платформой (Spring, Java EE).

- Транзакция – набор операций, выполняемых как “единое целое”.
- Пример – перевод денег между счетами.
- Понятие на самом деле шире, чем транзакции на уровне в БД.
- Разные технологические стеки предлагают разные реализации управления транзакциями.

- “Закрыть” веб-приложение http basic auth обычно недостаточно!
- Политики задают “сквозные” правила доступа к компонентам архитектуры приложения.
- Применимы не только для веб-приложений.
- Вторая важная задача – отображение набора прав доступа на набор ролей конкретного пользователя.

- Обеспечивает возможности горизонтального масштабирования БЛ.
- В общем случае – очень сложная задача.
- Реализация очень сильно зависит от выбранной платформы / фреймворка.
- “Каноничные” варианты – кластеризация и создание пулов компонентов.
- Более “модные” варианты – SOA (и, в частности, MSA).

Асинхронная обработка и обмен сообщениями

- Нужна для “тяжёлых” или фоновых задач, не требующих интерактивного взаимодействия.
- Много инструментов реализации в зависимости от платформы.
- Могут использоваться разные стандарты и протоколы (не только HTTP!).
- Существует отдельный класс ПО, реализующего такую обработку.

- Отвечают за запуск задач в определённое время или по расписанию.
- Присутствуют на всех уровнях ПО – от ядра ОС до прикладных сервисов.
- В зависимости от задачи можно использовать разные реализации планировщиков.

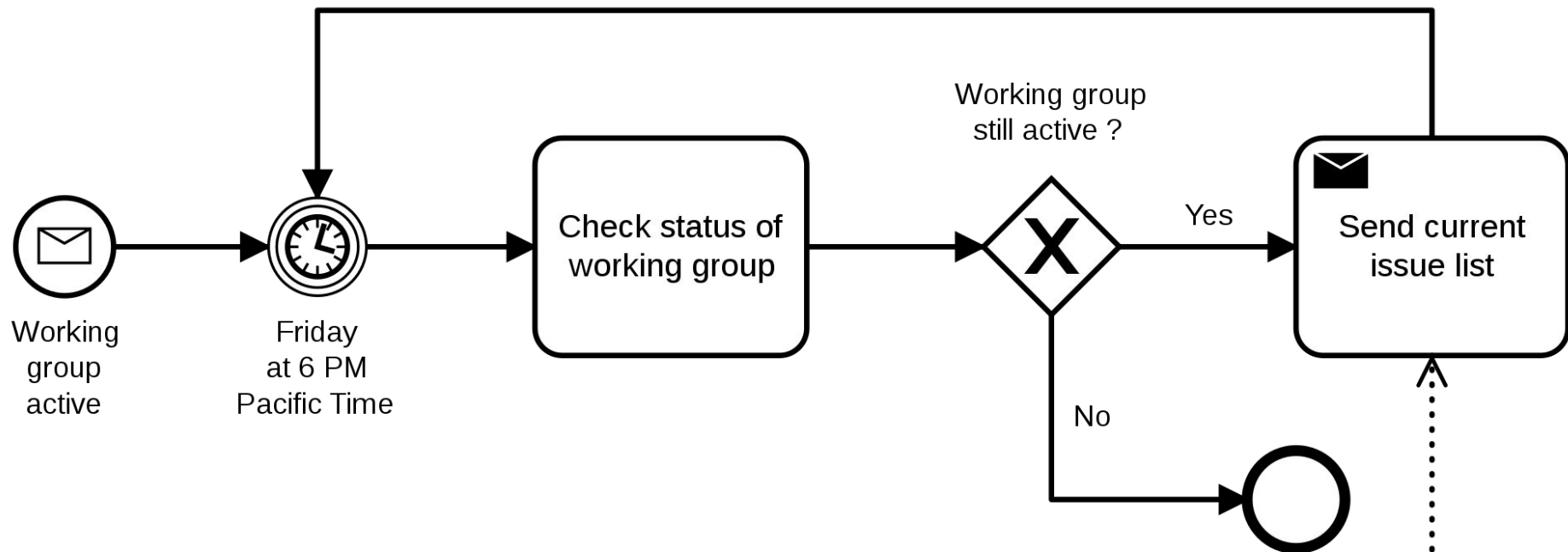
- Класс задач и предназначенного для их решения ПО.
- BPM vs BPMN.
- Позволяет автоматизировать разработку БЛ (и даже интерфейса!).
- Полезно даже в случае реализации БЛ “вручную”.

2. Моделирование бизнес-процессов

- Набор формальных подходов к описанию бизнес-процессов.
- Бизнес-процесс разбивается на состояния и переходы между ними.
- Существует ПО для визуальной отрисовки схем бизнес-процессов.
- Существует ПО для генерации кода по описанию бизнес-процесса.

- Система условных обозначений для моделирования бизнес-процессов.
- Транслируется в XML.
- Специфицирована Object Management Group, актуальная версия стандарта – 2.0.2 (2014 г.).
- Моделирование осуществляется путём составления диаграмм (моделей), состоящих из стандартизированных элементов.

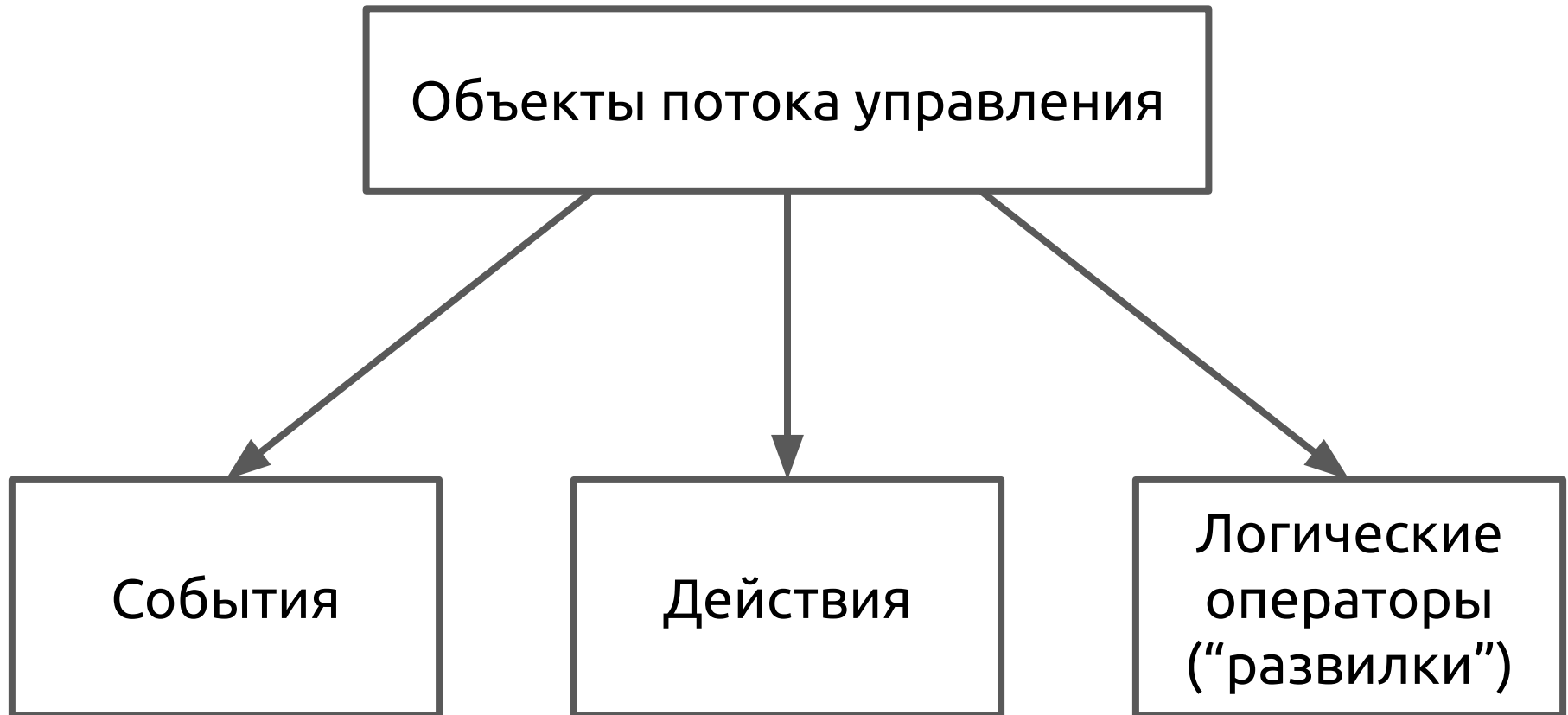
Пример диаграммы BPMN



http://www.bpmb.de/images/BPMN2_0_Poster_RU.pdf

- Объекты потока управления – управляют логикой реализации бизнес-процесса.
- Соединяющие объекты – связывают элементы модели бизнес-процесса между собой.
- Элементы swimlane – группируют элементы модели (например, по ролям).
- Артефакты – описывают объекты, порождаемые процессом.

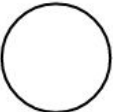
Объекты потока управления



- Изображаются окружностью.
- Означают какое-либо происшествие.
- Инициируют действия или являются их результатами.
- Могут быть классифицированы на:
 - начальные (start);
 - промежуточные (intermediate);
 - завершающие (end).
- Различают также события обработки и генерации.

Классификация событий (1)

События

События	Начальные			Промежуточные			Завершающие	
	Верхнеуровневые	Прерывающие событийный подпроцесс	Не прерывающие событийный подпроцесс	Обрабатывающие	Граничные прерывающие	Граничные непрерывающие		Генерирующие
Простое: нетипизированное событие, обычно показывающее начало или окончание процесса.								
Сообщение: получение и отправка сообщений.								
Таймер: циклические события, моменты времени, временные периоды и таймауты.								
Эскалация: перенос рассмотрения вопроса на более высокий уровень организационной иерархии								

Классификация событий (2)

События

События	Начальные			Промежуточные			Завершающие
	Верхнеуровневые	Прерывающие событийный подпроцесс	Не прерывающие событийный подпроцесс	Обрабатывающие	Граничные прерывающие	Граничные непрерывающие	Генерирующие
Условное: реакция на изменение бизнес-условий или интеграция бизнес-правил.							
Ссылка: пара соответствующих ссылок эквивалента потоку последовательности.							
Ошибка: генерация и обработка заданного типа ошибок.							
Отмена: обработка отмены транзакции или инициирование отмены.							

Классификация событий (3)

События

	Начальные			Промежуточные				Завершающие
	Верхнеуровневые	Прерывающие событийный подпроцесс	Не прерывающие событийный подпроцесс	Обрабатывающие	Граничные прерывающие	Граничные непрерывающие	Генерирующие	
Компенсация: обработка или инициирование компенсации.								
Сигнал: передается между процессами и может обрабатываться многими получателями.								
Составное: обработка одного события из множества или генерация всех определенных событий.								
Параллельное составное: обработка всего множества параллельных событий.								
Останов: вызывает немедленное прекращение выполнения процесса.								

- Изображаются прямоугольниками со скругленными углами.
- Делятся на две категории:
 - задания;
 - подпроцессы.
- Могут содержать маркеры.

Категории действий

Задача

Задача - единица работы. Если задача помечена символом, , то задача является подпроцессом и может быть детализирована.

Транзакция

Транзакция - набор логически связанных действий. Для транзакции может быть определен протокол выполнения.

Событийный
подпроцесс

Событийный подпроцесс помещается внутри другого процесса. Он начинает выполняться, если инициируется его начальное событие. Событийный подпроцесс может прерывать родительский подпроцесс или выполняться параллельно с ним.

Вызывающее
действие

Вызывающее действие является точкой входа для глобально определенного подпроцесса, который повторно используется в данном процессе.

Маркеры действий и типы задач

Маркеры действий

Маркер отражает поведение действия во время выполнения:



Маркер подпроцесса



Маркер цикла



Маркер параллельных множественных экземпляров (МЭ)



Маркер последовательных множественных экземпляров (МЭ)



Маркер ad hoc



Маркер компенсации

Типы задач

Тип определяет природу действия, которое будет выполнено:



Задача отправки сообщения



Задача получения сообщения



Пользовательская задача



Неавтоматизированная задача



Задача-бизнес-правило



Задача-сервис



Задача-сценарий

Логические операторы (“развилки”)

- Изображаются ромбами.
- Представляют собой точки принятия решений в процессе.
- Позволяют организовать ветвление и синхронизацию потоков управления.

Логические операторы (1)

Оператор исключающего ИЛИ, управляемый данными



При ветвлении направляет поток лишь по одной из исходящих ветвей. При синхронизации потоков оператор ожидает завершения одной входящей ветви и активирует исходящий поток управления.

Оператор исключающего ИЛИ, событийный



Предшествует только событиям обработки или заданиям-обработчикам сообщений. Поток управления направляется по той ветви, где событие произошло раньше.

Оператор И



При разделении на параллельные потоки все ветви активируются одновременно. При синхронизации параллельных ветвей оператор ждет завершения всех входящих ветвей и затем активирует исходящий поток.

Логические операторы (2)



Оператор ИЛИ

При ветвлении активируется одна или более ветвей. При слиянии все выполняющиеся входящие ветви должны быть завершены.



Оператор исключающего ИЛИ, событийный (создает новый экземпляр)

Наступление каждого из последующих событий создает экземпляр процесса.



Сложный оператор

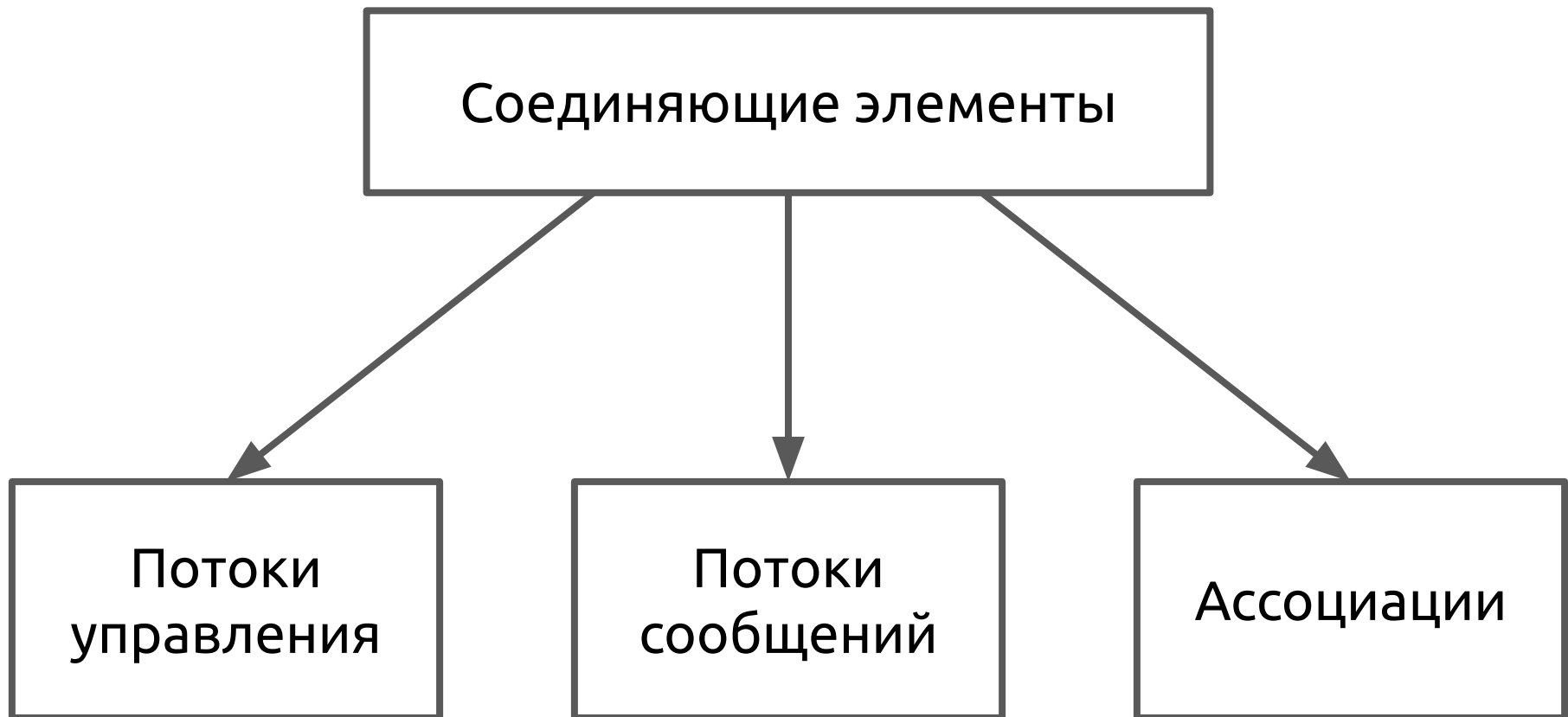
Моделирует сложные условия ветвления и слияния.



Оператор И, событийный (создает новый экземпляр)

Наступление всех последующих событий создает экземпляр процесса.

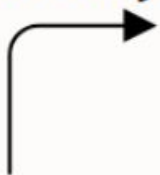
Соединяющие элементы



Потоки управления

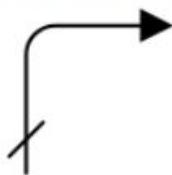
- Определяет переходы между состояниями бизнес-процесса.
- В BPMN 2.0 реализовано 3 вида потоков управления:

Поток управления



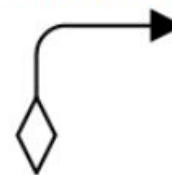
определяет порядок выполнения действий.

Поток по умолчанию



определяет ветвь процесса, выполняемую, когда все условия ветвления не выполнены.

Условный поток



связан с условием, определяющим будет ли выполнен данный поток.

Потоки сообщений

- Изображается штриховой линией, оканчивающейся открытой стрелкой.
- Показывает, какими сообщениями обмениваются участники.
- Может присоединяться к пулам, действиям или событиям-сообщениям.

○ — — — ▷ Поток сообщений

Ассоциации

- Изображаются пунктирной линией.
- В BPMN 2.0 реализованы два вида ассоциаций:
 - Простая ассоциация – связывает артефакты с другими элементами диаграммы.
 - Ассоциация данных (направленная ассоциация) – помимо связи, показывает направление движения потока информации:
 - Однонаправленные.
 - Двухнаправленные.

----->
Data association

Association

Роли, пулы и дорожки (swimlane)

- Роли — позволяют категоризировать действия со сходной функциональностью
- В BPMN 2.0 есть два типа ролей:
 - Пулы – изображаются прямоугольником, который содержит несколько объектов потока управления, соединяющих объектов и артефактов.
 - Дорожки – представляют собой часть пула, позволяют организовать объекты потока управления, связывающие объекты и артефакты.

Роли, пулы и дорожки (продолжение)

Пул	Дорожка	Дорожка	
		Дорожка	
	Дорожка		

Свёрнутый пул

- Позволяют отображать дополнительную информацию в диаграмме.
- Делают диаграмму более удобочитаемой и насыщенной информацией.
- В BPMN 2.0 реализованы 3 вида артефактов:
 - данные;
 - группы;
 - текстовые аннотации.

Показывают:

- Какие данные необходимы действиям для выполнения.
- Какие данные эти действия производят.

Типы данных



Входные данные - внешний вход для процесса. Действия могут использовать эти данные.

Выходные данные - результат выполнения процесса.



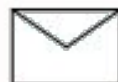
Объект данных представляет информацию, которая обрабатывается в ходе процесса, например, документ или письмо.



Коллекция объектов данных представляет группу объектов, несущих информацию, например, список заказанных товаров.



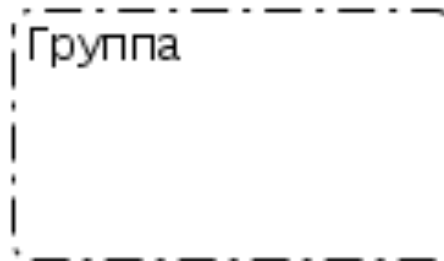
Хранилище данных - объект, который процесс может использовать для записи и извлечения данных, например, база данных. Хранилище данных позволяет хранить данные после окончания жизненного цикла экземпляра процесса.



Сообщение позволяет явно показать передачу информации в ходе общения двух участников.

Группы

- Изображаются прямоугольниками с закругленными углами, граница которых — штрихпунктирная линия.
- Позволяют объединять различные действия, но не влияют на поток управления в диаграмме.

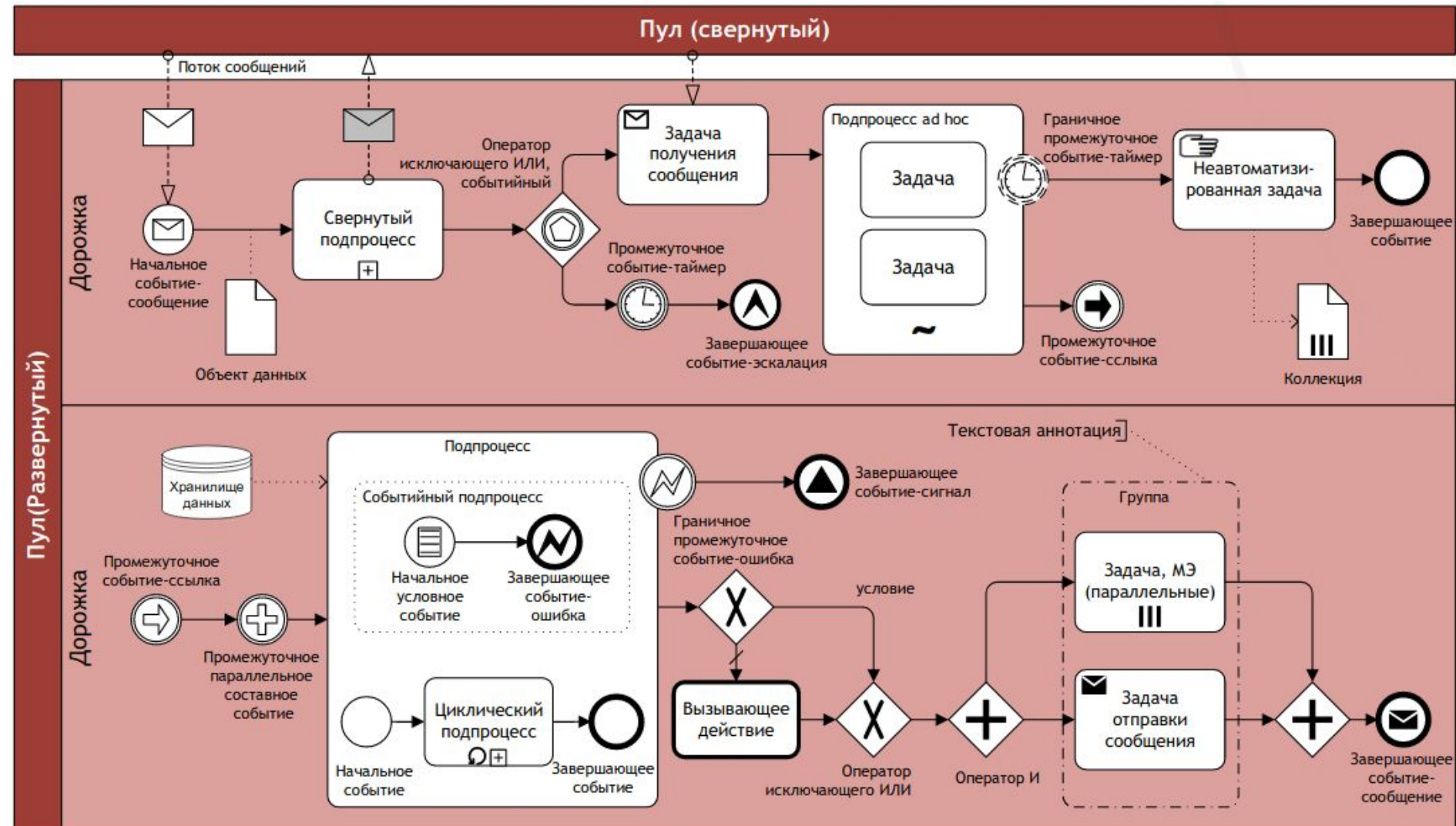


Текстовые аннотации

- Используются для уточнения значения элементов диаграммы и повышения её информативности.

Текстовая аннотация

Элементы на диаграмме



Диалоги (1)



Информационное взаимодействие задает цепочку логически связанных обменов сообщениями. Если информационное взаимодействие помечено символом $\boxed{+}$, то оно может быть детализировано.

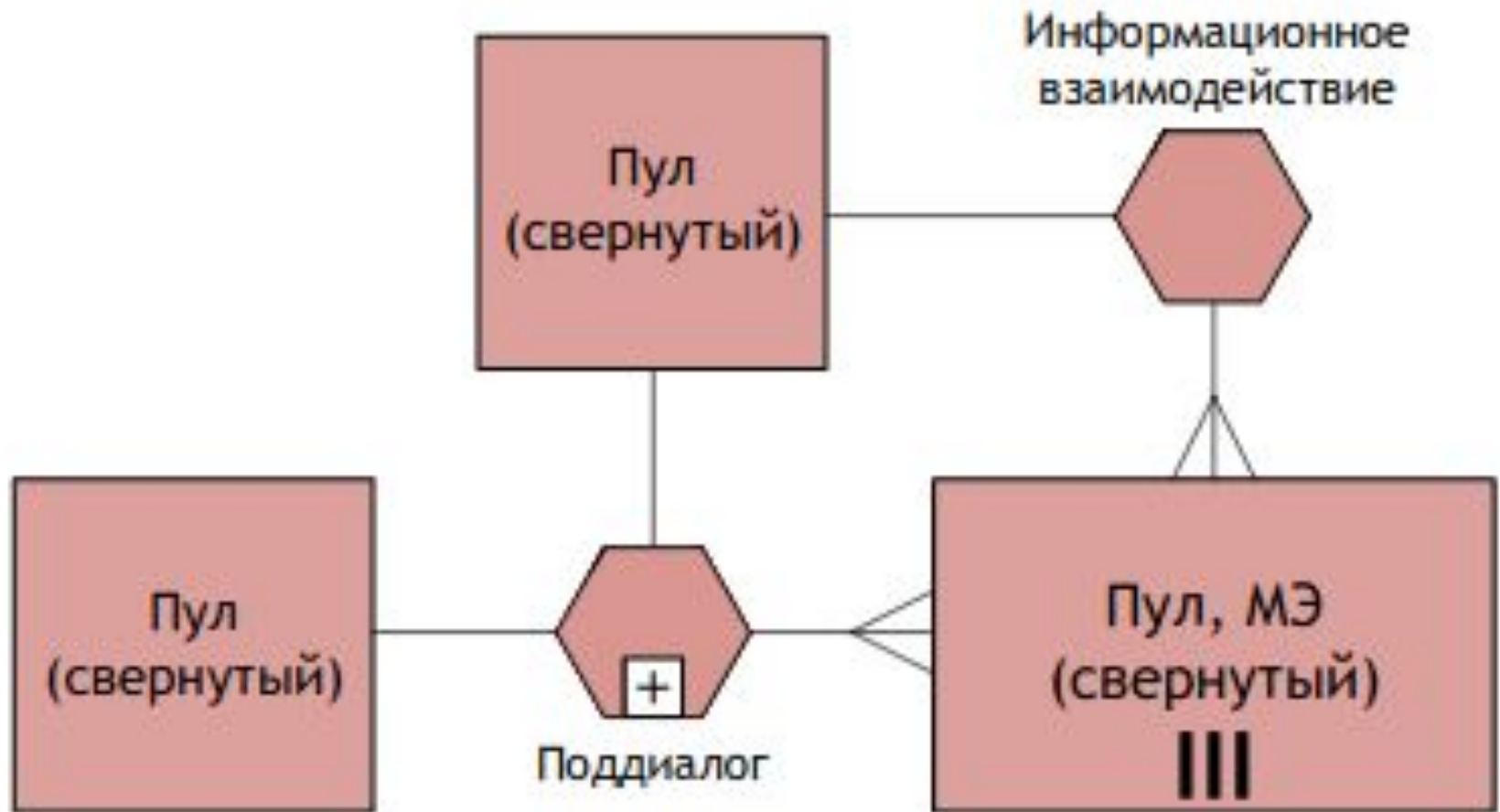


Связь соединяет информационное взаимодействие с одним участником



Разветвляющаяся связь соединяет информационное взаимодействие с несколькими участниками

Диалоги (2)



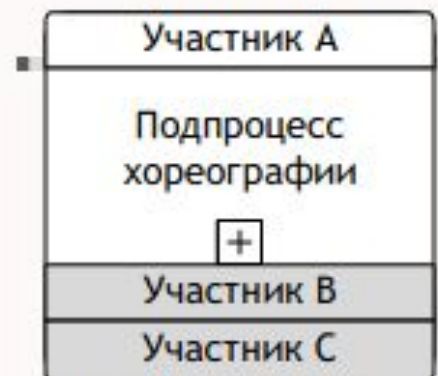
Хореографии (1)



Задача хореографии отражает взаимодействие (обмен сообщениями) между двумя участниками.

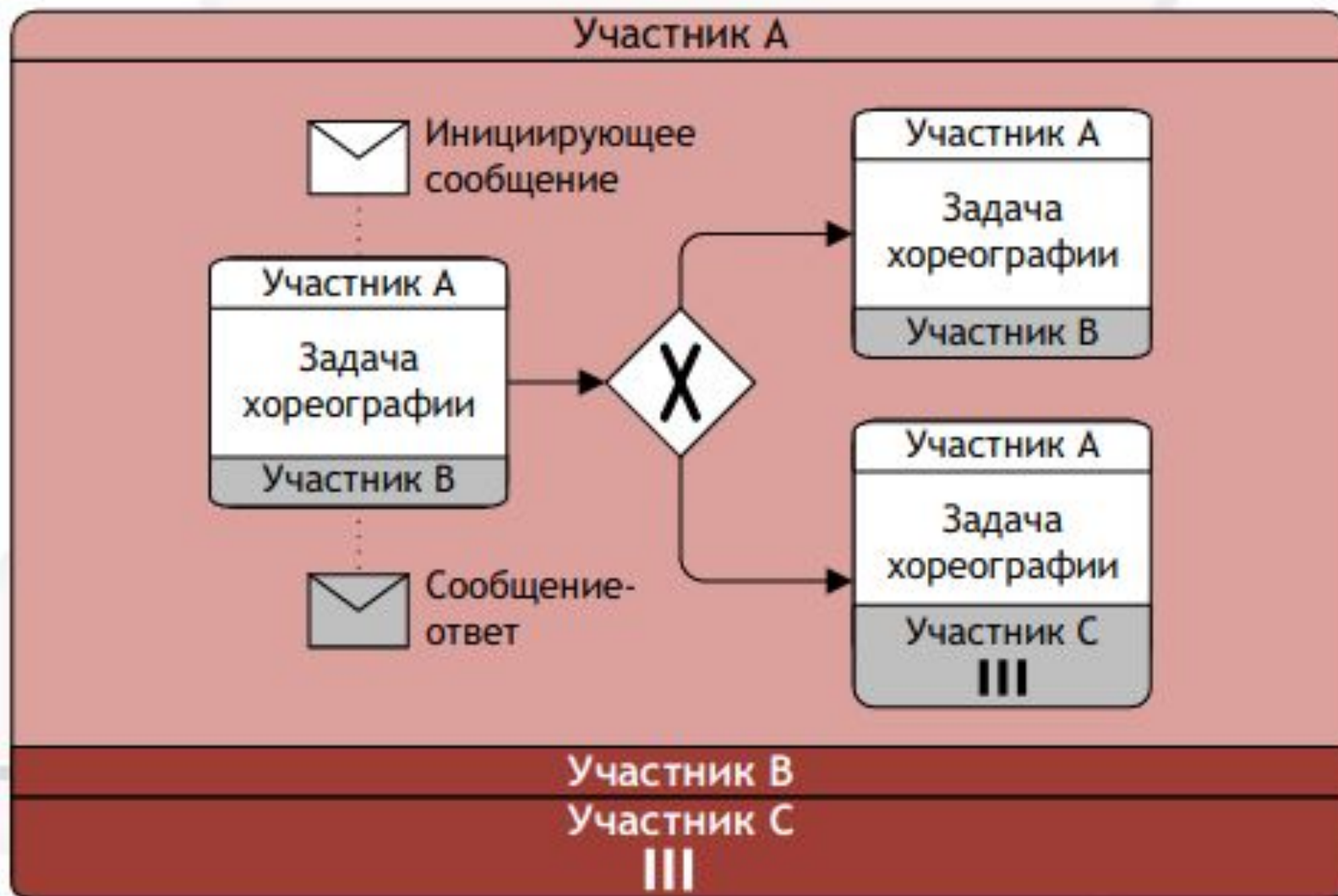
III

Маркер множественных участников показывает, что несколько участников одного типа участвуют в хореографии.



Подпроцесс хореографии содержит детализацию хореографии, включающую несколько взаимодействий.

Хореографии (2)

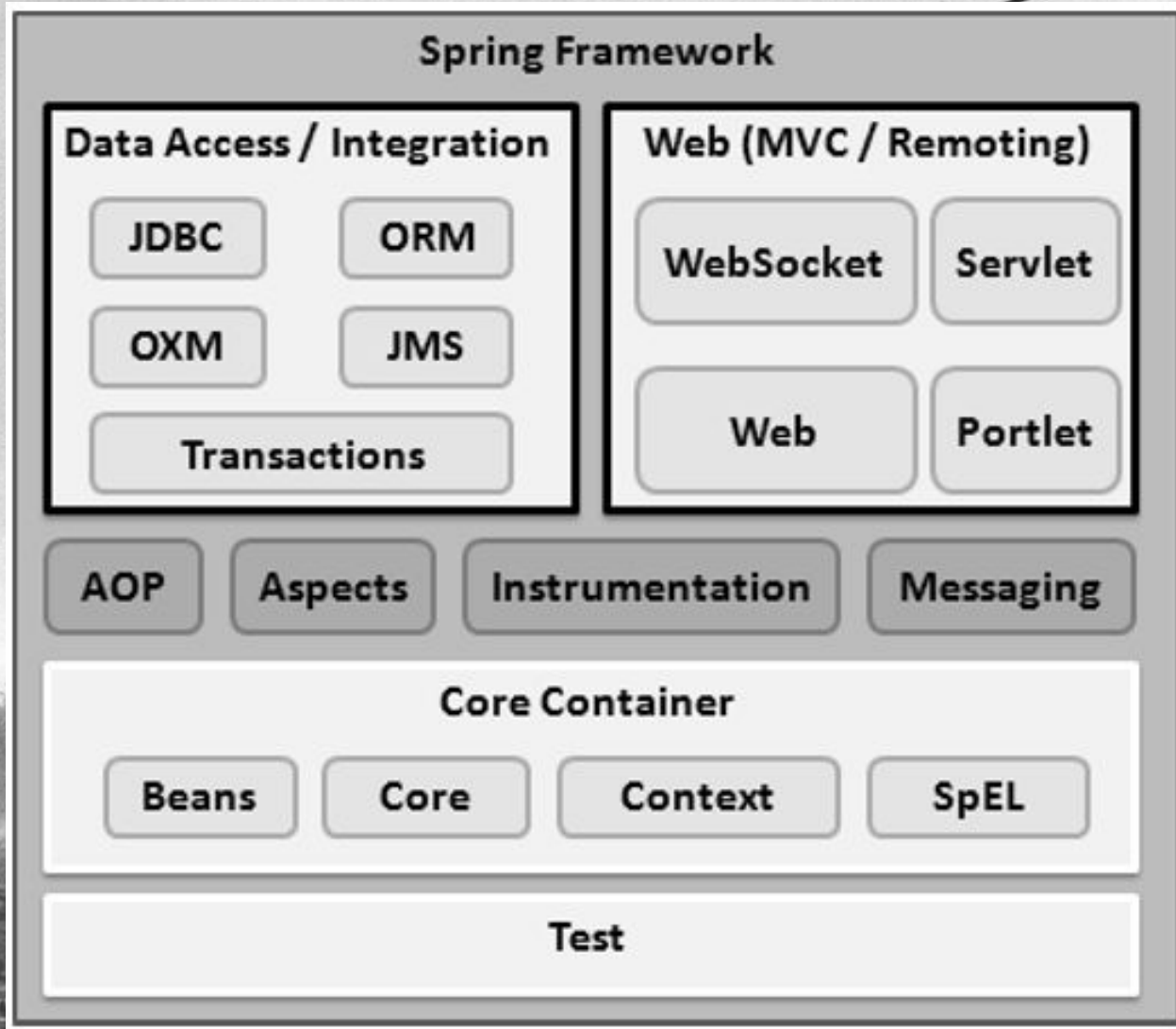


3. Spring и бизнес-логика

- Универсальный фреймворк для разработки приложений на Java.
- Open Source (MIT); первая версия вышла в 2002 г.
- “Фреймворк фреймворков”.
- Каждый из фреймворков может быть использован по-отдельности от остальных.

- CDI и IoC-контейнер.
- Управление транзакциями.
- Удалённый доступ.
- Аутентификация, авторизация, политики безопасности.
- Работа с сообщениями.

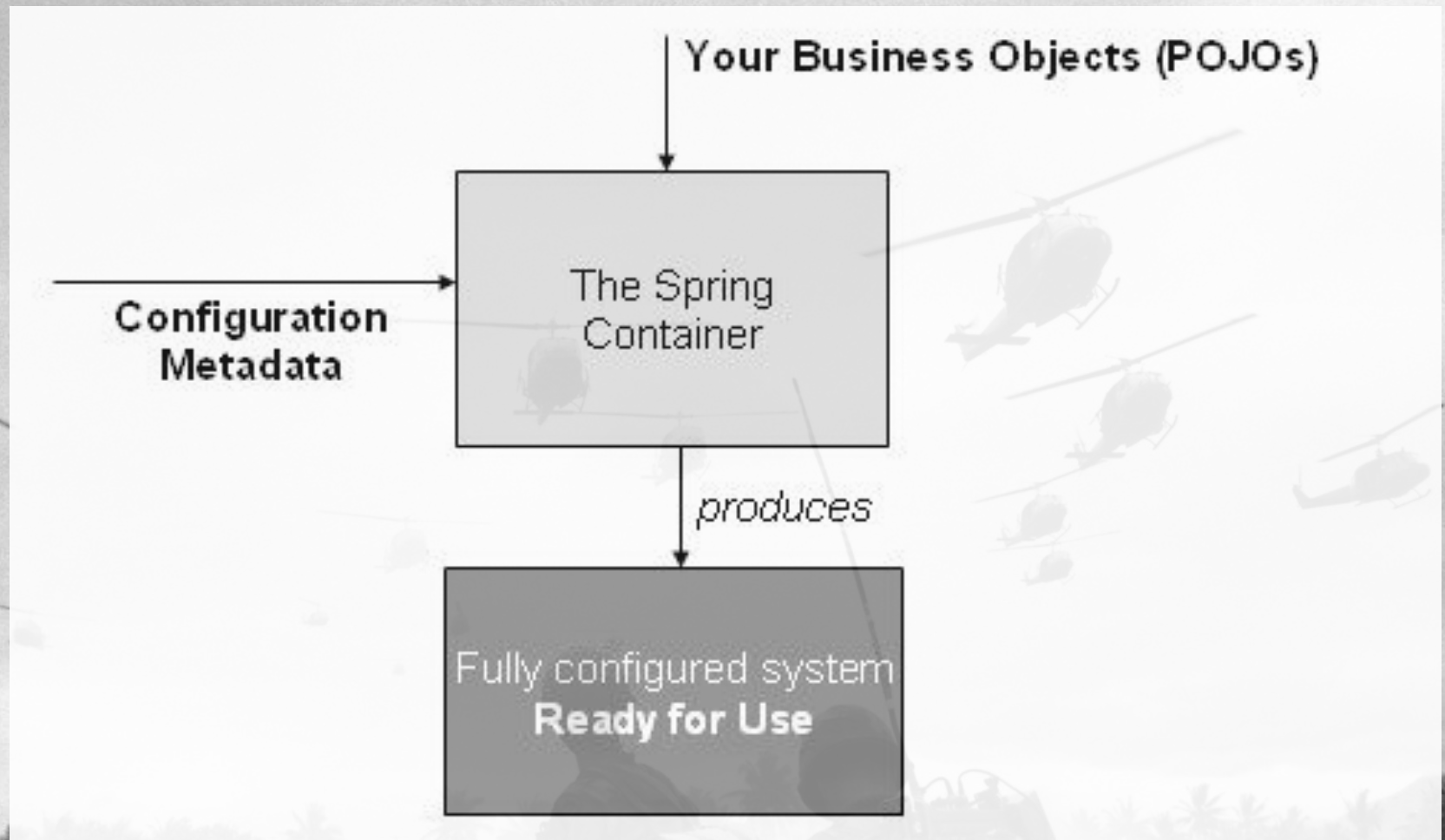
Архитектура Spring



Spring Core Container

- Состоит из следующих модулей:
 - Core и Beans – основы фреймворка; в частности, реализация IoC и CDI.
 - Context – механизмы доступа к объектам (а-ля JNDI).
 - Expression Language (SpEL) – механизмы манипулирования элементами объектного графа в рантайме.

IoC в Spring



Пример DI в Spring

```
public class Store {  
    @Autowired  
    @Qualifier("item1")  
    private Item item;  
}
```

Spring Context

- Работает на основе Core & Beans.
- Добавляет следующие возможности:
 - i18n (с использованием бандлов ресурсов).
 - Обработка и передача событий.
 - Загрузка ресурсов.
 - “Прозрачное” создание контекстов (например, веб-контейнером).
- Позволяет интегрироваться с “внешними” фреймворками (в частности, EJB, JMX, RMI).
- Основной API – интерфейс `ApplicationContext`.

...см. раздел 4

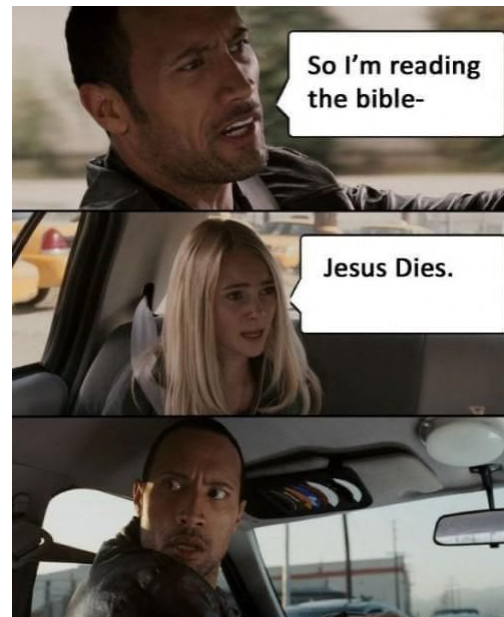


Spring Expression Language

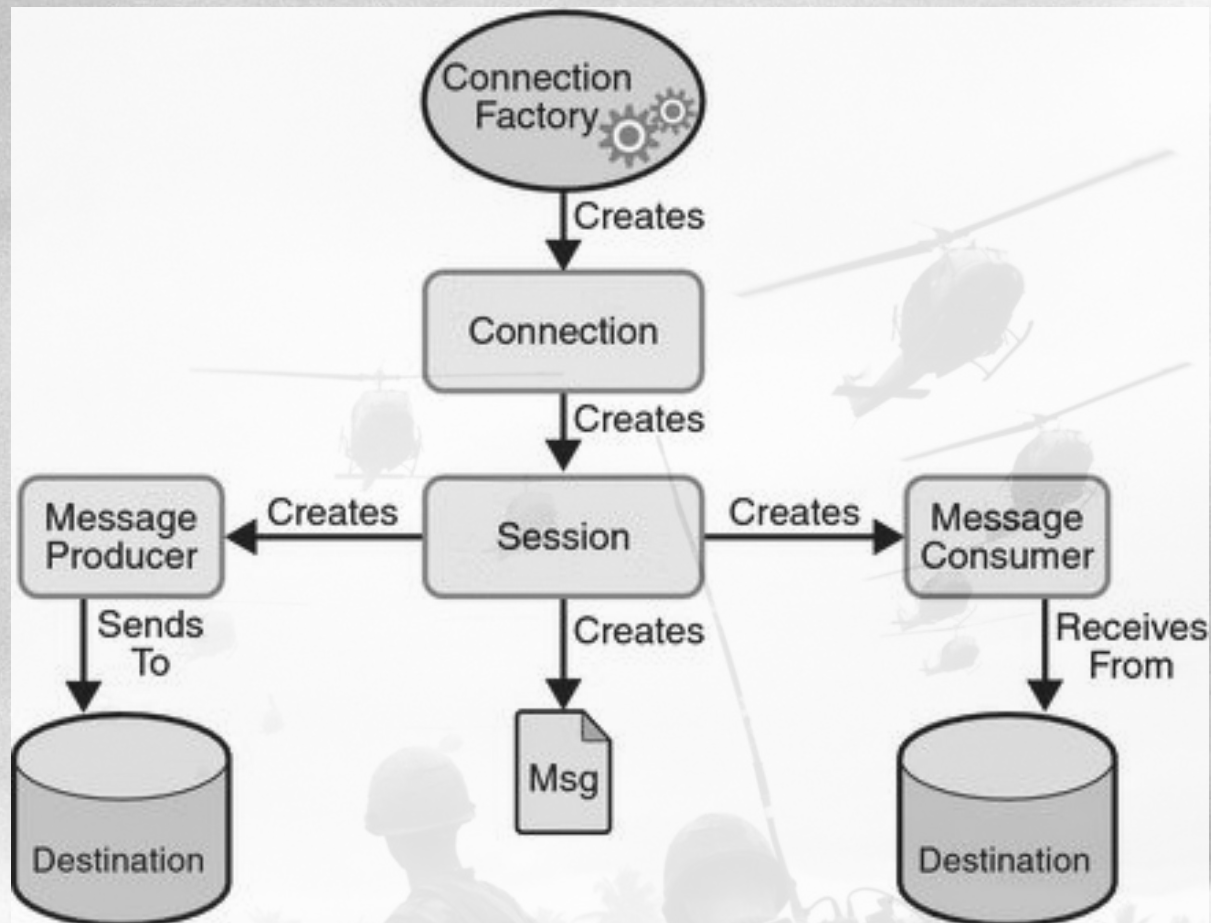
- Является расширением EL, используемого в JSP / JSF.
- Ключевые возможности:
 - Работа со свойствами (чтение, запись).
 - Вызов методов.
 - Работа с массивами, коллекциями и индексами.
 - Поддержка арифметических и логических операций.
 - Получение объектов по имени из контейнера Spring IoC.
 - Работа со списками – проекция, выборка и агрегация.

Инструментирование в Spring

- Инструментирование – возможность “встроиться” в код приложения в рантайме.
- Типичные решаемые задачи – трассировка кода и мониторинг производительности.
- В Spring реализуется средствами JVM (например, JMX) и с помощью АОП (снова спойлеры).

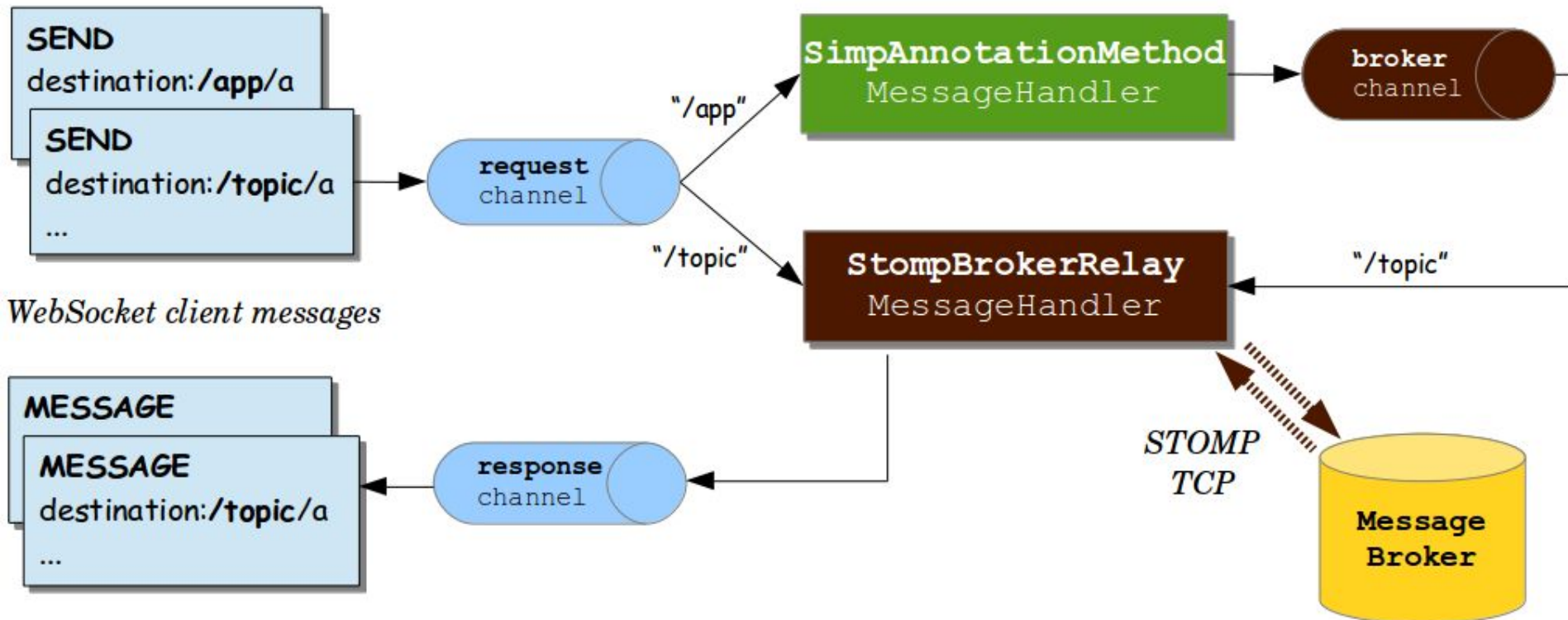


Асинхронный обмен сообщениями



Асинхронный обмен сообщениями в Spring

- Spring поддерживает JMS – для этого есть специальные аннотации `@EnableJms` и `@JmsListener`.



- Набор утилит для автоматизации процессов настройки, создания и развёртывания приложения.
- Избавляет от необходимости повторять рутинные действия.
- Позволяет более эффективно управлять зависимостями и упростить конфигурацию приложения.

Spring Boot: пример приложения

```
package com.example.springboot;

import java.util.Arrays;

import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class Application {

    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }

    @Bean
    public CommandLineRunner commandLineRunner(ApplicationContext ctx) {
        return args -> {

            System.out.println("Let's inspect the beans provided by Spring Boot:");

            String[] beanNames = ctx.getBeanDefinitionNames();
            Arrays.sort(beanNames);
            for (String beanName : beanNames) {
                System.out.println(beanName);
            }

        };
    }
}
```

4. Аспектно-ориентированное программирование

Аспектно-ориентированное программирование

- АОП – парадигма программирования.
- Является развитием принципов процедурного программирования и ООП.
- Суть – внедрение в код “сквозной” функциональности – *аспекта*.
- “Каноничная” реализация в Java – AspectJ, она же используется в Spring.

Выделение “сквозной” функциональности

Было:

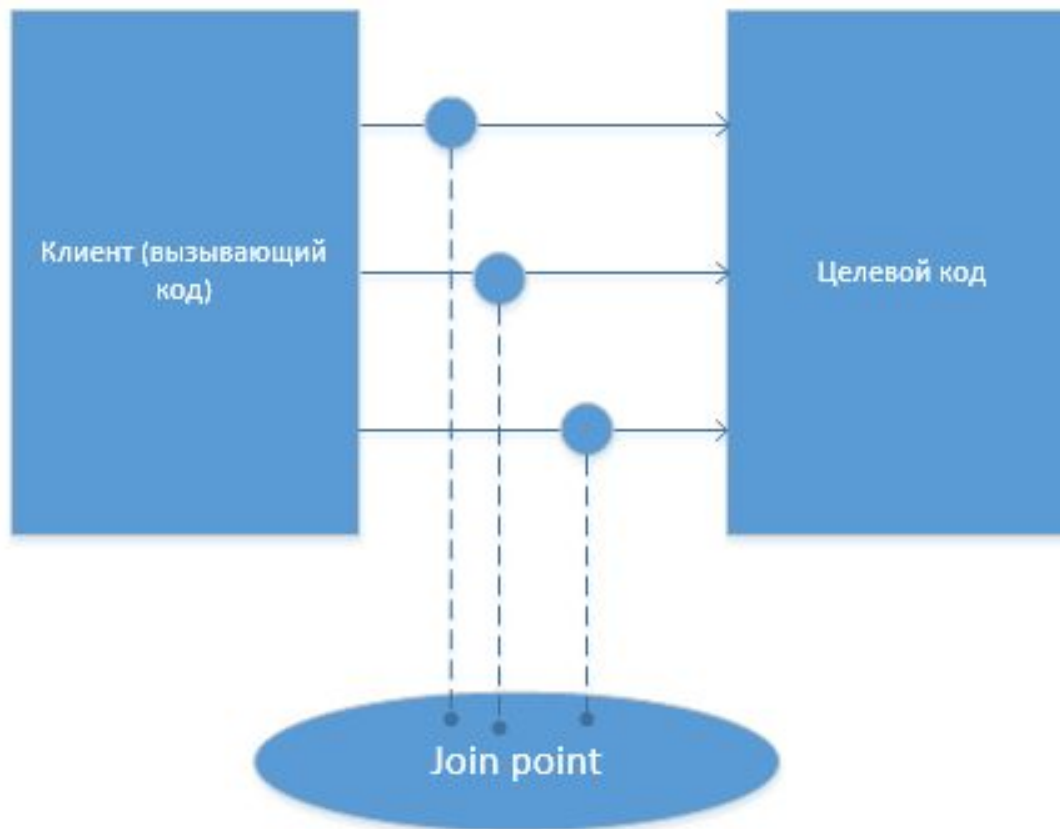


Стало:



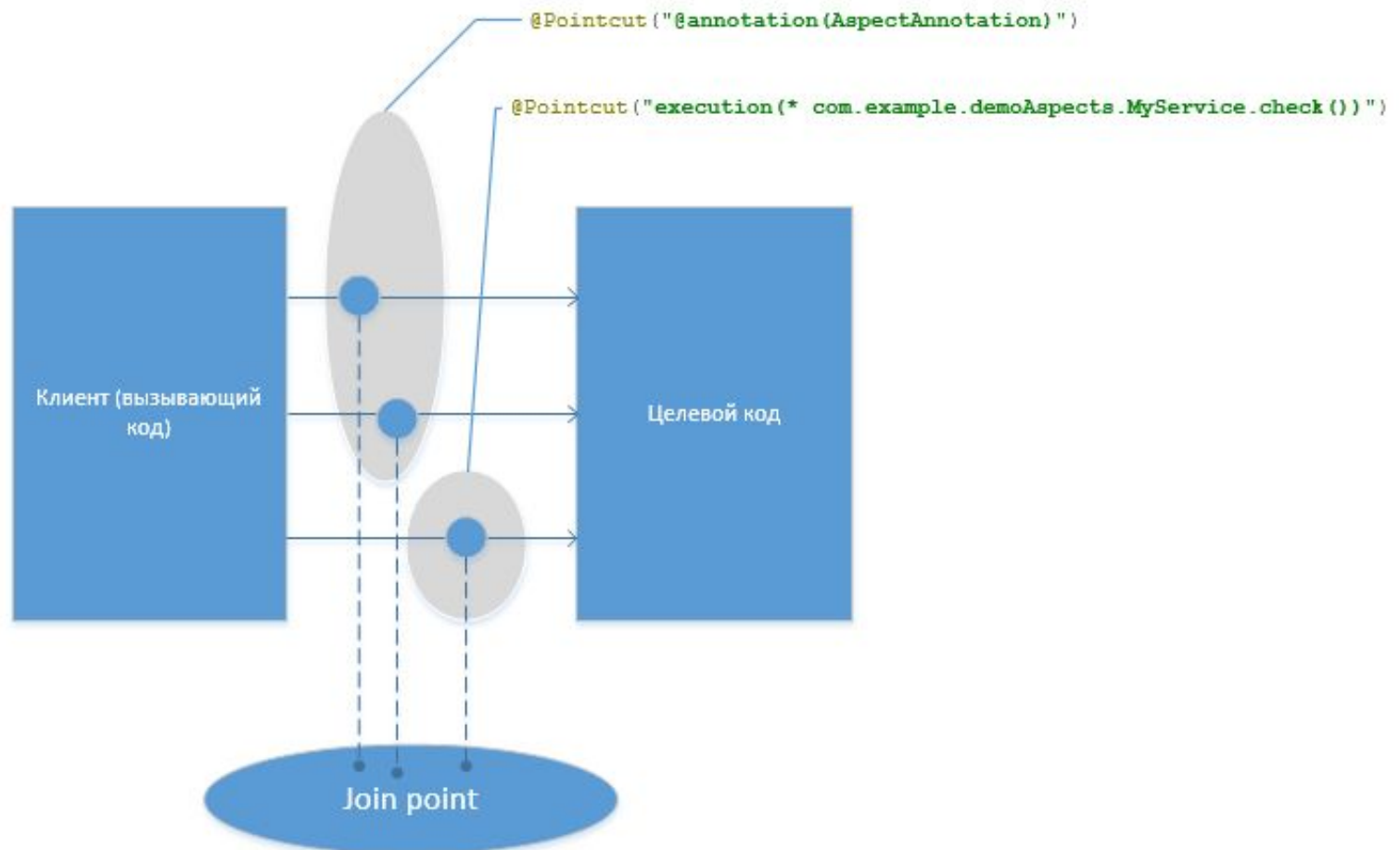
AspectJ: Join Point

“Точка подключения” к коду, в которой к нему добавляется “сквозная” функциональность.



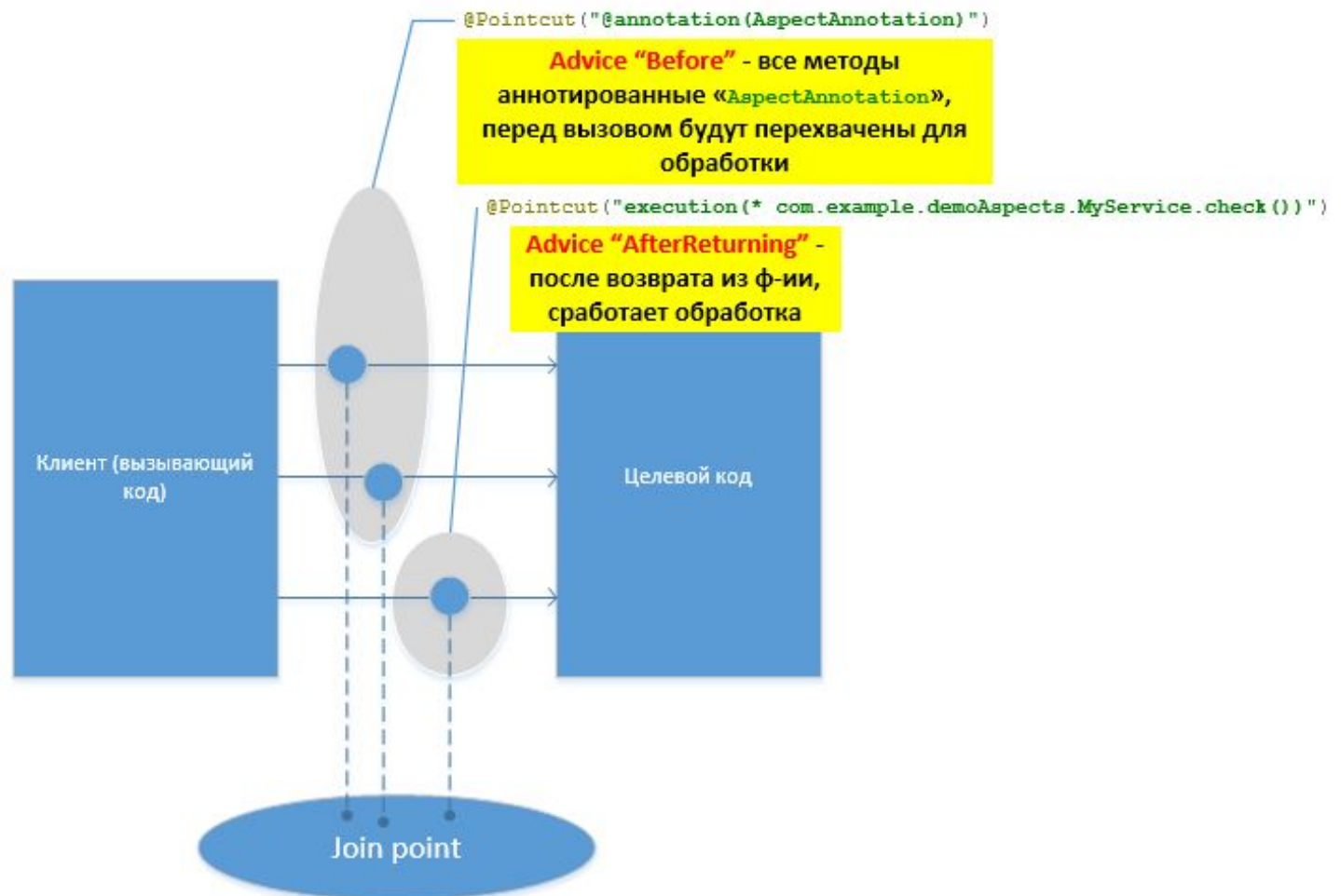
AspectJ: Pointcut

Именованный “срез” точек подключения.



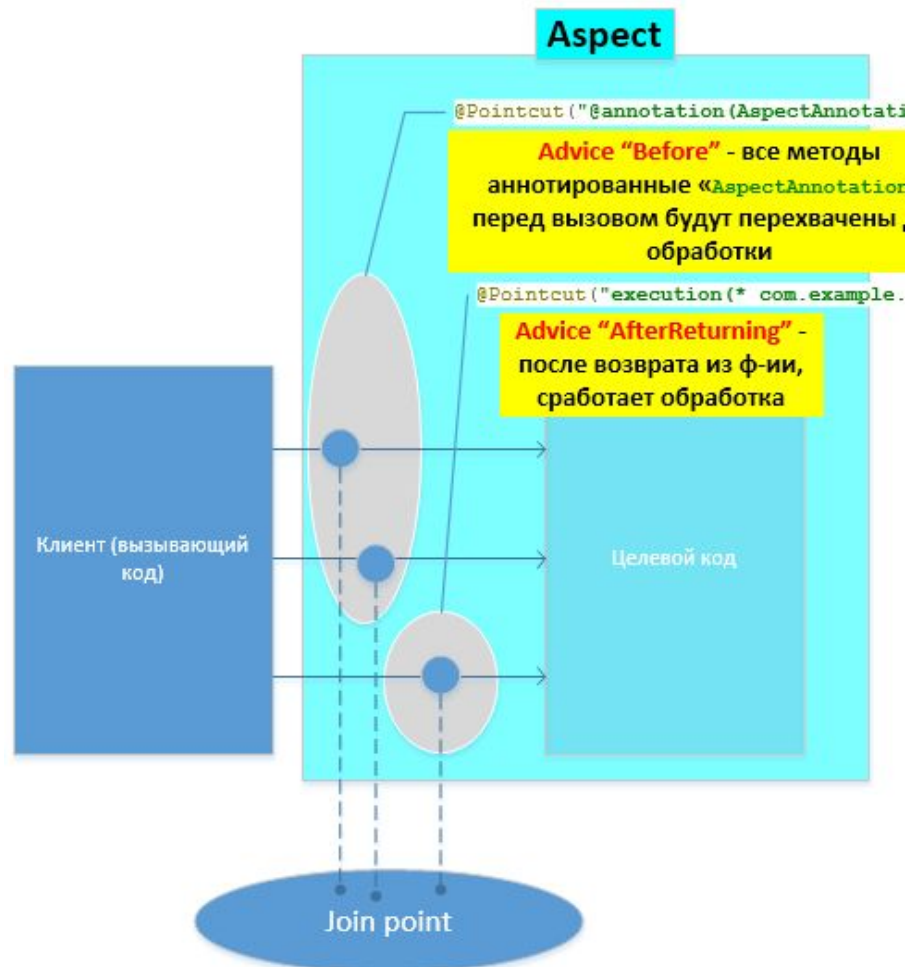
AspectJ: Advice

Набор инструкций, выполняемых на pointcut'ах.



AspectJ: Aspect

Модуль, в котором собраны описания advice'ов и pointcut'ов.



Пример AspectJ: сервис

```
@Service
public class MyService {

    public void method1(List<String> list) {
        list.add("method1");
        System.out.println("MyService method1 list.size=" + list.size());
    }

    @AspectAnnotation
    public void method2() {
        System.out.println("MyService method2");
    }

    public boolean check() {
        System.out.println("MyService check");
        return true;
    }
}
```

Пример AspectJ: аспект

```
@Aspect
@Component
public class MyAspect {

    private Logger logger = LoggerFactory.getLogger(this.getClass());

    @Pointcut("execution(public * com.example.demoAspects.MyService.*(..))")
    public void callAtMyServicePublic() { }

    @Before("callAtMyServicePublic()")
    public void beforeCallAtMethod1(JoinPoint jp) {
        String args = Arrays.stream(jp.getArgs())
            .map(a -> a.toString())
            .collect(Collectors.joining(", "));
        logger.info("before " + jp.toString() + ", args=[" + args + "]);
    }

    @After("callAtMyServicePublic()")
    public void afterCallAt(JoinPoint jp) {
        logger.info("after " + jp.toString());
    }
}
```

Пример AspectJ: тестовый код

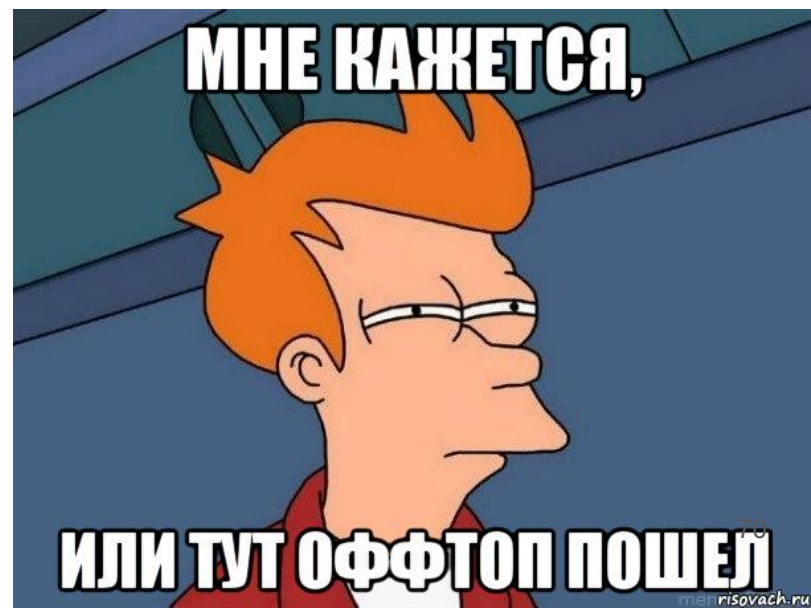
```
@RunWith(SpringRunner.class)
@SpringBootTest
public class DemoAspectsApplicationTests {

    @Autowired
    private MyService service;

    @Test
    public void testLoggable() {
        List<String> list = new ArrayList();
        list.add("test");

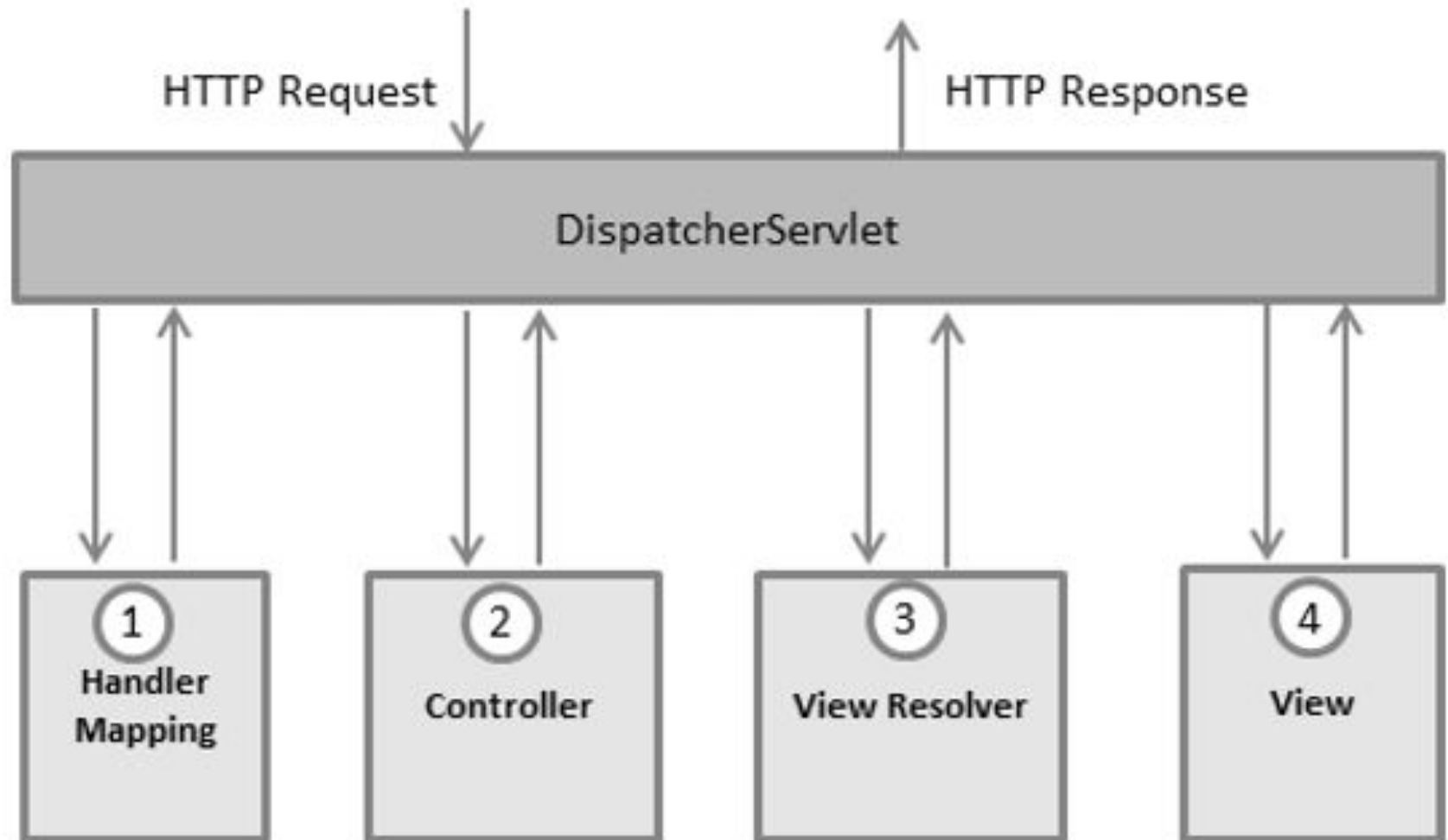
        service.method1(list);
        service.method2();
        Assert.assertTrue(service.check());
    }
}
```

5. Spring Web MVC



- Фреймворк для разработки веб-приложений в составе Spring.
- Основан на сервлетах.
- Представление (View) может быть реализовано на множестве технологий – JSP, Thymeleaf, etc.

Архитектура Spring Web MVC

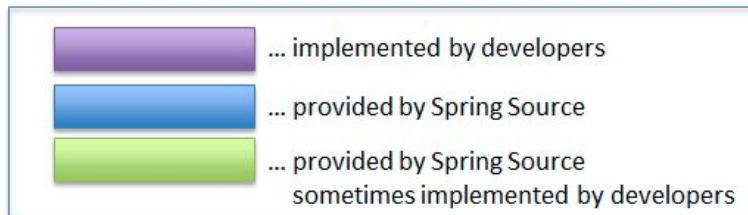
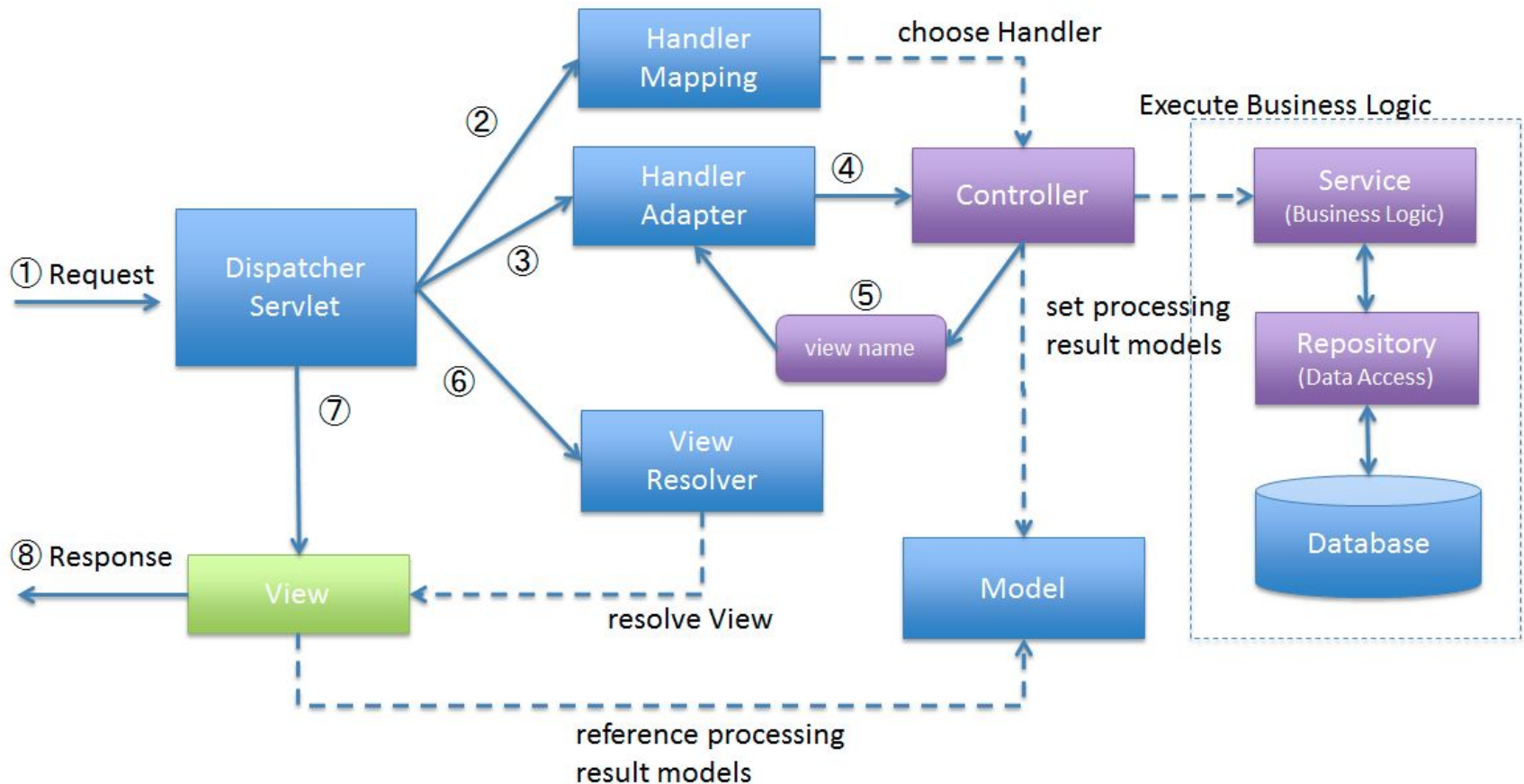


- **Model** – инкапсулирует данные приложения (состоят из POJO или бинов).
- **View** – отвечает за отображение данных модели.
- **Controller** – обрабатывает запрос пользователя, создаёт соответствующую модель и передаёт её для отображения в представление.

Dispatcher Servlet

- Обработывает все запросы и формирует ответы на них.
- Связывает между собой все элементы архитектуры Spring MVC.
- Обычный сервлет – конфигурируется в `web.xml`.

Обработка запроса



- Универсальный движок шаблонизации.
- Написан на Java.
- Может использоваться как на уровне веб-контейнера, так в standalone-приложениях.
- Часто используется в решениях на Spring Web MVC вместо JSP.

Поддерживается 6 типов:

- HTML.
- XML.
- TEXT.
- JAVASCRIPT.
- CSS.
- RAW.

Пример HTML-шаблона

```
1 <table>
2   <thead>
3     <tr>
4       <th th:text="#{msgs.headers.name}">Name</th>
5       <th th:text="#{msgs.headers.price}">Price</th>
6     </tr>
7   </thead>
8   <tbody>
9     <tr th:each="prod: ${allProducts}">
10      <td th:text="${prod.name}">Oranges</td>
11      <td th:text="${#numbers.formatDecimal(prod.price, 1, 2)}">0.99</td>
12    </tr>
13  </tbody>
14 </table>
```

Проще всего использовать Spring Boot:

- Делаем обычное приложение на Spring Web MVC.
- Подключаем зависимость `spring-boot-starter-thymeleaf`.
- Размещаем шаблоны в `src/main/resources/templates`.

Пример: конфигурация проекта

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>2.4.0</version>
</parent>
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
  </dependency>
</dependencies>
```

Пример: сущность и репозиторий

```
@Entity
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private long id;

    @NotBlank(message = "Name is mandatory")
    private String name;

    @NotBlank(message = "Email is mandatory")
    private String email;

    // standard constructors / setters / getters / toString
}
```

```
@Repository
public interface UserRepository extends CrudRepository<User, Long> {}
```

Пример: контроллер

```
@Controller
public class UserController {

    @GetMapping("/signup")
    public String showSignUpForm(User user) {
        return "add-user";
    }

    @PostMapping("/adduser")
    public String addUser(@Valid User user, BindingResult result, Model model) {
        if (result.hasErrors()) {
            return "add-user";
        }

        userRepository.save(user);
        return "redirect:/index";
    }

    // additional CRUD methods
}
```

Пример: шаблон UI

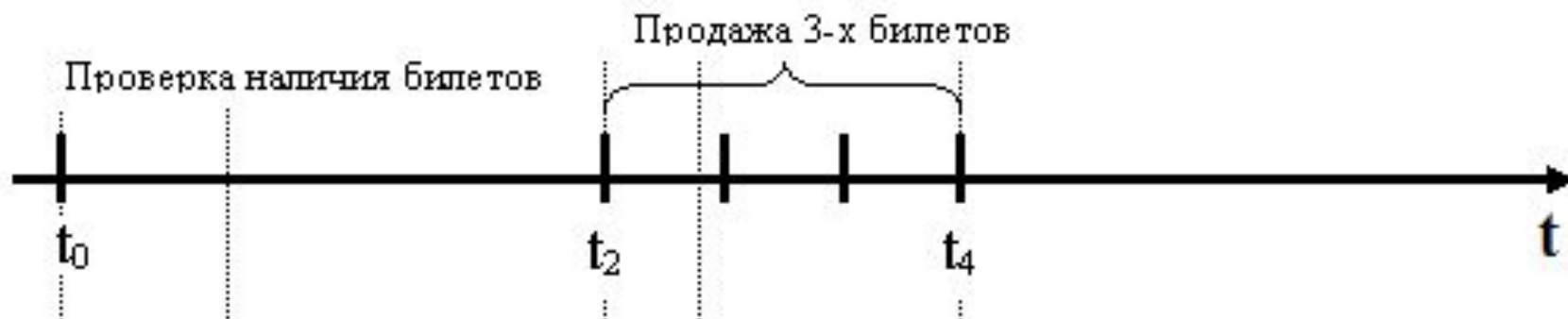
```
<form action="#" th:action="@{/adduser}" th:object="${user}" method="post">
  <label for="name">Name</label>
  <input type="text" th:field="*{name}" id="name" placeholder="Name">
  <span th:if="${#fields.hasErrors('name')}" th:errors="*{name}"></span>
  <label for="email">Email</label>
  <input type="text" th:field="*{email}" id="email" placeholder="Email">
  <span th:if="${#fields.hasErrors('email')}" th:errors="*{email}"></span>
  <input type="submit" value="Add User">
</form>
```

6. Управление транзакциями

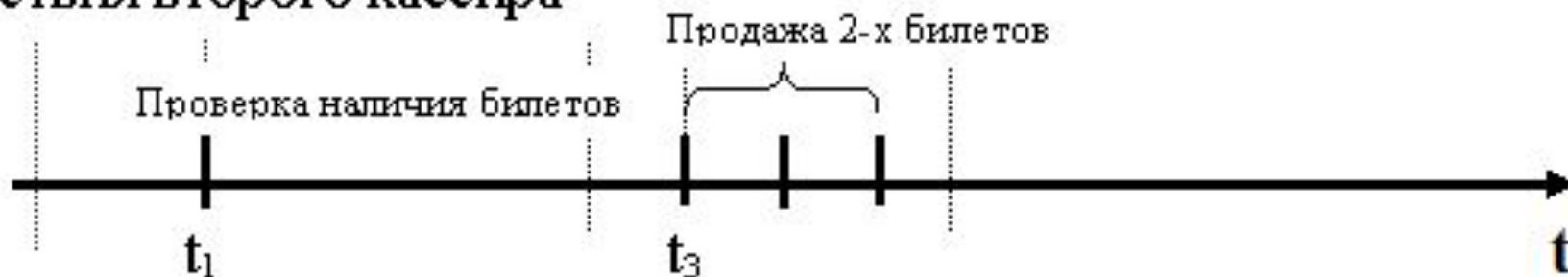
- Транзакция – группа последовательных операций, представляющая собой логическую единицу работы с данными.
- Транзакции бывают:
 - Последовательные (обычные).
 - Параллельные (в многопользовательских системах).
 - Распределённые (в распределённых системах).

Параллельные транзакции

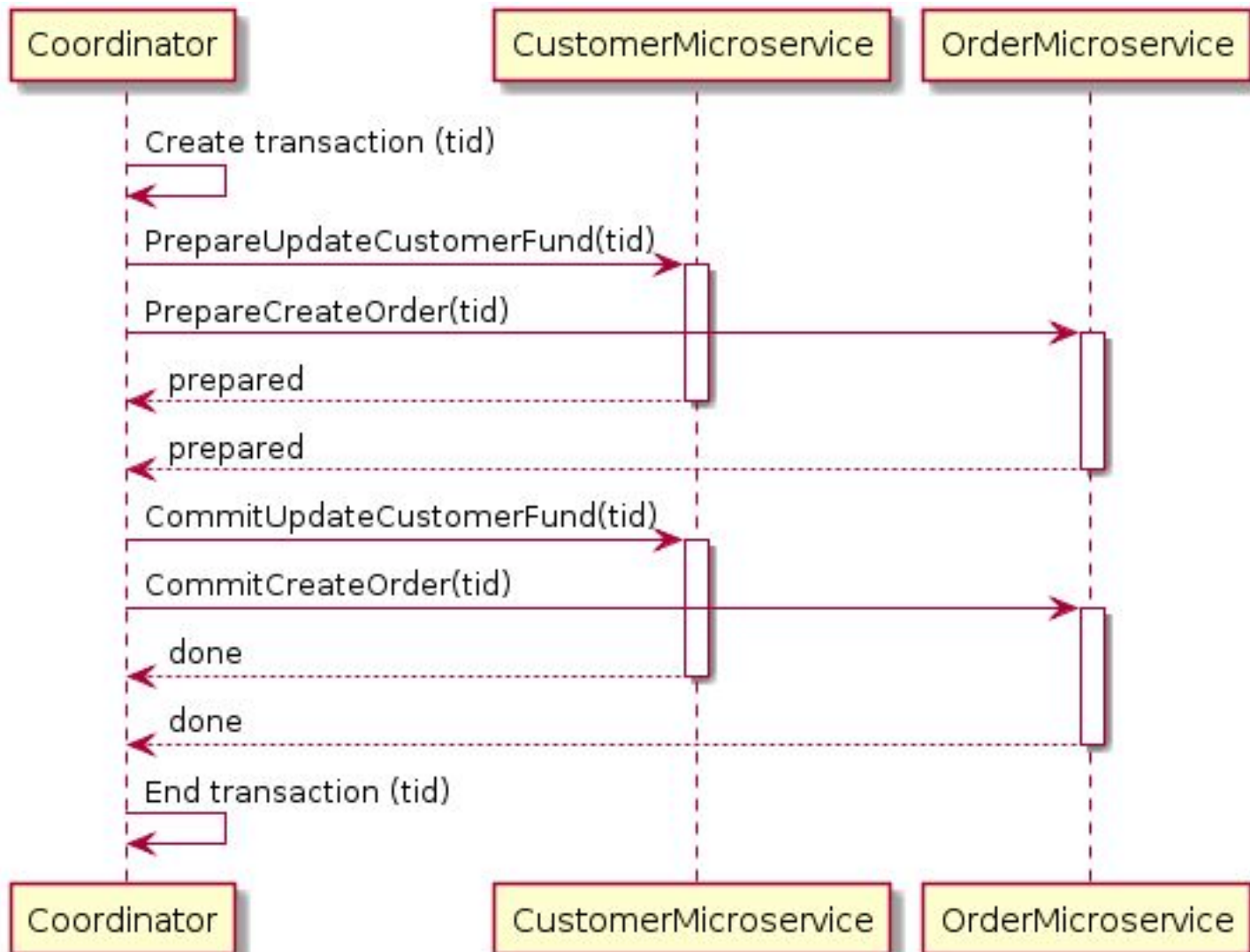
Действия первого кассира



Действия второго кассира

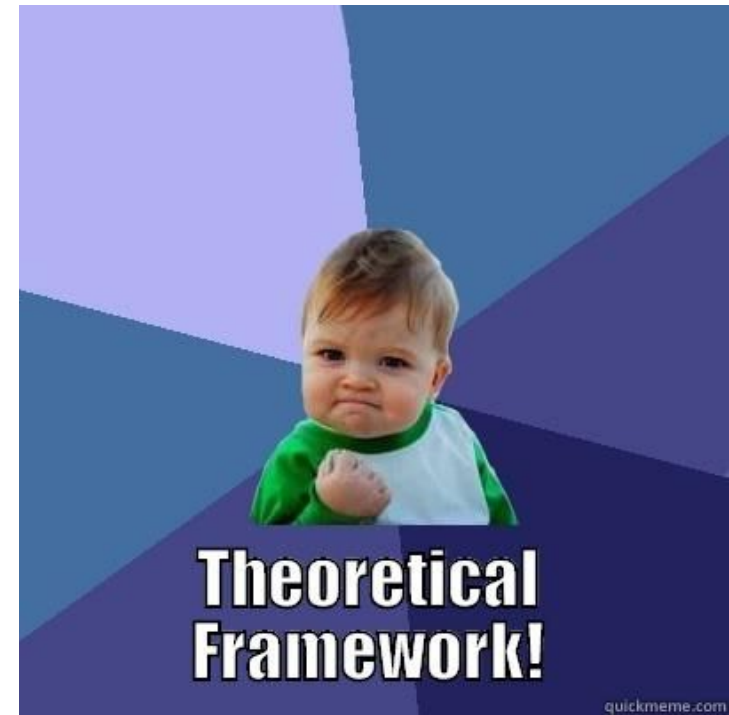


Распределённые транзакции



Требования к транзакциям. Принцип ACID.

- Сформулирован в конце 1970-х Дж. Греем.
- Состоит из компонентов:
 - Atomicity – атомарность.
 - Consistency – согласованность.
 - Isolation – изолированность.
 - Durability – долговечность.



- Принцип гарантирует, что никакая транзакция не будет выполнена частично.
- Должны быть выполнены либо все операции транзакции, либо не выполнено ни одной.
- Вводится понятие *отката* транзакции (rollback).

Согласованность данных

- Принцип гарантирует, что транзакция, достигшая своего завершения, сохраняет согласованность хранилища данных.
- Каждая успешная транзакция фиксирует только допустимые результаты.
- При этом, в ходе выполнения транзакции (до её завершения) согласованность не требуется!

Изолированность

- Во время выполнения транзакции параллельные транзакции не должны оказывать влияния на её результат.
- Полноценная реализация требует очень много ресурсов!
- В реальных системах используется концепция *уровней изолированности* транзакций.

Уровни изолированности транзакций

'+' — not possible

'-' — possible

Read phenomena Isolation level	Dirty reads	Lost updates	Non-repeatable reads	Phantoms
Read Uncommitted	-	-	-	-
Read Committed	+	-	-	-
Repeatable Read	+	+	+	-
Serializable	+	+	+	+

Долговечность транзакции

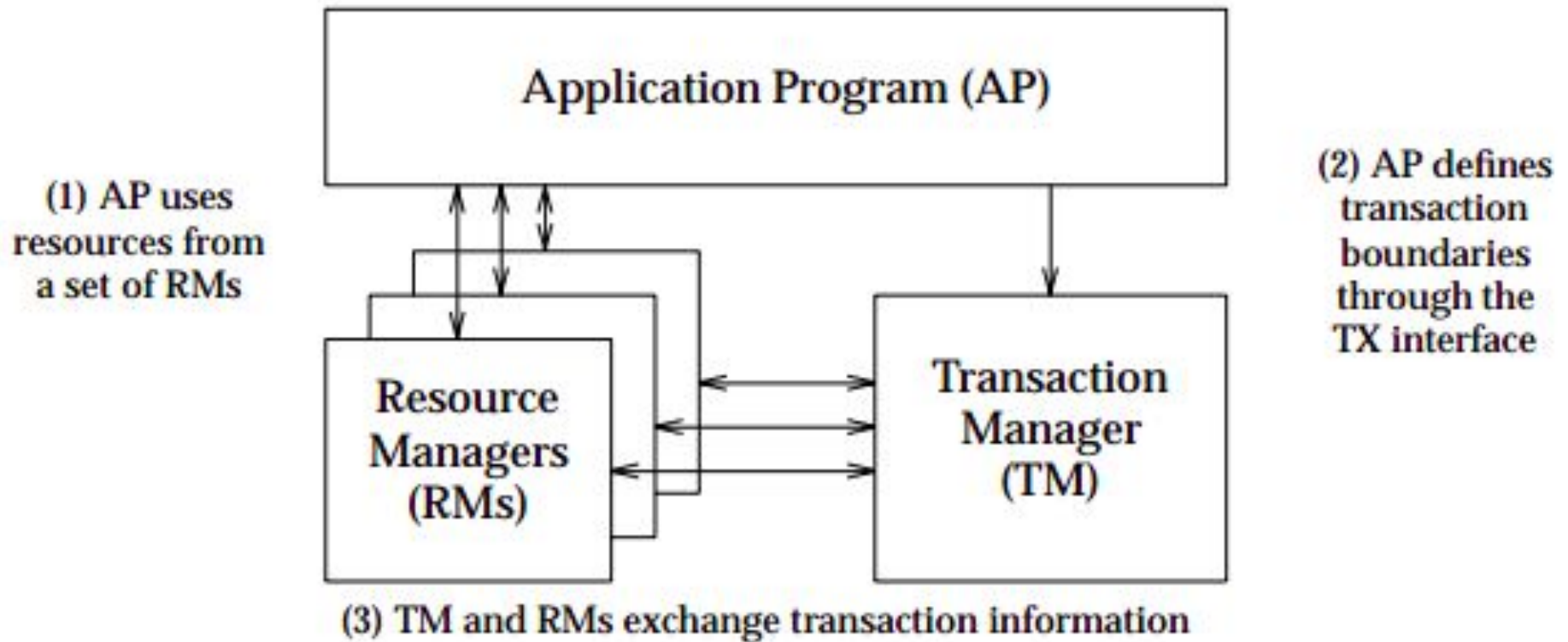
- Принцип гарантирует сохранение изменений, внесённых успешно завершённой транзакцией.
- Пользователь, получивший подтверждение о завершении транзакции, может быть уверен, что внесённые им изменения не могут быть отменены.
- Принцип должен работать даже в случае системных сбоев!

Транзакции в современных ИС

- “Классическая” модель реализации на уровне СУБД часто не работает:
 - Распределённый и гетерогенный уровень хранения – разные БД на разных узлах и т.д.
 - Распределённая и гетерогенная архитектура на уровне бизнес-логики.
- Возможные решения:
 - Не использовать “глобальные” транзакции.
 - Управлять транзакциями со стороны бизнес-логики.

- XA, X/OpenXA (e**X**tended **A**rchitecture) – спецификация распределённых транзакций.
- Впервые описана в 1992 г., является индустриальным стандартом в реализации менеджеров транзакций.
- Определяет принципы взаимодействия с транзакционными ресурсами в распределённых системах.

Модель ХА



Элементы модели ХА

- *Менеджеры ресурсов (RM)* – управляют конкретными ресурсами (например, СУБД).
- *Менеджер транзакций (ТМ, он же координатор)* – координирует работу RM и принимает решение о фиксации или откате транзакции.
- *Прикладная программа (AP)* – описывает бизнес-логику, управляет составом транзакций и используемыми в ней RM.
- В ХА специфицировано только взаимодействие между RM и ТМ, а за AP отвечает программист.

Виды транзакций в ХА

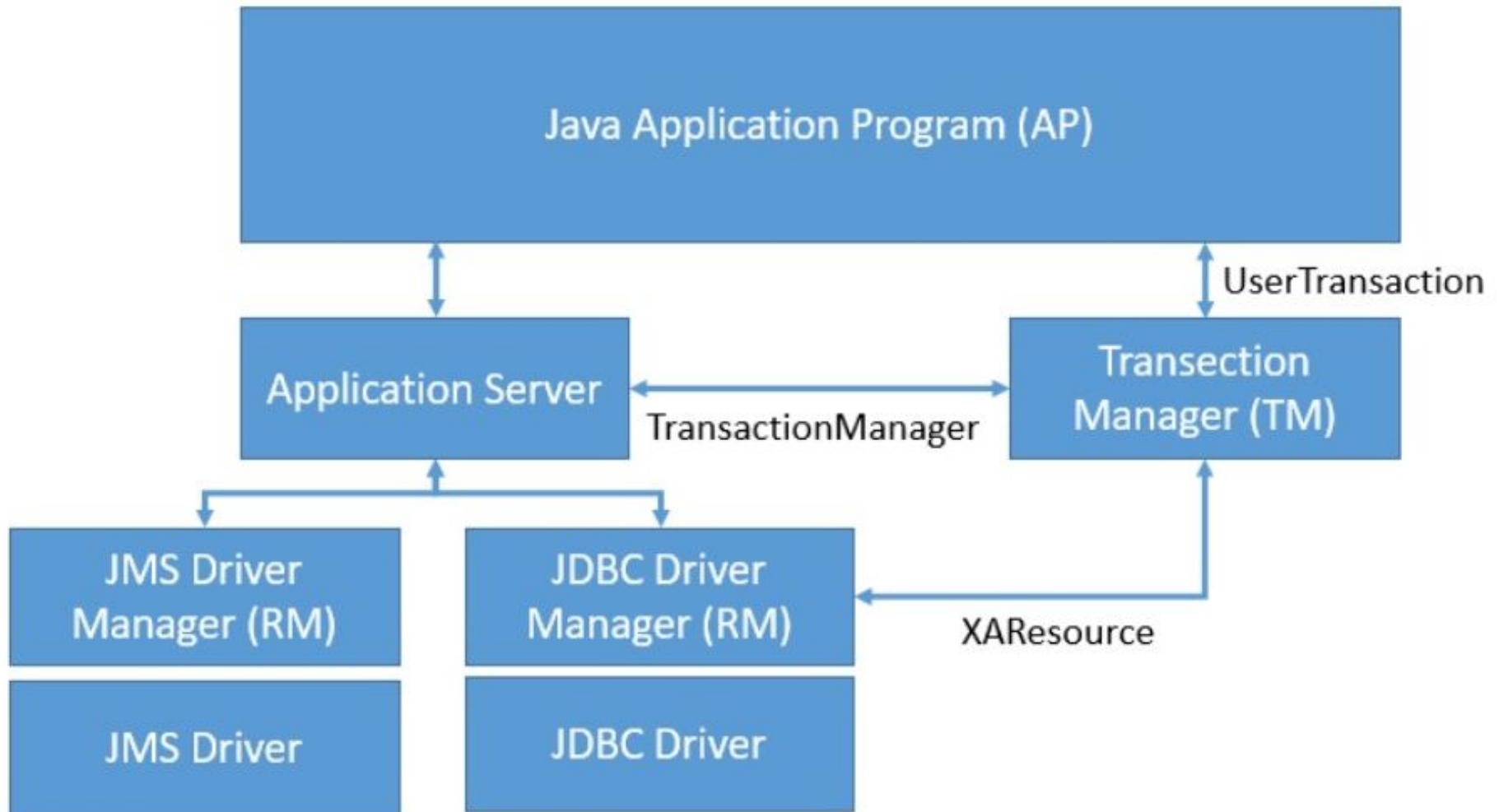
- *Локальные* – используется только одно хранилище, роль ТМ отводится АР.
- *Глобальные* (распределённые):
 - Несколько хранилищ.
 - АР задаёт границы транзакций.
 - Управлением транзакциями занимается координатор.
- *Вложенные*:
 - Транзакция запускается внутри существующей.
 - Изменения фиксируются только при фиксации “верхней” транзакции.

7. Java Transaction API

Java Transaction API

- API для управления распределёнными транзакциями.
- Специфицирован, входит в состав Java EE.
- Контейнер содержит *менеджер транзакций*, манипулирующий *ресурсами*.
- Реализует стандарт X/OpenXA.

Архитектура JTA



Особенности JTA

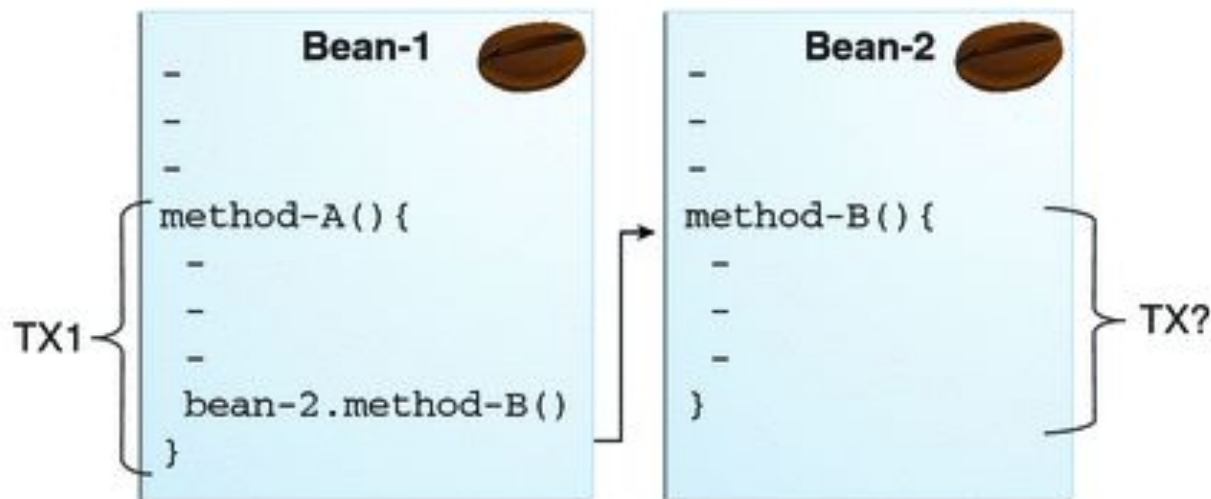
- Поддержка реализуется на уровне сервера приложений (`DataSource`) и JDBC-драйвера (`XAResource` и др.).
- На несовместимых с XA JDBC-драйверах распределённые транзакции работать не будут!
- Два высокоуровневых режима управления транзакциями (в EJB) – программный (BMT) и декларативный (CMT).

Использование JTA

```
XADataSource xaDS = getDataSource();
XAConnection xaCon = xaDS.getXAConnection("jdbc_user", "jdbc_password");
XAResource xaRes = xaCon.getXAResource();
Xid xid = new MyXid(100, new byte[]{0x01}, new byte[]{0x02});
Connection con = xaCon.getConnection();
Statement stmt = con.createStatement();
int ret;
try {
    xaRes.start(xid, XAResource.TMNOFLAGS);
    stmt.executeUpdate("insert into test_table values (100)");
    xaRes.end(xid, XAResource.TMSUCCESS);
    ret = xaRes.prepare(xid);
    if (ret == XAResource.XA_OK) {
        xaRes.commit(xid, false);
    }
}
catch (XAException e) { e.printStackTrace();}
finally {
    stmt.close();
    con.close();
    xaCon.close();
}
```

- Решения об открытии и закреплении транзакции принимает контейнер.
- Границы транзакции определяются границами методов и используемыми аннотациями.
- Что делать при вызовах методов друг из друга, также определяется аннотациями.

СМТ: пример



- Есть 6 ~~способов~~ способов включения методов в транзакции друг друга.
- Они определяются значением атрибута `@Transactional` над классом или методом.
- Значение атрибута `@Transactional` над методом переопределяет значение "по умолчанию", взятое из класса.

@TransactionAttribute (1)

- *Required* – “включает” вызываемый метод в контекст транзакции вызывающего. Вызов “извне” транзакции порождает новую.
- *RequiresNew* – всегда создаёт новую транзакцию. Если есть существующая – она приостанавливается до завершения выполнения новой.
- *Mandatory* – аналогично *Required*, но вызов “извне” транзакции выбрасывает `TransactionRequiredException`.

@TransactionAttribute (2)

- *NotSupported* – приостанавливает текущую транзакцию до возврата из метода. При вызове “извне” транзакции ничего не делает.
- *Supports* – если текущая транзакция есть – “включает” в неё вызываемый метод. Если её нет – ничего не делает.
- *Never* – если текущая транзакция есть – выбрасывает `RemoteException`. Если её нет – ничего не делает.

Пример использования CMT

```
@TransactionAttribute(NOT_SUPPORTED)
@Stateful
public class TransactionBean implements Transaction {
    ...
    @TransactionAttribute(REQUIRES_NEW)
    public void firstMethod() {...}

    @TransactionAttribute(REQUIRED)
    public void secondMethod() {...}

    public void thirdMethod() {...}

    public void fourthMethod() {...}
}
```

JTA@CDI: BMT

```

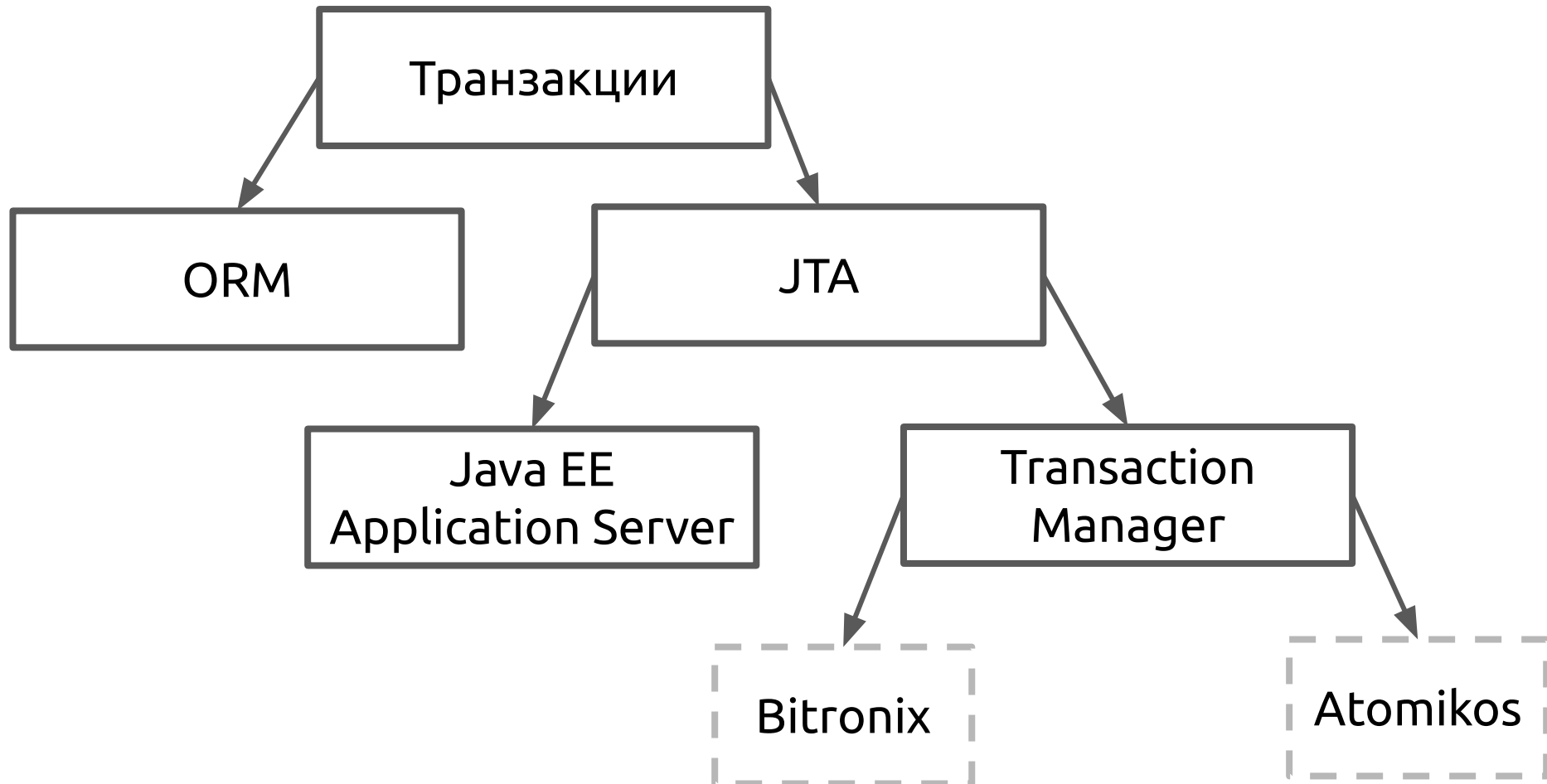
@Stateless
@TransactionManagement(TransactionManagementType.BEAN)  ← ❶ Uses BMT
public class OrderManagerBean {
    @Resource
    private UserTransaction userTransaction;  ❷ Injects UserTransaction

    public void placeSnagItOrder(Item item, Customer customer){
        try {
            userTransaction.begin();  ← ❸ Starts transaction
            if (!bidsExisting(item)){
                validateCredit(customer);
                chargeCustomer(customer, item);
                removeItemFromBidding(item);
            }
            userTransaction.commit();  ← ❹ Commits transaction
        } catch (CreditValidationException cve) {
            userTransaction.rollback();
        } catch (CreditProcessingException cpe){
            userTransaction.rollback();
        } catch (DatabaseException de) {
            userTransaction.rollback();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
    
```

❺ Rolls back transaction on exception

8. Транзакции в Spring

Транзакции в Spring (1)



Транзакции в Spring (2)

- Как и в Java EE, два режима – программный и декларативный.
- Разработчики Spring рекомендуют использовать декларативный режим.
- Режим “по умолчанию” использует менеджер транзакций ORM-фреймворка.
- Можно использовать транзакции JTA, для этого нужен провайдер – сервер приложений Java EE или отдельный менеджер транзакций.

Два варианта действий:

- Используем `TransactionTemplate`, в нём задаём параметры транзакции.
- Используем `TransactionManager` напрямую.

Используем TransactionTemplate (1)

Файл context.xml:

```
<!-- Initialization for data source -->
<bean id="dataSource" class="org.springframework.jdbc.datasource.DriverManagerDataSource">
<property name="driverClassName" value="com.mysql.jdbc.Driver"/>
<property name="url" value="jdbc:mysql://localhost:3306/TEST"/>
<property name="username" value="root"/>
<property name="password" value="password"/>
</bean>

<!-- Initialization for TransactionManager -->
<bean id="transactionManager" class="org.springframework.jdbc.datasource.DataSourceTransaction
Manager">
<property name="dataSource" ref="dataSource" />
</bean>

<!-- Definition for ServiceImpl bean -->
<bean id="serviceImpl" class="com.service.ServiceImpl">
<constructor-arg ref="transactionManager"/>
</bean>
```

Используем TransactionTemplate (2)

Реализация сервиса:

```
public class ServiceImpl implements Service {
    private final TransactionTemplate transactionTemplate;
    public ServiceImpl(PlatformTransactionManager transactionManager){
        this.transactionTemplate = new TransactionTemplate(transactionManager);

        //Конфигурация параметров транзакции
        this.transactionTemplate.setIsolationLevel(TransactionDefinition
                                                    .ISOLATION_READ_UNCOMMITTED);

        this.transactionTemplate.setTimeout(30);
        // и так далее
    }

    public Object someServiceMethod(){
        return transactionTemplate.execute(new TransactionCallback(){
            public Object doInTransaction(TransactionStatus status){
                updateOperation1();
                return resultOfUpdateOperation2();
            }
        });
    }
}
```

Используем TransactionManager напрямую (1)

Файл context.xml:

```
<!-- Initialization for data source -->
<bean id="dataSource"
      class="org.springframework.jdbc.datasource.DriverManagerDataSource">

    <property name="driverClassName" value="com.mysql.jdbc.Driver"/>
    <property name="url" value="jdbc:mysql://localhost:3306/TEST"/>
    <property name="username" value="root"/>
    <property name="password" value="password"/>
</bean>

<!-- Initialization for TransactionManager -->
<bean id="transactionManager"

class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
    <property name="dataSource" ref="dataSource" />
</bean>
```

Используем TransactionManager напрямую (2)

Реализация сервиса:

```
public class ServiceImpl implements Service {
    private PlatformTransactionManager transactionManager;

    public void setTransactionManager(PlatformTransactionManager
                                    transactionManager) {
        this.transactionManager = transactionManager;
        DefaultTransactionDefinition def = new DefaultTransactionDefinition();
        def.setName("SomeTxName"); // Transaction name
        def.setPropagationBehavior(TransactionDefinition.PROPROPAGATION_REQUIRED);
        TransactionStatus status = txManager.getTransaction(def);
        try {
            // Place your business logic here
        } catch (Exception ex) {
            txManager.rollback(status);
            throw ex;
        }
        txManager.commit(status);
    }
}
```

Декларативное управление транзакциями в Spring

- Похоже на JTA (и может использовать его).
- Включается в XML-конфигурации приложения.
- Используются те же менеджеры транзакций, что и в программном режиме.
- Основной элемент – аннотация `@Transactional`.

Активация декларативного управления транзакциями

Происходит в конфигурации приложения:

```
@Configuration  
@EnableTransactionManagement  
public class AppConfig {  
    ...  
}
```

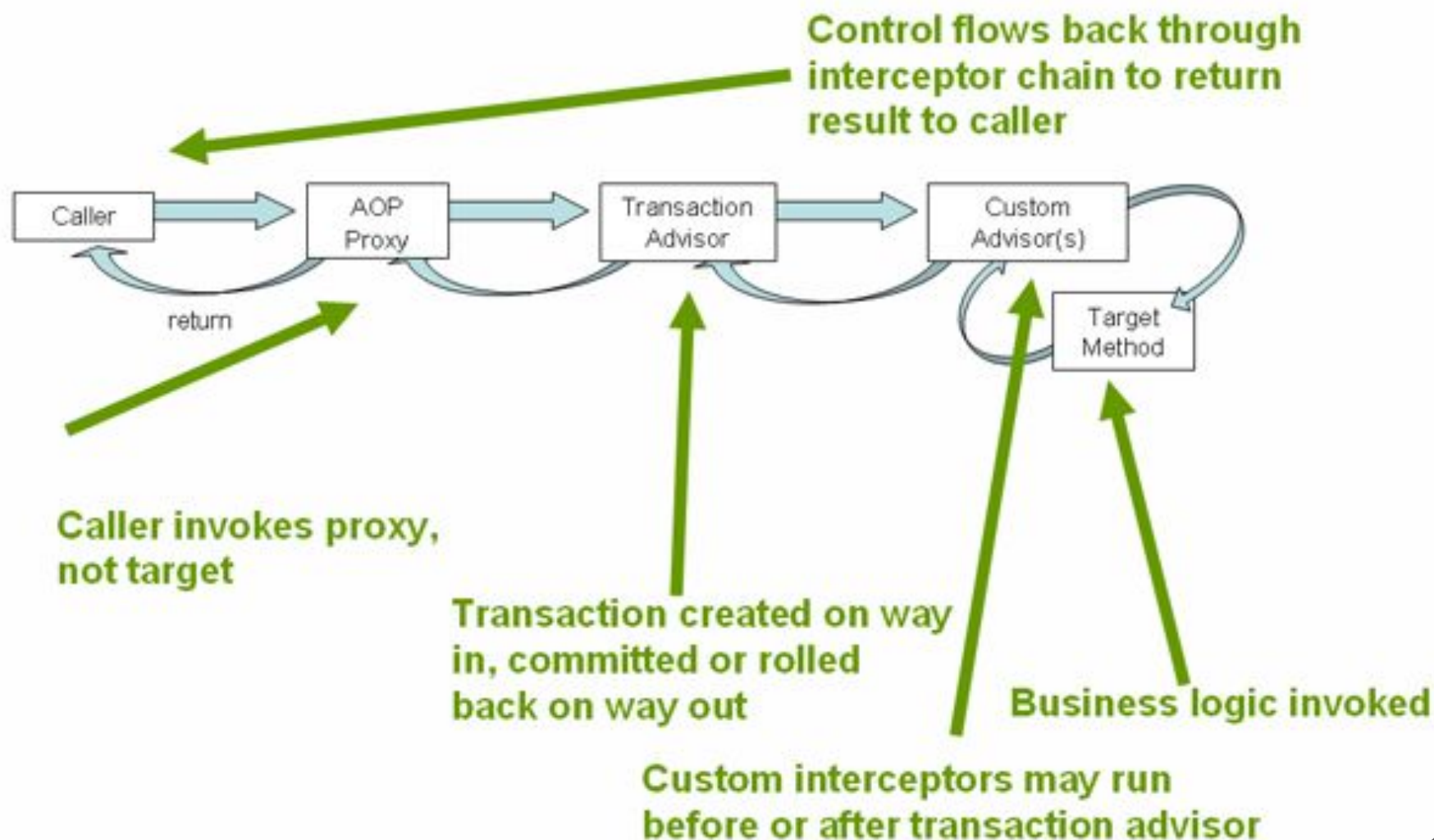
- Применяется к интерфейсу, классу или методу (как интерфейса, так и класса).
- Аннотация на уровне метода переопределяет “глобальные” настройки класса (или интерфейса).
- Режим по умолчанию –
`proxy-target-class="false"`:
перехватываются только “внешние”
вызовы методов класса.

Изоляция транзакций

- Задаётся атрибутом `isolation` аннотации `@Transactional`.
- Значение по умолчанию – `DEFAULT`.
- Другие поддерживаемые режимы – `READ_COMMITTED`, `READ_UNCOMMITTED`, `REPEATABLE_READ`, `SERIALIZABLE`.

- Задаётся атрибутом `propagation` аннотации `@Transactional`.
- Значение по умолчанию – `REQUIRED`.
- Возможные значения те же, что и в JTA – `REQUIRED`, `REQUIRES_NEW`, `MANDATORY`, `SUPPORTS`, `NOT_SUPPORTED`.

Схема работы @Transactional



Пример сервиса

```
@Service
public class ServiceTest{
    @Transactional
    public void test1{
        test2();
    }

    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void test2(){
    }
}
```

- Приложение на базе Spring Boot может работать с распределёнными транзакциями JTA.
- Два варианта реализации:
 - Использование сервера приложений Java EE с поддержкой JTA.
 - Использование встроенного менеджера транзакций с поддержкой JTA.

Менеджеры транзакций

- Специальное приложение или библиотека, реализующее управление распределёнными транзакциями.
- Реализует спецификацию JTA.
- Примеры – Bitronix, Atomikos.
- Вместо него Spring Boot может использовать возможности сервера приложений.

- Независимая реализация спецификации JTA 1.1.
- Open Source, лицензия Apache v2.
- Не развивается активно с 2016 г., но совместим с современными версиями Spring и ORM.

Добавляем зависимость в `pom.xml`:

```
<dependency>
  <groupId>
    org.springframework.boot
  </groupId>
  <artifactId>
    spring-boot-starter-jta-bitronix
  </artifactId>
</dependency>
```



Конфигурация Spring + Bitronix (1)

```
@Configuration
@EnableTransactionManagement
public class DbConfig {

    @Bean(name = "bitronixTransactionManager")
    @DependsOn
    public BitronixTransactionManager
        bitronixTransactionManager() throws Throwable {
        BitronixTransactionManager bitronixTransactionManager =
            TransactionManagerServices.getTransactionManager();

        bitronixTransactionManager.setTransactionTimeout(10000);
        CustomJtaPlatform
            .setUserTransaction(bitronixTransactionManager);
        CustomJtaPlatform
            .setTransactionManager(bitronixTransactionManager);
        return bitronixTransactionManager;
    }
}
```



Конфигурация Spring + Bitronix (2)

```
@Bean(name = "transactionManager")
@DependsOn({"bitronixTransactionManager"})
public PlatformTransactionManager
    transactionManager(TransactionManager
        bitronixTransactionManager) throws Throwable {
    return
        new JtaTransactionManager(bitronixTransactionManager);
}

@Bean(name = "primaryMySqlDataSource")
@Primary
public DataSource primaryMySqlDataSource() {
    PoolingDataSource bitronixDataSourceBean =
        new PoolingDataSource();
    bitronixDataSourceBean.setMaxPoolSize(5);
    bitronixDataSourceBean
        .setUniqueName("primaryMySqlDataSourceResource");
}
```



Конфигурация Spring + Bitronix (3)

```
bitronixDataSourceBean.setClassName(  
    "com.mysql.jdbc.jdbc2.optional.MysqlXADataSource");  
Properties properties = new Properties();  
properties.put("user", "dbuser");  
properties.put("password", "dbpasswd");  
properties.put("url",  
    "jdbc:mysql://localhost/databaseOne");  
bitronixDataSourceBean.setDriverProperties(properties);  
return bitronixDataSourceBean;  
}  
  
@Bean(name = "primaryMySqlEntityManagerFactory")  
@Primary  
@DependsOn({"transactionManager", "primaryMySqlDataSource"})  
public LocalContainerEntityManagerFactoryBean  
    primaryMySqlEntityManagerFactory(  
        EntityManagerFactoryBuilder builder,  
        DataSource primaryMySqlDataSource) {
```

Конфигурация Spring + Bitronix (4)

```
        return builder
            .dataSource(primaryMySqlDataSource)
            .persistenceUnit("primaryUnit")
            .packages(Cat.class)
            .build();
    }

    @Bean(name = "secondaryMySqlDataSource")
    public DataSource secondaryMySqlDataSource() { ... }

    @Bean(name = "secondaryMySqlEntityManagerFactory")
    @DependsOn({"transactionManager",
        "secondaryMySqlDataSource"})
    public LocalContainerEntityManagerFactoryBean
        secondaryMySqlEntityManagerFactory(
            EntityManagerFactoryBuilder builder,
            DataSource secondaryMySqlDataSource) { ... }
}
```

Atomikos Transaction Manager

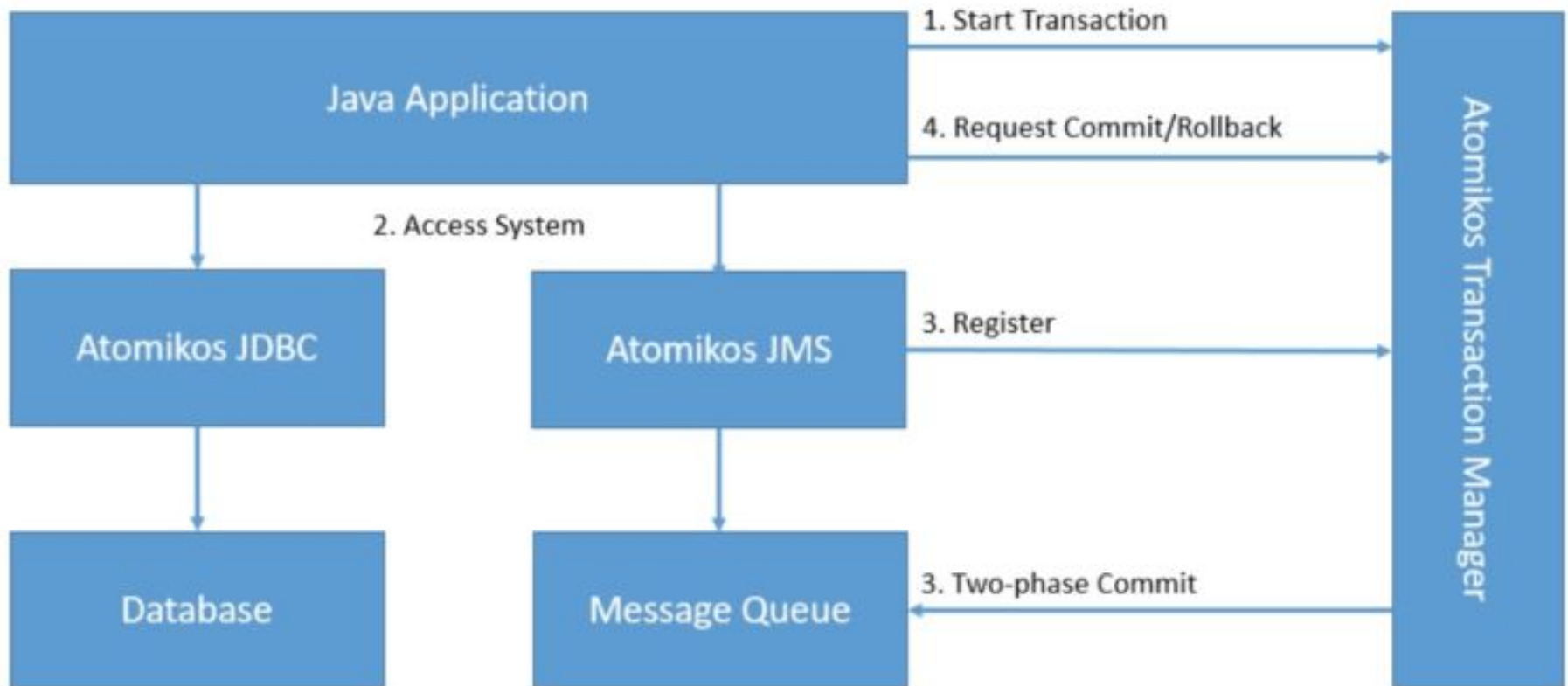
- Похож на Bitronix.
- Open Source, лицензия Apache v2.
- Активно поддерживается.

Atomikos vs Bitronix

Feature	Benefit	Atomikos	Bitronix (BTM)
Easy to configure	Works out-of-the-box	✓	✓
Built-in connection pools	No need to integrate with third-party pools that are not designed for XA	✓	✓
Embeddable in your application	Light-weight, easy to deploy, easy to test in IDE, fast turnaround times for development	✓	✓
XA transactions	Simple reliability	✓	✓
Spring Boot integration	Works in the most popular Java VM	✓	✓
Tomcat integration	Works in the most popular web container	✓	✓
Commercial support available	Fast solutions for problems	✓	□
Microservices support	Commit/rollback across several microservices	✓	□
Elastic scaling	One simple configuration that can scale up and down	✓	□

✓ = included ✓ = unclear □ = not included

Архитектура Atomikos



Добавляем зависимость в `pom.xml`:

```
<dependency>  
  <groupId>  
    org.springframework.boot  
  </groupId>  
  <artifactId>  
    spring-boot-starter-jta-atomikos  
  </artifactId>  
</dependency>
```

...см. слайды с Bitronix.

Spring Boot + JTA@AppServer

- Достаточно использовать Full-Profile Java EE Application Server.
- Spring Boot попыбует самостоятелыно выполнить lookup по стандартным путям `java:comp/UserTransaction`, `java:comp/TransactionManager` и т.д.
- Spring Boot попыбует автоматически сконфигурировать JMS, используя ресурсы `java:/JmsXA` и `java:/XAConnectionFactory`.

9. Безопасность и разграничение доступа

Контекст проблемы

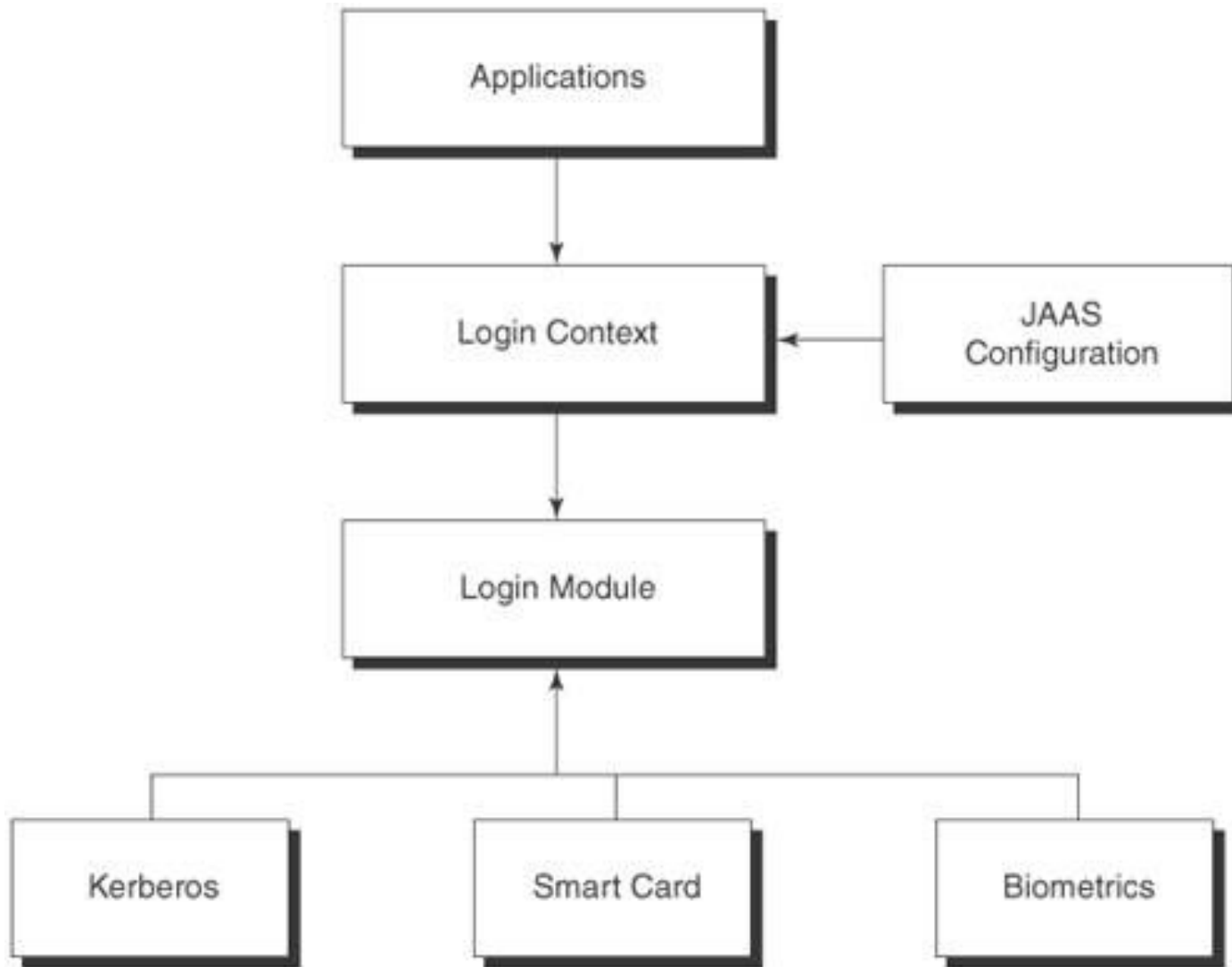
- Разграничения доступа на уровне пользовательского интерфейса недостаточно.
- Необходимо обеспечить:
 - “Сквозное” разграничение полномочий в рамках всего приложения.
 - Отделение друг от друга понятий “роль” и “полномочия”.
 - Механизм сопоставления ролей и наборов полномочий на уровне конкретного приложения.

Ключевые термины и определения

- *Аутентификация* – проверка того, что пользователь является тем, за кого он себя выдаёт.
- *Авторизация* – контроль доступа; выполняется после успешной аутентификации.
- *Полномочие* – право на выполнение какого-либо действия.
- *Роль* – именованный набор полномочий. Каждая учётная запись может иметь свой набор ролей.

- Стандарт API и реализация сервиса информационной безопасности.
- Входит в состав Java SE, появилась в Java 1.3.
- С точки зрения системного администратора – набор конфигурационных файлов.
- С точки зрения программиста – API для проверки ролей и полномочий.

Архитектура JAAS



Конфигурационные файлы JAAS

- `*.login.conf` – определяет, какие логин-модули необходимо задействовать в программе.
- `*.policy` – определяет, какими привилегиями наделены пользователи или программы.

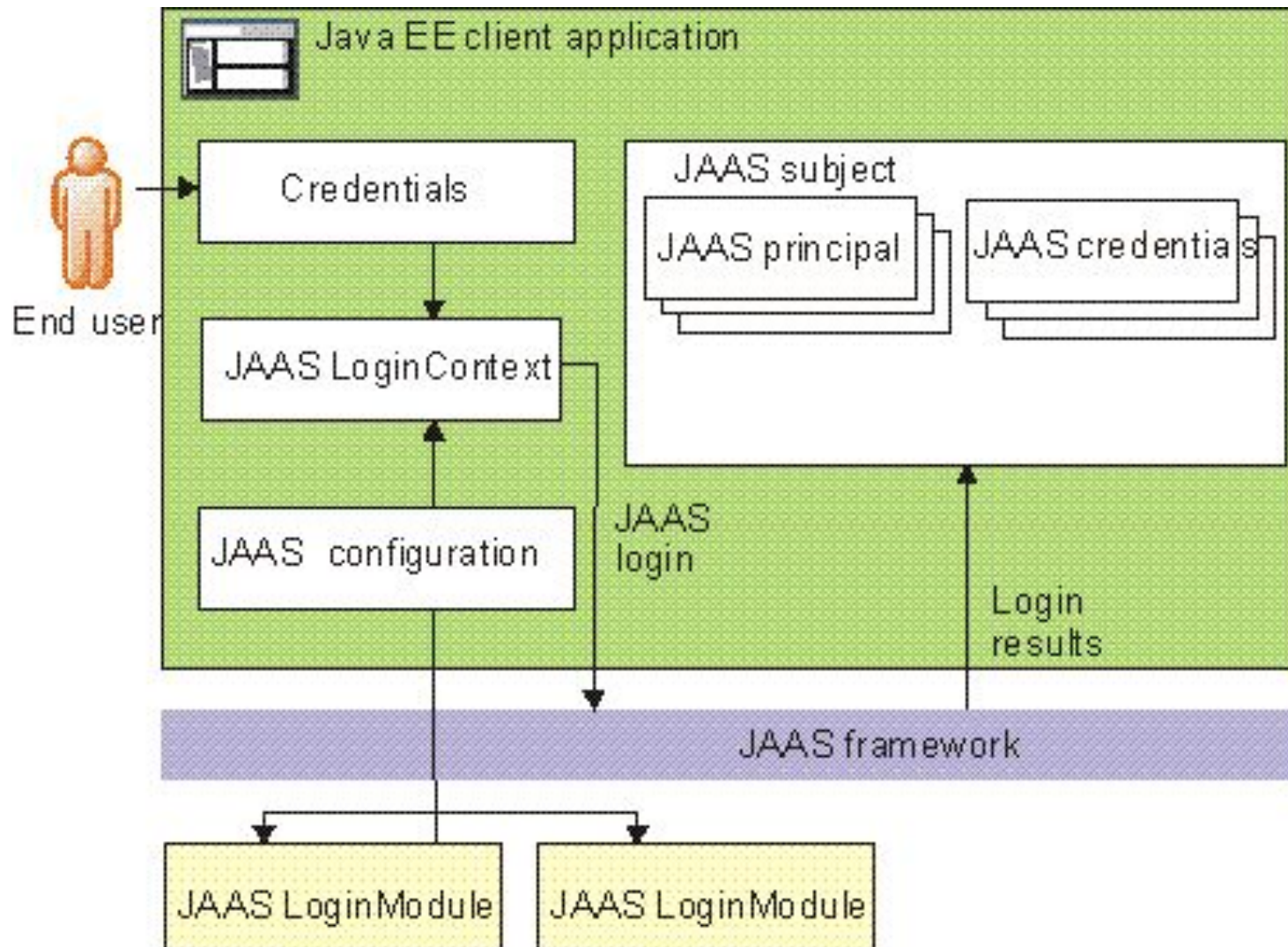
- *Subject* – субъект аутентификации; носитель или обладатель прав.
- Субъект – источник (*source*) запроса (*request*) на выполнение некоторого действия.
- *Principal* – представление субъекта с некоторой точки зрения.
- *Credential* – то, чем субъект подтверждает, что это он.

- API, реализация которой осуществляет аутентификацию и авторизацию пользователей в приложении.
- Содержит 5 абстрактных методов:
 - `initialize.`
 - `login.`
 - `commit.`
 - `abort.`
 - `logout.`

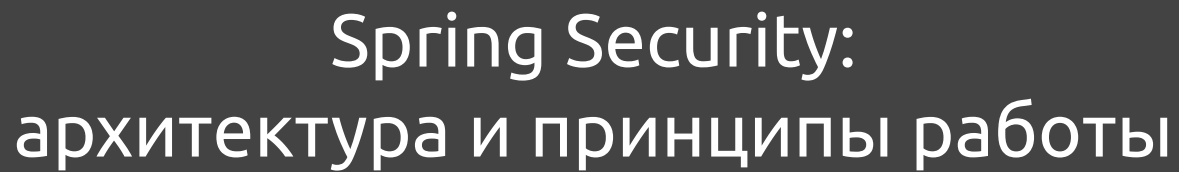
Access Control Policy File

```
/** Java 2 Access Control Policy for the JaasAzn Application */  
/** Code-Based Access Control Policy for JaasAzn */  
grant codebase "file:../JaasAzn.jar" {  
    permission javax.security.auth.AuthPermission  
        "createLoginContext.JaasSample";  
    permission javax.security.auth.AuthPermission "doAsPrivileged";  
};  
  
/** User-Based Access Control Policy for the SampleAction class  
    ** instantiated by JaasAzn  
    **/  
grant    codebase "file:../SampleAction.jar",  
        Principal javax.security.auth.kerberos.KerberosPrincipal  
            "your_user_name@your_realm" {  
    permission java.util.PropertyPermission "java.home", "read";  
    permission java.util.PropertyPermission "user.home", "read";  
    permission java.io.FilePermission "foo.txt", "read";  
};
```

Использование JAAS в Java EE



- Фреймворк аутентификации, авторизации и ролевого разграничения доступа в составе Spring.
- Разрабатывается с 2003 г.
- В рамках архитектуры Spring обеспечивает “сквозное” разграничение доступа на всех уровнях.
- Полномочий нет – только роли.
- Есть возможность интеграции с JAAS.



Spring Security: конфигурация

```
@Profile({"production"})
@EnableWebSecurity
@EnableGlobalMethodSecurity(prePostEnabled = true)
public class ProductionSecurityConfig extends WebSecurityConfigurerAdapter {

    @Override
    public void configure(HttpSecurity http) throws Exception {
        http
            .csrf().disable()
            .authorizeRequests()
                .antMatchers("/admin/**").hasAnyAuthority(Role.SUPER_USER.getAuthority(),
                    Role.SUPER_USER.getAuthority())
                .antMatchers("/student/**").hasAnyAuthority(Role.STUDENT.getAuthority(),
                    Role.SUPER_USER.getAuthority())
                .antMatchers("/university/**").hasAnyAuthority(Role.UNIVERSITY.getAuthority(),
                    Role.SUPER_USER.getAuthority())
                .antMatchers("/public/**").permitAll()
                .antMatchers("/**").authenticated();
    }
}
```

Spring Security:

разграничение доступа к методам

```
@GetMapping("/{certId}")
@PreAuthorize("hasAuthority(T(ru.edu.portfolio.domain.ApiPermission).COURSE_CERTIFICATE_READ)")
public List<SomeDTO> read(@PathVariable long certId) {
    // Some logic
}

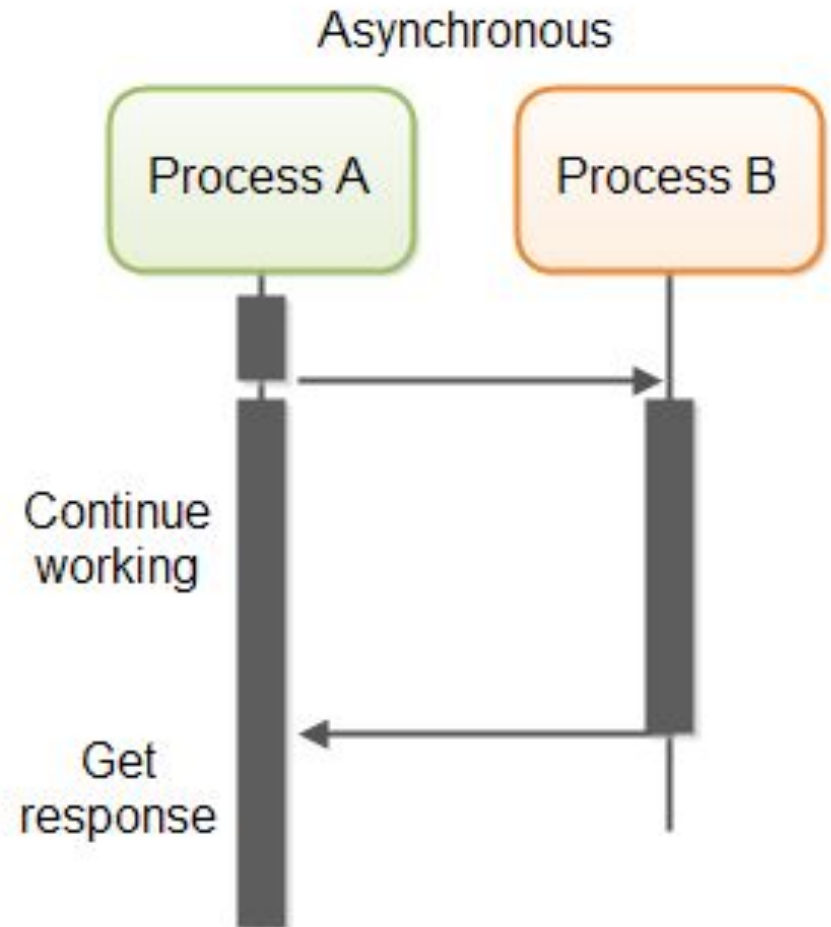
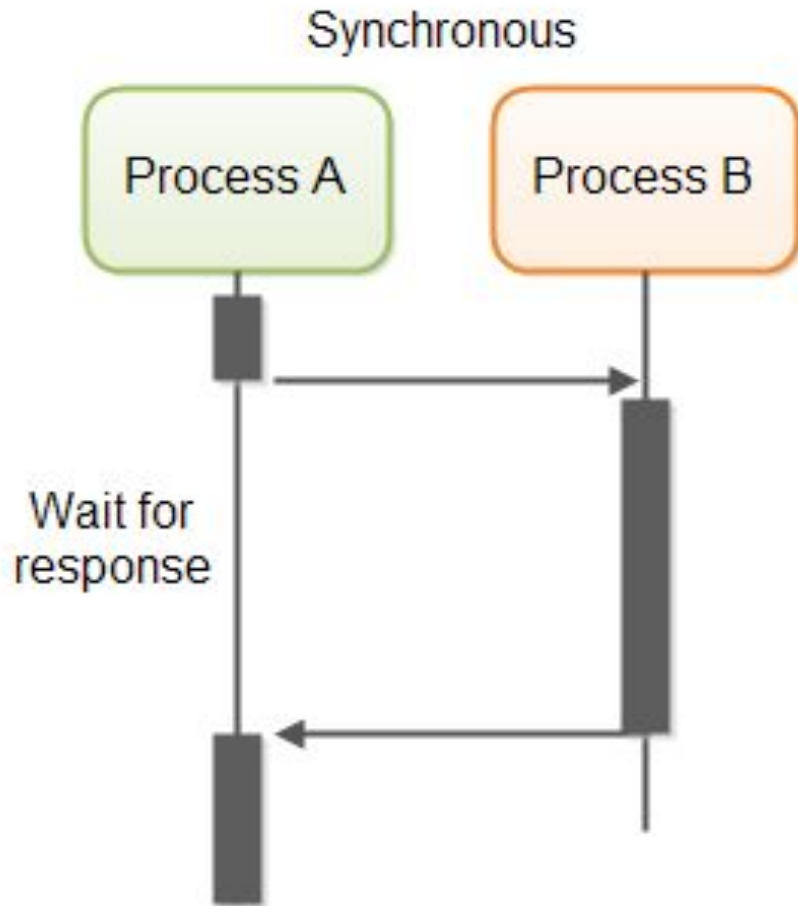
@GetMapping("/{certId}")
@PreAuthorize("authentication.details.SSLAuth")
public List<SomeDTO> readWithAuth(@PathVariable long certId) {
    // Some logic
}
```

Интеграция с JAAS

- Spring Security может интегрироваться с JAAS, делегируя туда запросы на аутентификацию.
- Для этого нужно реализовать интерфейс `AbstractJaasAuthenticationProvider` и настроить правила интеграции модулей аутентификации Spring с инфраструктурой JAAS.
- Инструкция по интеграции есть в официальной документации Spring.

10. Асинхронное выполнение задач

Синхронная и асинхронная обработка



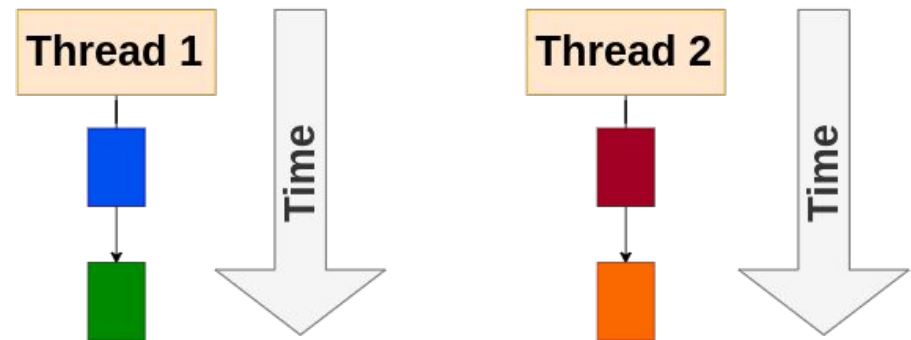
Зачем оно может быть нужно

- Вызывающий процесс может продолжать свою работу во время выполнения задачи.
- Эффективно при выполнении ресурсоёмких задач.
- Эффективно утилизирует ресурсы многоядерных ЦП.
- Можно горизонтально масштабировать в распределённом окружении.
- Подходит далеко не для любых задач!

Асинхронная обработка vs МНОГОПОТОЧНОСТЬ

- Одно не подразумевает другое!
- Чаще всего, всё-таки, используются вместе.

Multithreading



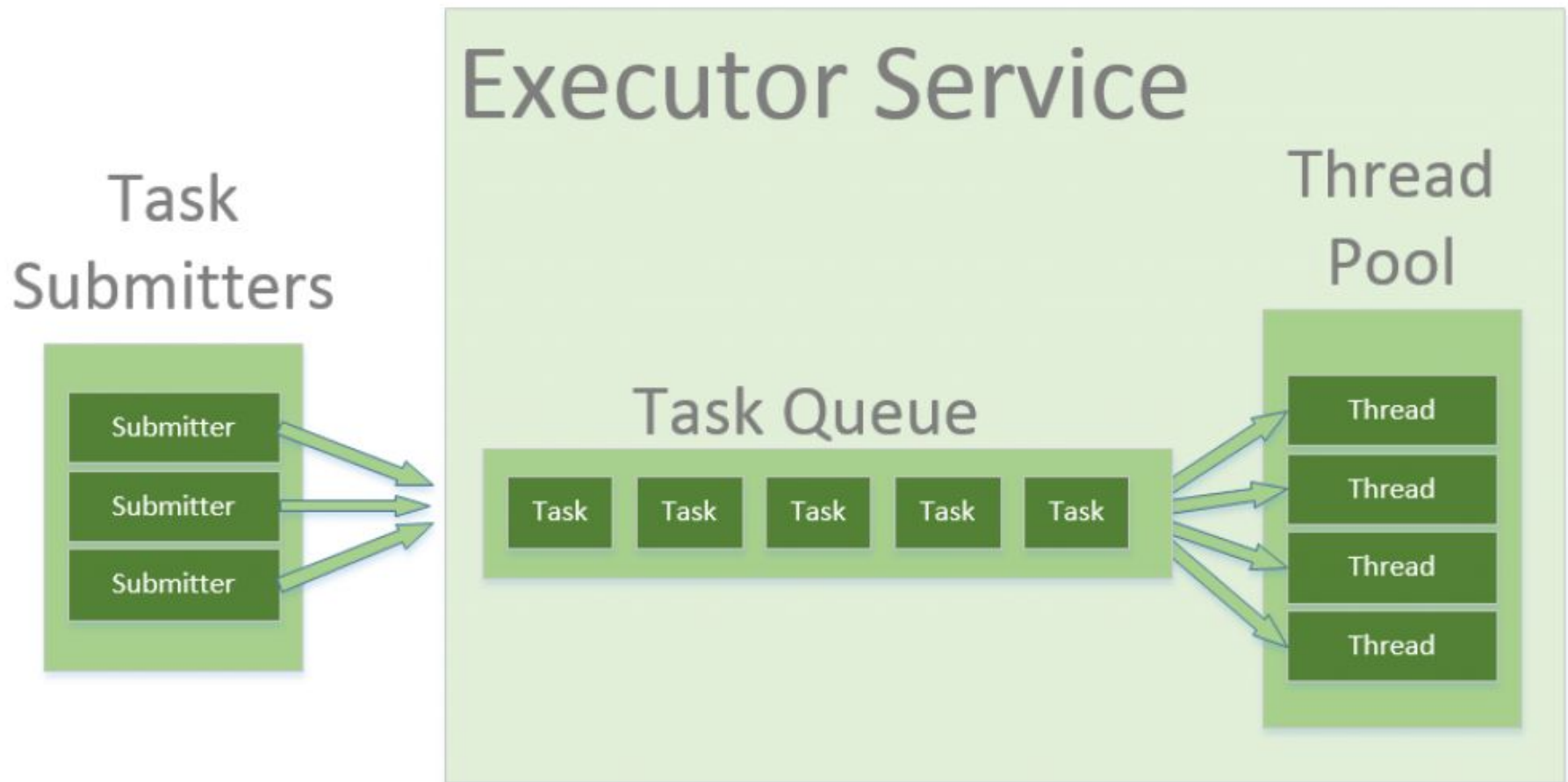
Asynchrony



Подходы к реализации

- “Вручную” – много взаимодействующих процессов и / или потоков.
- С помощью инфраструктурного ПО – ActiveMQ, RabbitMQ, Kafka...
- Абстрактный API а-ля JMS с реализацией “под капотом” сервера приложений.
- Второй и третий пути не исключают друг друга!

Обработка “вручную”: ExecutorService



Использование ExecutorService

```
public static void main(String[] args) throws ExecutionException, InterruptedException {  
    Callable<String> task = () -> Thread.currentThread().getName();  
    ExecutorService service = Executors.newFixedThreadPool(2);  
    for (int i = 0; i < 5; i++) {  
        Future result = service.submit(task);  
        System.out.println(result.get());  
    }  
    service.shutdown();  
}
```

Обработка “вручную”: интерфейс Future

Описывает API для работы с задачами, результат которых планируется получить в будущем.

```
import java.util.concurrent.Callable;
import java.util.concurrent.FutureTask;

public class HelloWorld {

    public static void main(String []args) throws Exception {
        Callable task = () -> {
            return "Hello, World!";
        };
        FutureTask<String> future = new FutureTask<>(task);
        new Thread(future).start();
        System.out.println(future.get());
    }
}
```

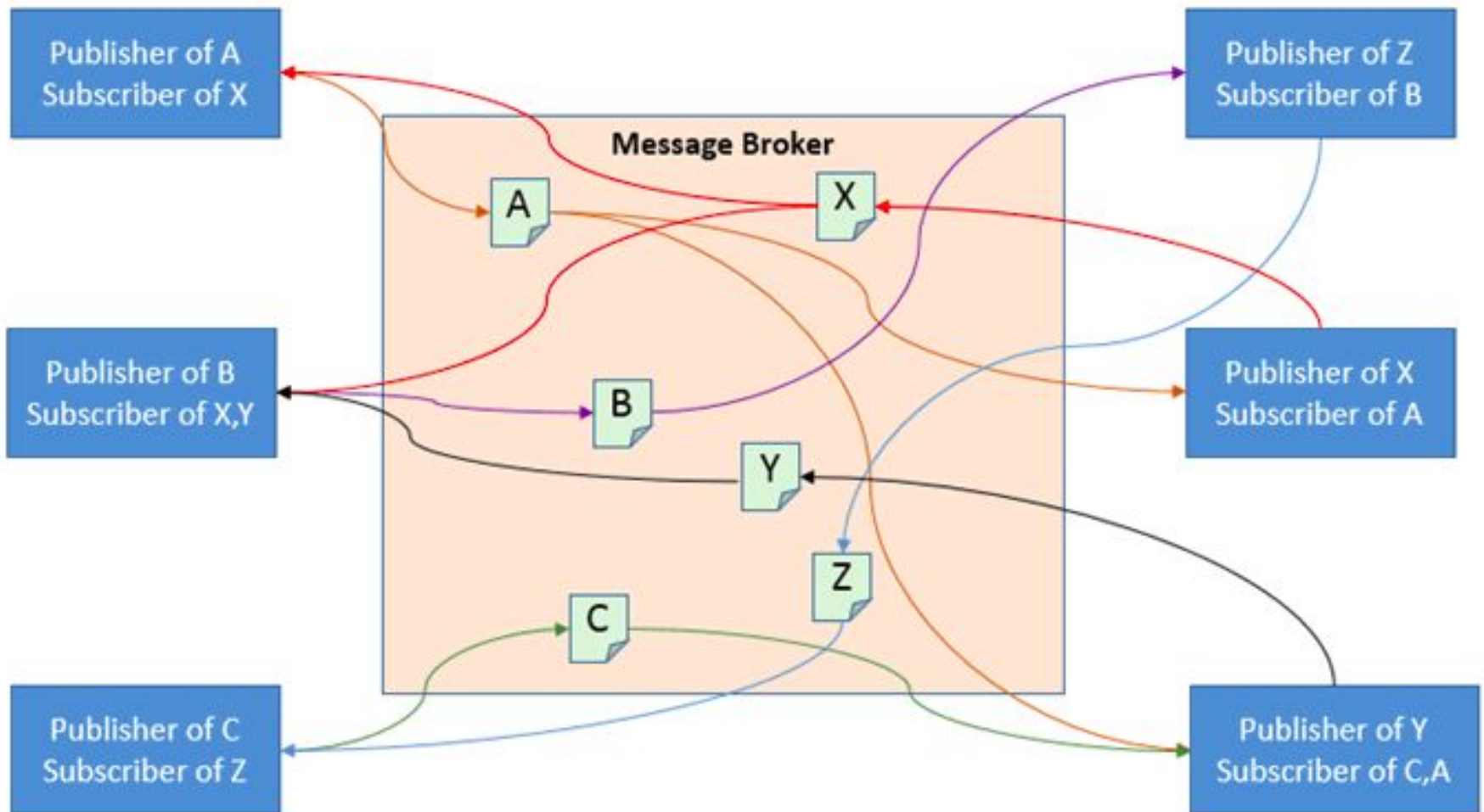
Обработка “вручную”: CompletableFuture и лямбды

CompletableFuture позволяет проверять статус выполнения задачи.

```
public static void main(String []args) throws Exception {  
    AtomicLong longValue = new AtomicLong(0);  
    Runnable task = () -> longValue.set(new Date().getTime());  
    Function<Long, Date> dateConverter = (longvalue) -> new Date(longvalue);  
    Consumer<Date> printer = date -> {  
        System.out.println(date);  
        System.out.flush();  
    };  
    // CompletableFuture computation  
    CompletableFuture.runAsync(task)  
        .thenApply((v) -> longValue.get())  
        .thenApply(dateConverter)  
        .thenAccept(printer);  
}
```

- Позволяет “переложить ответственность” за асинхронную обработку на отдельную подсистему.
- Может добавлять дополнительные возможности – гарантию доставки сообщения, “прозрачную” распределённую обработку и т. д.
- Для обмена сообщениями можно использовать различные протоколы.
- Частный случай – *брокер сообщений*.

Брокеры сообщений



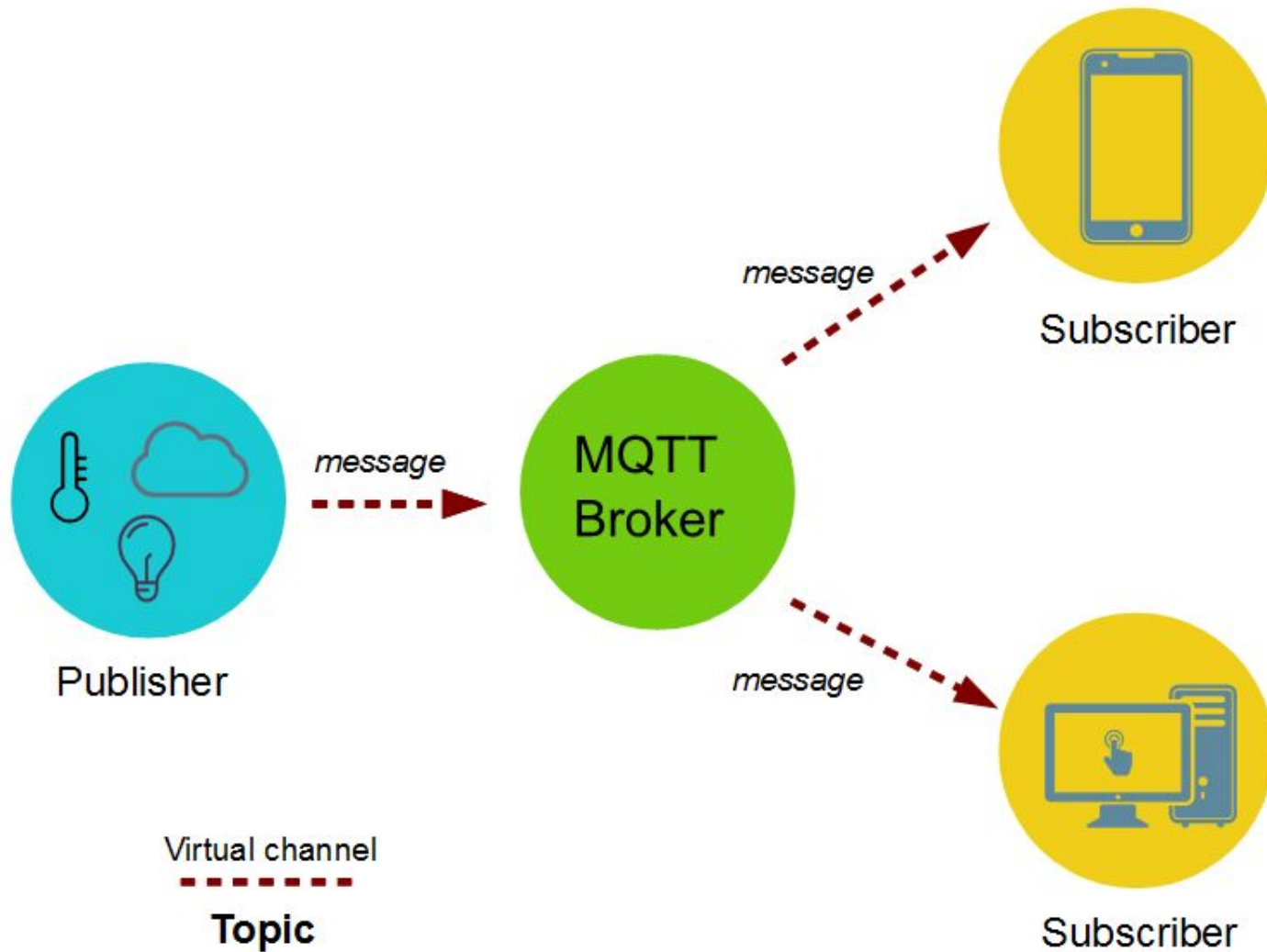
- Определяют правила передачи сообщений инфраструктурному ПО.
- В качестве транспортного уровня обычно используется TCP, UDP или HTTP.
- Набор поддерживаемых протоколов у каждого инфраструктурного ПО свой.
- Чаще всего используются MQTT, AMQP, STOMP, XMPP, а также “связка” HTTP + WebSockets.

Message Queue Telemetry Transport

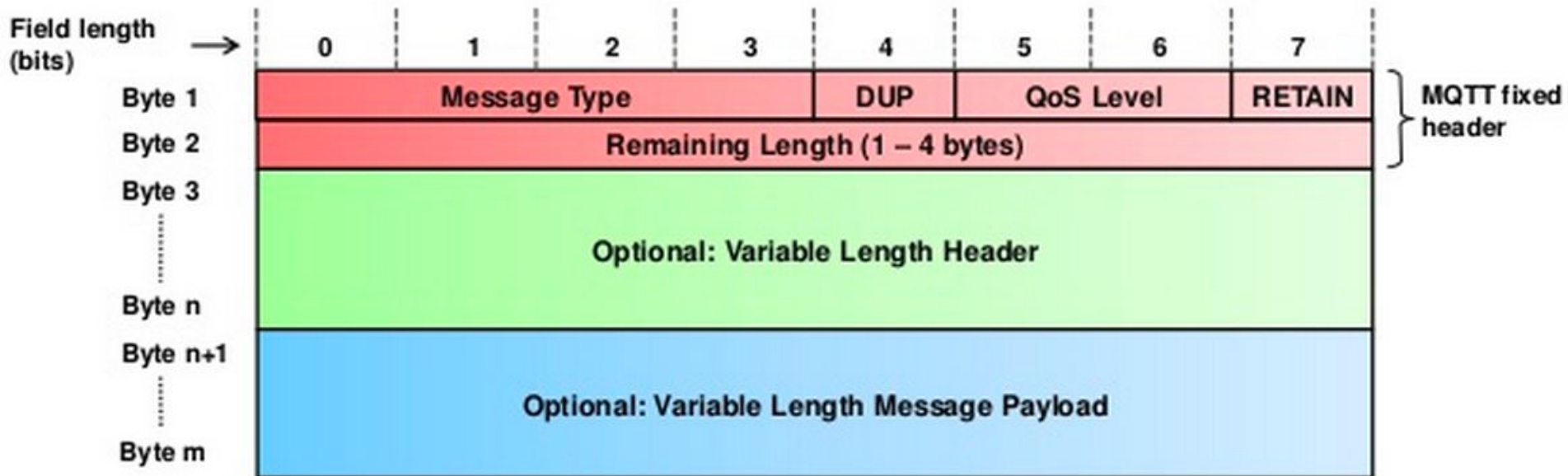
- Сетевой протокол прикладного уровня.
- Стандартизирован OASIS, актуальная версия – 5.0 (2019 г.).
- Двоичный протокол, работает поверх TCP/IP.
- Использует модель “издатель-подписчик”.
- Изначально предназначен для организации взаимодействия между устройствами.
- Легковесный и простой в использовании.
- Широко распространён в мире IoT.



Архитектура MQTT



Формат сообщений MQTT



Клиент MQTT на Java

- Много разных библиотек от разных разработчиков.
- Наиболее популярные – Eclipse Paho, HiveMQ.
- Пример клиента – см. следующие два слайда.



Paho MQTT Client (1)

```
public static void main(String[] args) {  
    String topic          = "MQTT Examples";  
    String content        = "Message from MqttPublishSample";  
    int qos               = 2;  
    String broker         = "tcp://mqtt.eclipse.org:1883";  
    String clientId       = "JavaSample";  
    MemoryPersistence persistence = new MemoryPersistence();  
    try {  
        MqttClient sampleClient =  
            new MqttClient(broker, clientId, persistence);  
        MqttConnectOptions connOpts =  
            new MqttConnectOptions();  
        connOpts.setCleanSession(true);  
        System.out.println("Connecting to broker: "+broker);  
        sampleClient.connect(connOpts);  
        System.out.println("Connected");  
    }  
}
```

Paho MQTT Client (2)

```
System.out.println("Publishing message: "+content);
MqttMessage message = new MqttMessage(content.getBytes());
message.setQos(qos);
sampleClient.publish(topic, message);
System.out.println("Message published");
sampleClient.disconnect();
System.out.println("Disconnected");
System.exit(0);
} catch (MqttException me) {
    System.out.println("reason "+me.getReasonCode());
    System.out.println("msg "+me.getMessage());
    System.out.println("loc "+me.getLocalizedMessage());
    System.out.println("cause "+me.getCause());
    System.out.println("excep "+me);
    me.printStackTrace();
}
}
```

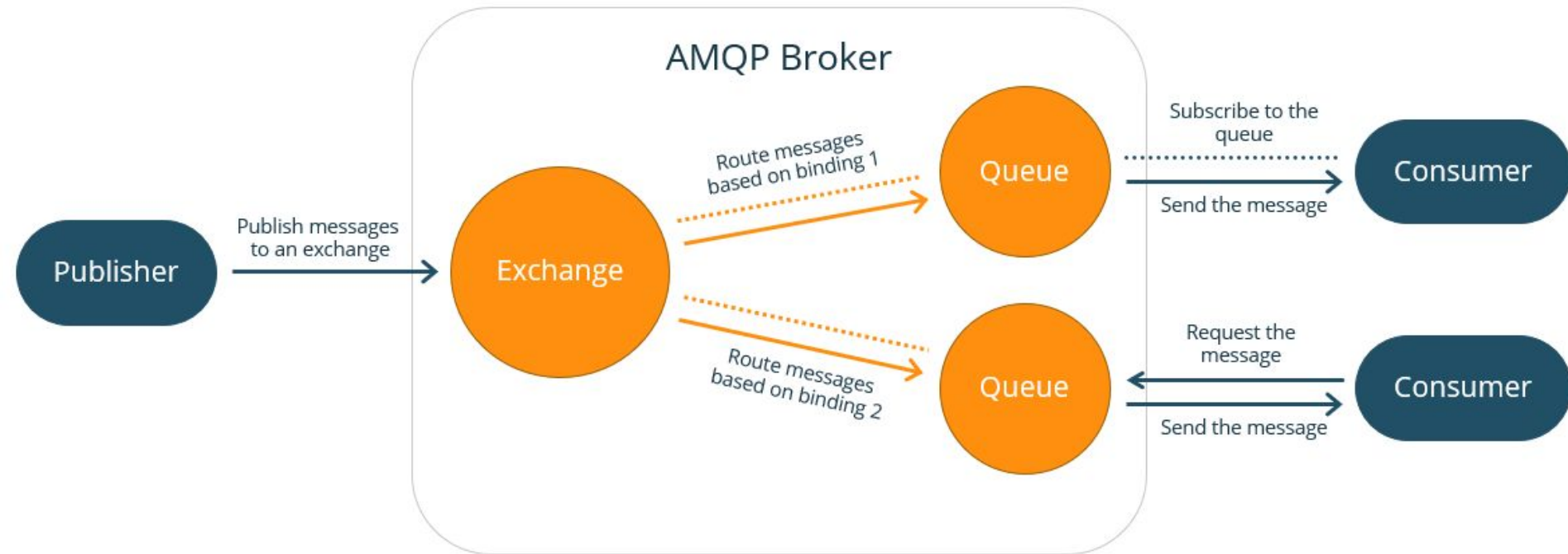
Преимущества и недостатки MQTT

- [+] Легковесный, быстрый.
- [+] Нетребователен к пропускной способности сети.
- [+] Реализует несколько моделей доставки сообщений.
- [-] Негибкий: к примеру, поддерживается всего 5 видов сообщений об ошибке и нет возможности сообщить клиенту об ошибке после установления HTTP-соединения.
- [-] Могут быть проблемы с масштабируемостью.

Advanced Message Queuing Protocol

- Протокол передачи сообщений между компонентами системы.
- Открытый, предложен специалистами банка JP Morgan Chase в 2003 г.
- Двоичный, прикладного уровня, обычно функционирует “поверх” TCP.
- Ключевой элемент инфраструктуры – AMQP-брокер.
- AMQP 0-9-1 и AMQP 1.0 – это *разные* протоколы!

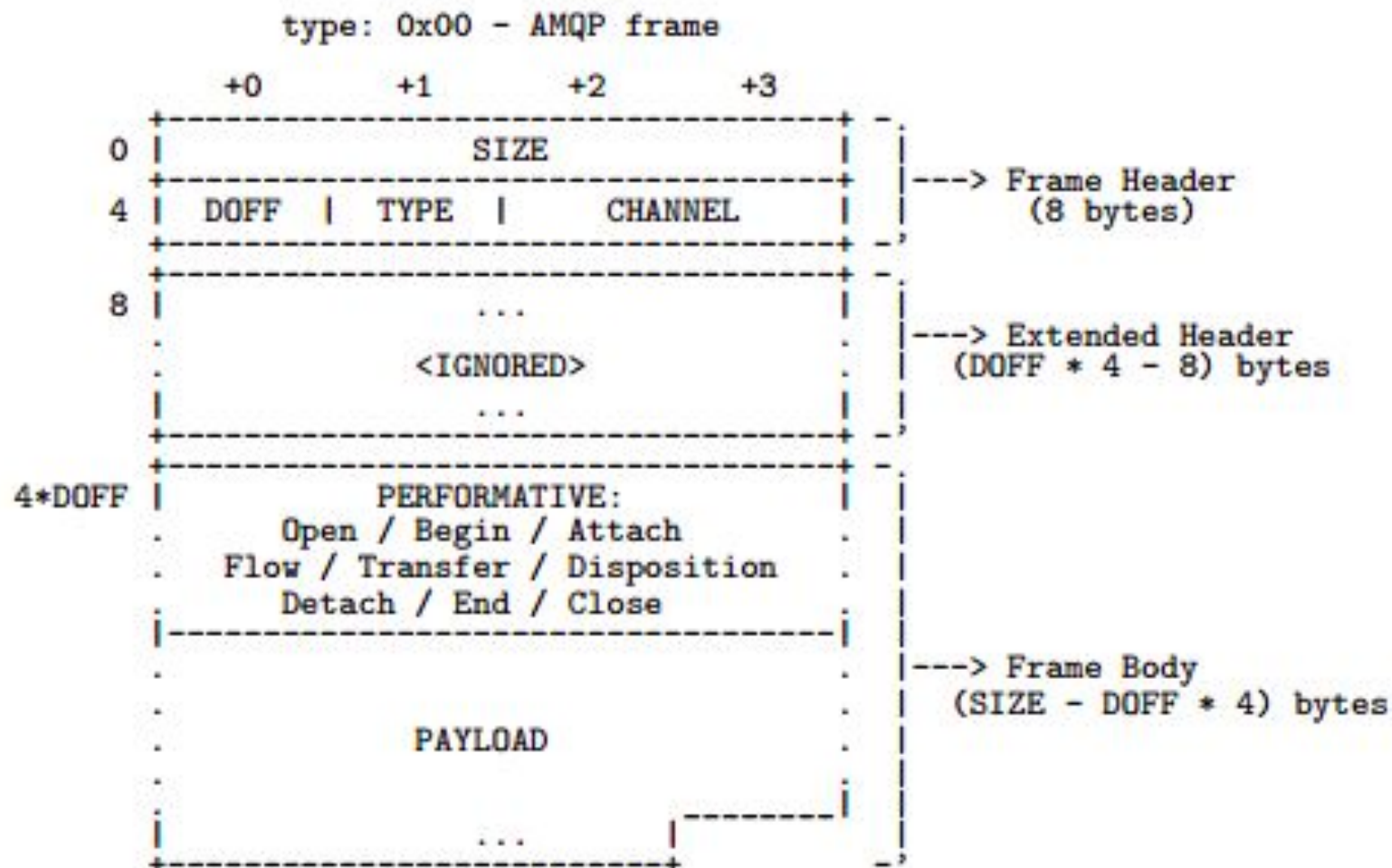
Архитектура AMQP



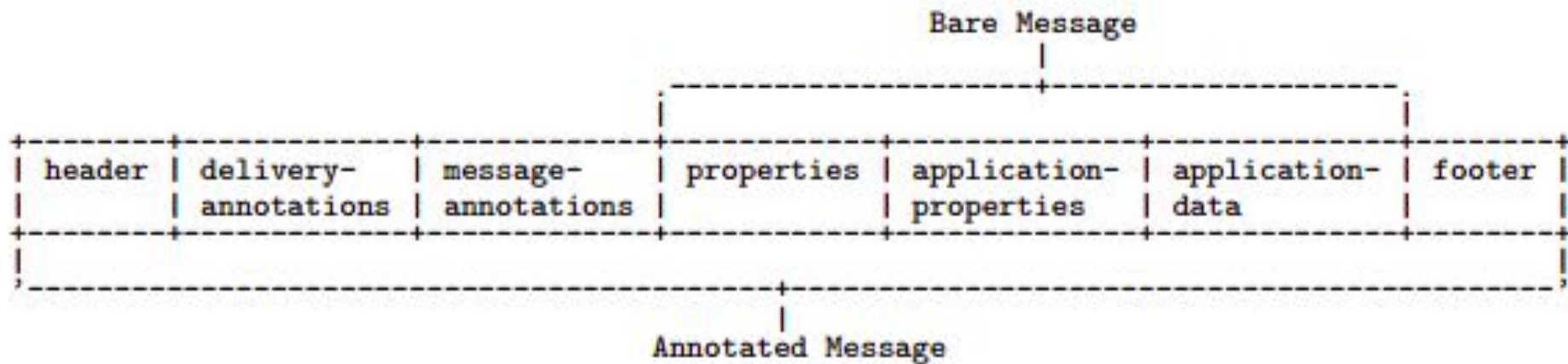
AMQP: термины и определения

- Сообщения описывают сами себя (self-described).
- Основной элемент, на базе которого строится взаимодействие – *фрейм*.
- Сообщения передаются после установления соединения (link).
- Содержимое фрейма бывает 9 видов – `open (connection)`, `begin (session)`, `attach (link)`, `transfer (message)`, `flow`, `disposition`, `detach (link)`, `end (session)`, `close (connection)`.¹⁷⁵

Структура фрейма AMQP



Структура сообщения AMQP



Клиенты AMQP на Java

- Есть несколько реализаций от разных разработчиков.
- Самая популярная – RabbitMQ AMQP Client Library.
- Есть поддержка в Spring Boot (тоже на базе RabbitMQ Client).

Преимущества и недостатки AMQP

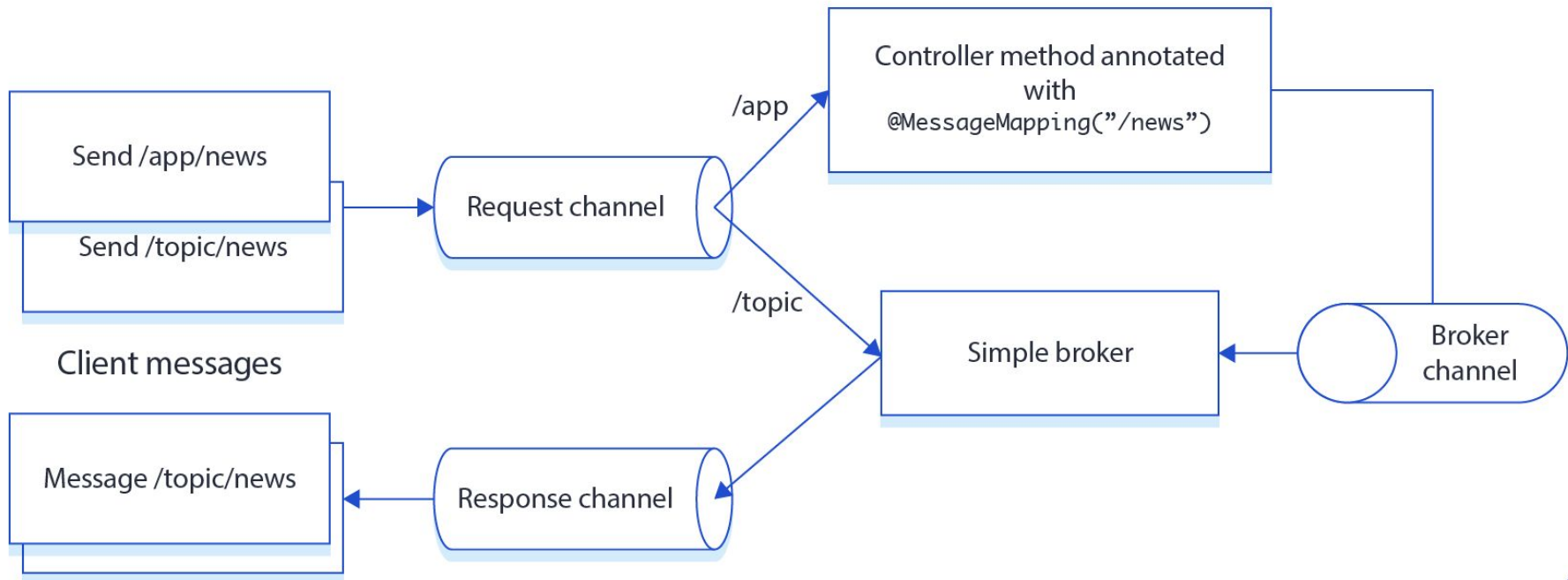
- **[+]** Мощнее, чем MQTT – больше видов сообщений, больше возможностей для их обработки.
- **[+]** Поддерживает транзакции.
- **[+]** Поддерживает различные механизмы оповещения об обработке сообщений.
- **[-]** “Тяжелее”, чем MQTT.
- **[-]** Небогатый выбор библиотек и API.

Streaming Text Oriented Messaging Protocol

- Текстовый протокол передачи сообщений.
- Прикладного уровня, похож на HTTP.
- Основная “философия” – простота и читаемость сообщений, отсутствие в них лишней метаданных и разметки.
- Обычно работает “поверх” TCP, есть реализации, работающие “поверх” WebSockets.
- Актуальная версия – 1.2 (2012 г.).

Stomp 

Пример использования STOMP



Сообщения STOMP

- Единица обмена информацией – *фрейм*, состоящий из *команды*, необязательных *заголовков* и необязательного *тела*.
- Пример сообщения:

```
SUBSCRIBE  
id:sub-1  
destination:/topic/price.stock.*
```

Основные команды STOMP

CONNECT

COMMIT

SEND

ABORT

SUBSCRIBE

ACK

UNSUBSCRIBE

NACK

BEGIN

DISCONNECT

MESSAGE

subscription:sub-1

message-id:wm2si1tj-4

content-type: application/json

destination:/topic/stocks.PRICE.STOCK.NASDAQ.EMC

```
{\"ticker\": \"EMC\", \"price\": 24.19}
```

На базе stomp.js и SockJS:

```
var socket = new SockJS('/spring-websocket-portfolio/portfolio');
var client = Stomp.over(socket);

var onConnect = function() {
    client.subscribe("/topic/price.stock.*", function(message) {
        // обработка квоты
    });
};

client.connect('guest', 'guest', onConnect);
```

Клиент STOMP на Java (1)

На базе библиотек Spring STOMP и SockJS:

```
public ListenableFuture<StompSession> connect() {  
  
    Transport websocketTransport = new WebSocketTransport(new StandardWebSocketClient());  
    List<Transport> transports = Collections.singletonList(websocketTransport);  
  
    SockJsClient sockJsClient = new SockJsClient(transports);  
    sockJsClient.setMessageCodec(new Jackson2SockJsMessageCodec());  
  
    WebSocketStompClient stompClient = new WebSocketStompClient(sockJsClient);  
  
    String url = "ws://{host}:{port}/hello";  
    return stompClient.connect(url, headers, new MyHandler(), "localhost", 8080);  
}
```

Клиент STOMP на Java (2)

```
public void subscribeGreetings(StompSession stompSession) throws ExecutionException, InterruptedException {
    stompSession.subscribe("/topic/greetings", new StompFrameHandler() {

        public Type getPayloadType(StompHeaders stompHeaders) {
            return byte[].class;
        }

        public void handleFrame(StompHeaders stompHeaders, Object o) {
            logger.info("Received greeting " + new String((byte[]) o));
        }
    });
}

public void sendHello(StompSession stompSession) {
    String jsonHello = "{ \"name\" : \"Nick\" }";
    stompSession.send("/app/hello", jsonHello.getBytes());
}
```

Клиент STOMP на Java (3)

```
private class MyHandler extends StompSessionHandlerAdapter {
    public void afterConnected(StompSession stompSession, StompHeaders stompHeaders) {
        logger.info("Now connected");
    }
}

public static void main(String[] args) throws Exception {
    HelloClient helloClient = new HelloClient();

    ListenableFuture<StompSession> f = helloClient.connect();
    StompSession stompSession = f.get();

    logger.info("Subscribing to greeting topic using session " + stompSession);
    helloClient.subscribeGreetings(stompSession);

    logger.info("Sending hello message" + stompSession);
    helloClient.sendHello(stompSession);
    Thread.sleep(60000);
}
```

Преимущества и недостатки STOMP

- [+] Текстовый.
- [+] Простой.
- [+] Хорошо читаемый.

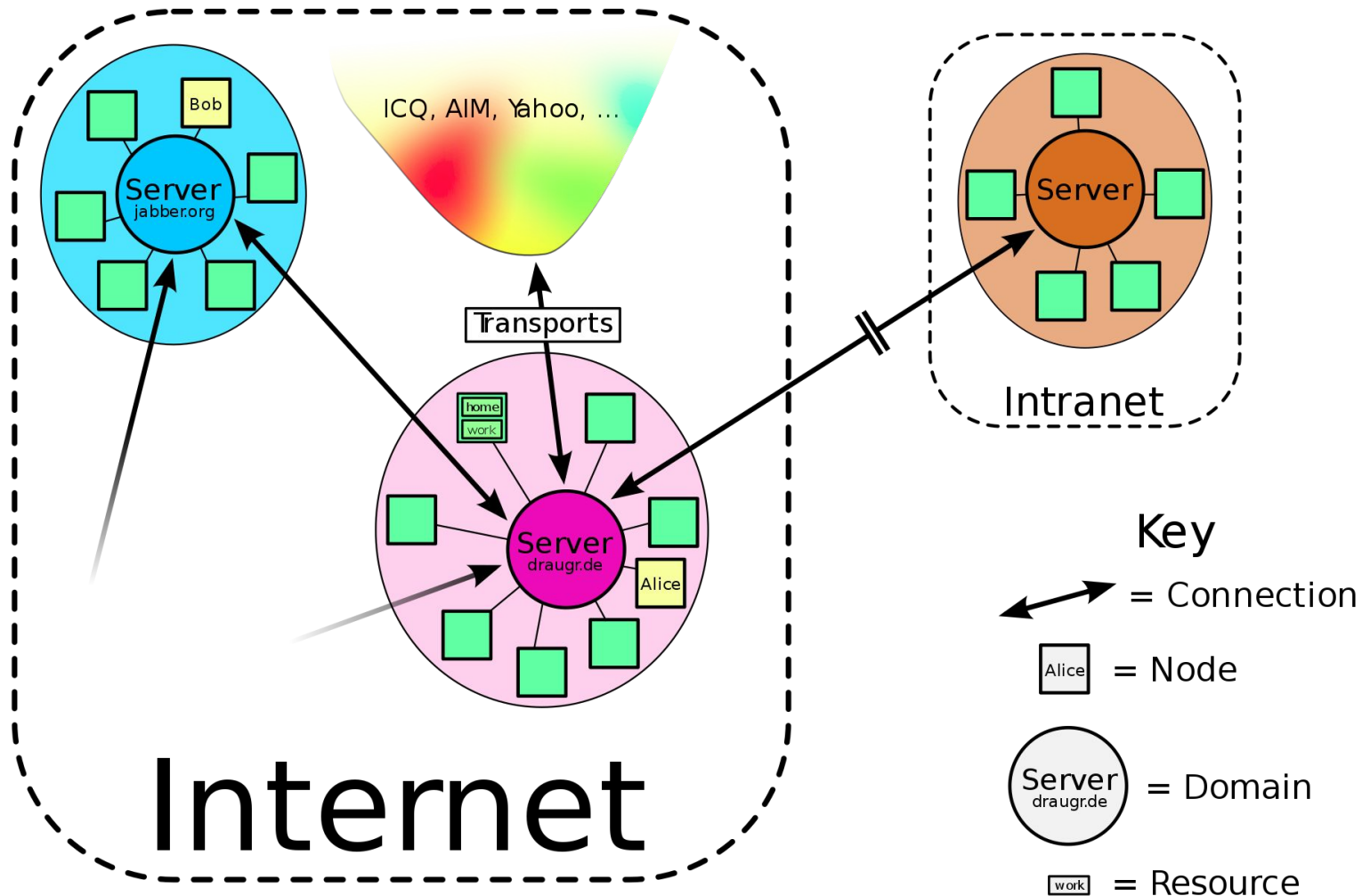
- [-] Текстовый.
- [-] Простой.

eXtensible Messaging and Presence Protocol

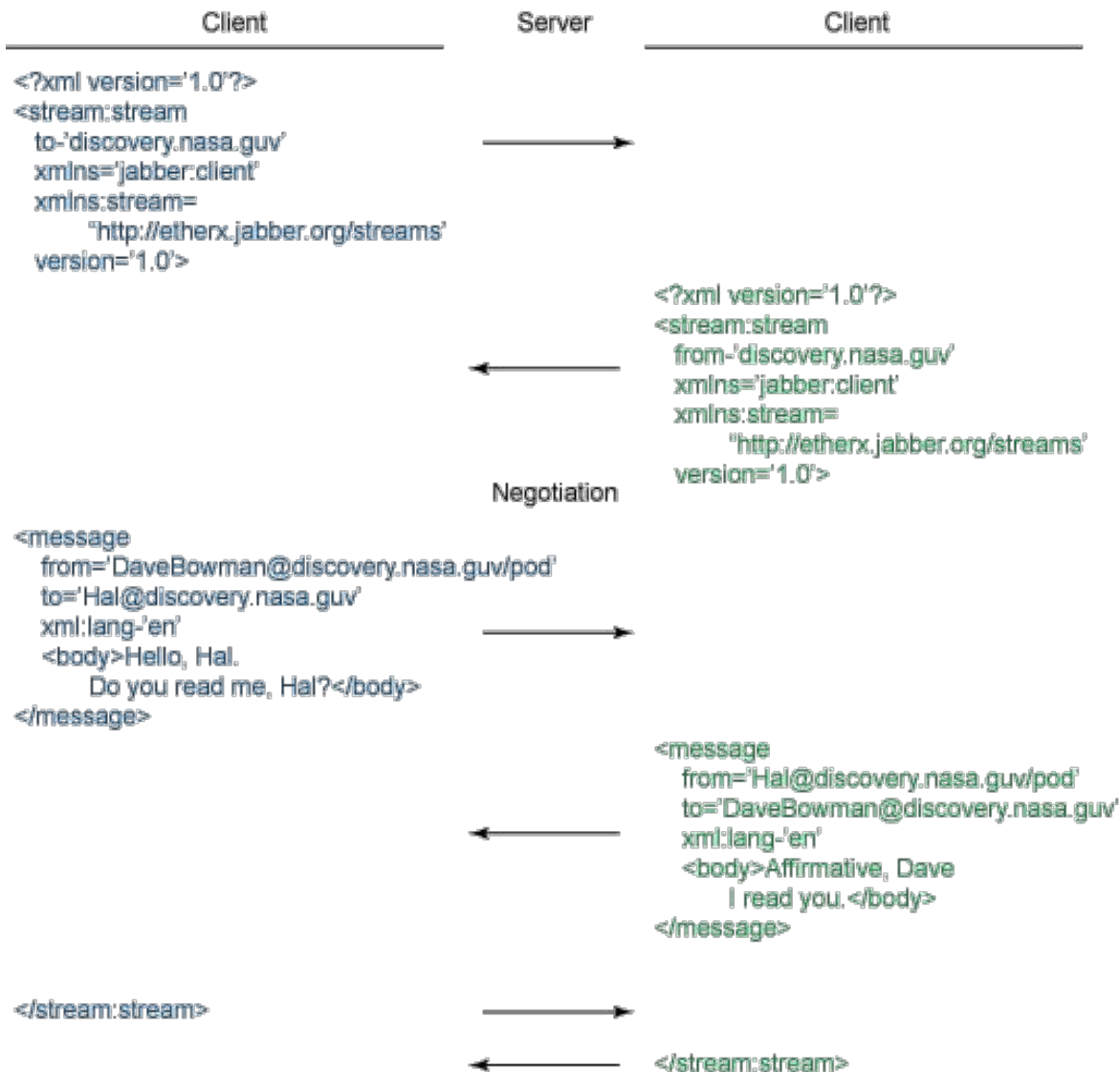
- Старое название – Jabber.
- Изначально предназначен для обмена сообщениями и информацией о присутствии.
- Прикладного уровня, может работать “поверх” TCP или HTTP.
- Текстовый, основан на XML.
- Изначально спроектирован расширяемым – поддерживает передачу голоса, видео, файлов и т. д.
- Для интеграции с другими протоколами есть механизм *транспортов* (transports).



Архитектура XMPP



Обмен сообщениями XMPP



- Очень много клиентских библиотек.
- Можно организовать отправку сообщений “вручную”:

```
String connect = "<?xml version='1.0'?> "  
    + "<stream:stream "  
    + "to='jabber.ru' version='1.0' "  
    + "xmlns='jabber:client' "  
    + "xmlns:stream='http://etherx.jabber.org/streams'>";  
  
String msg = "<starttls xmlns=\"urn:ietf:params:xml:ns:xmpp-tls\">";  
  
try {  
    InetAddress address = InetAddress.getByName(host);  
    Socket socket = new Socket(address, port);  
    socket.setKeepAlive(true);  
  
    OutputStream os = socket.getOutputStream();  
    OutputStreamWriter osw = new OutputStreamWriter(os);  
    BufferedWriter bw = new BufferedWriter(osw);  
    bw.write(connect);  
    bw.flush();  
    System.out.println("Message sent to the server : " + connect);  
}
```

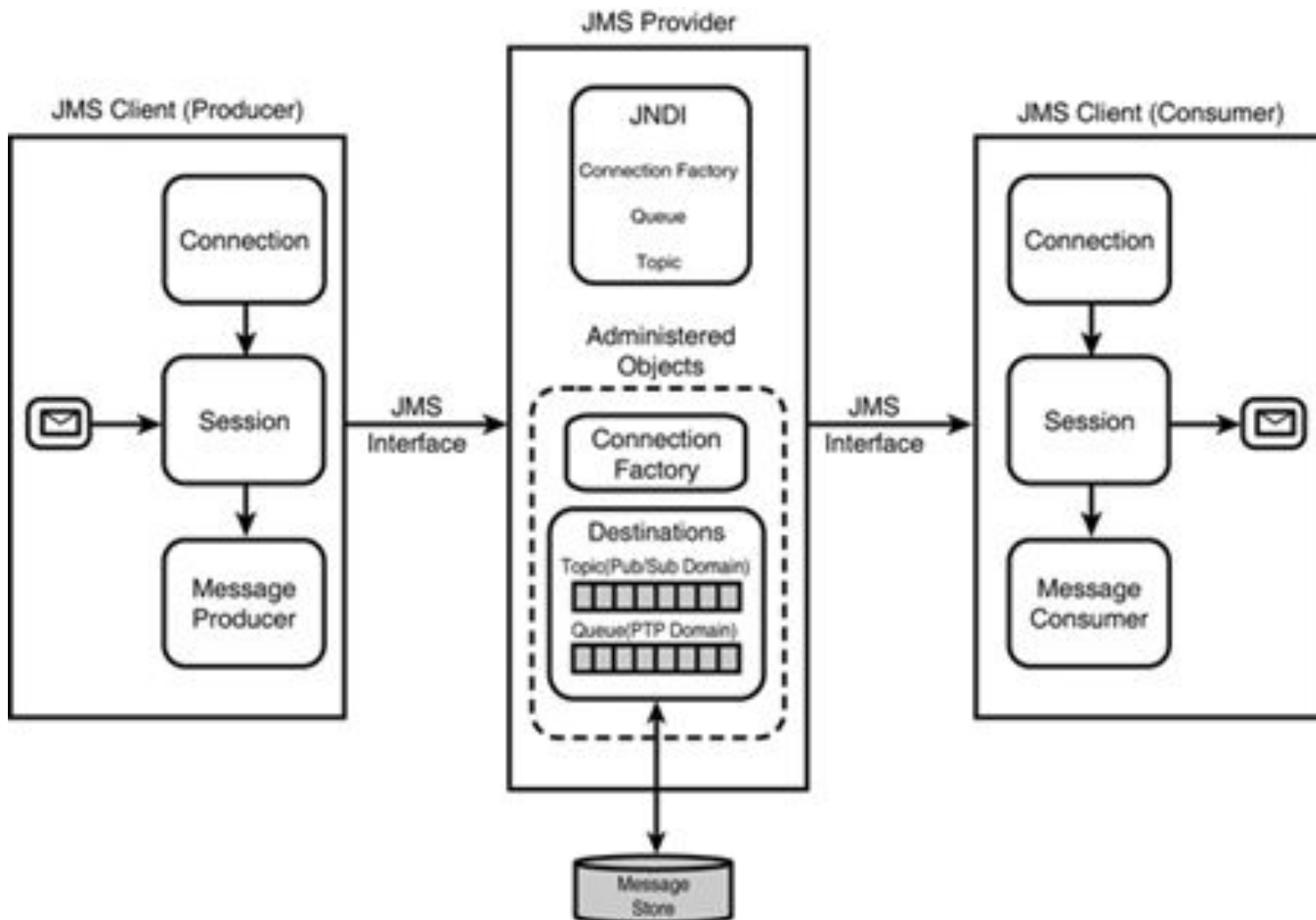
Преимущества и недостатки XML

- **[+]** Гибкий.
- **[+]** Мощный.
- **[+]** Расширяемый.
- **[+]** Текстовый.
- **[+]** Популярный, хорошая поддержка на уровне библиотек.
- **[-]** Текстовый.
- **[-]** У XML читаемость хуже, чем у plain text.
- **[-]** Достаточно “тяжеловесный”.
- **[-]** Предназначен для IM – “забивание гвоздей микроскопом”.

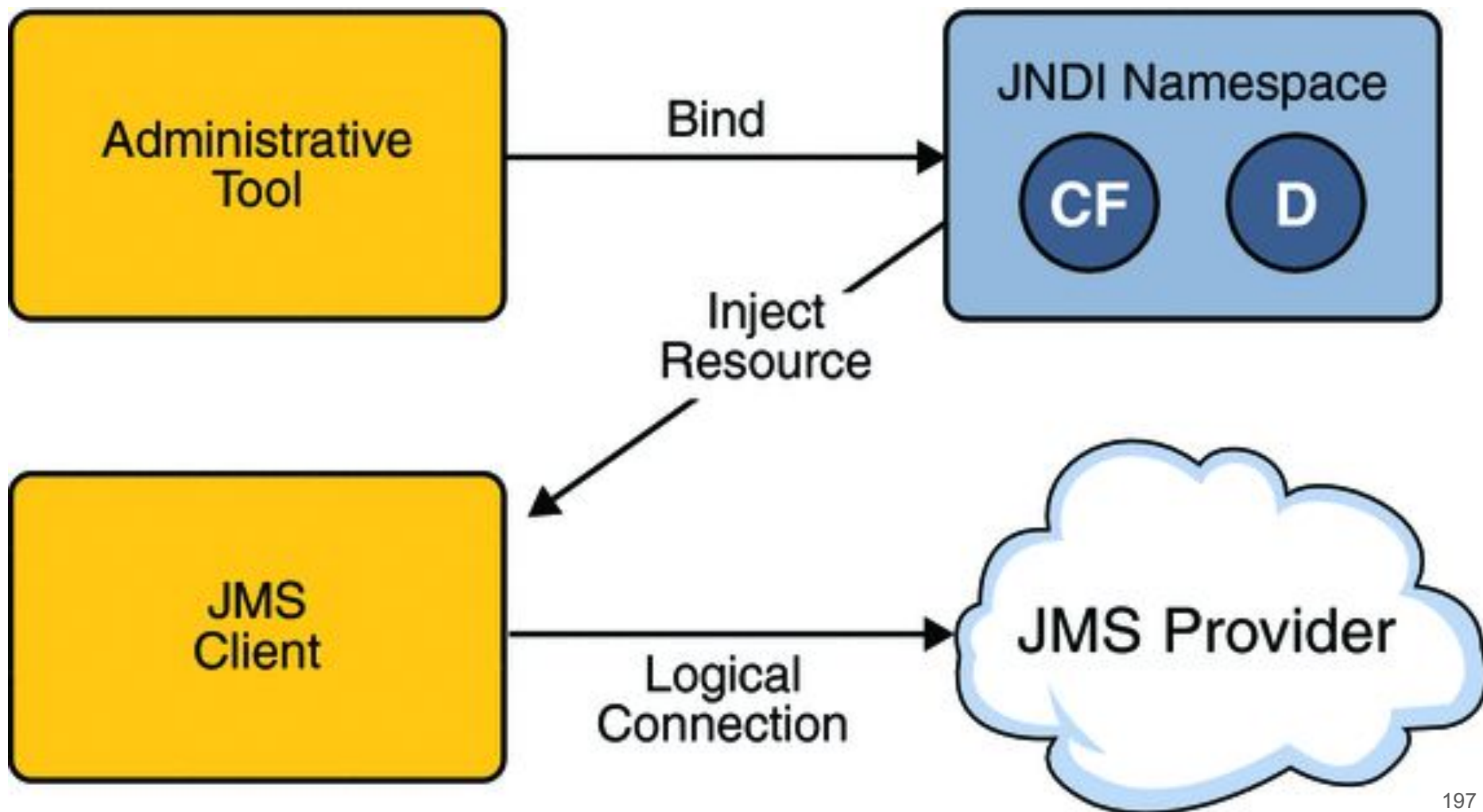
Jakarta Messaging

- Старое название – Java Message Service.
- Спецификация API для обмена сообщениями в составе платформы Jakarta / Java EE.
- Широко используется при асинхронном выполнении задач.
- Позволяет инкапсулировать инфраструктурное ПО и протоколы обмена сообщениями за универсальным API.

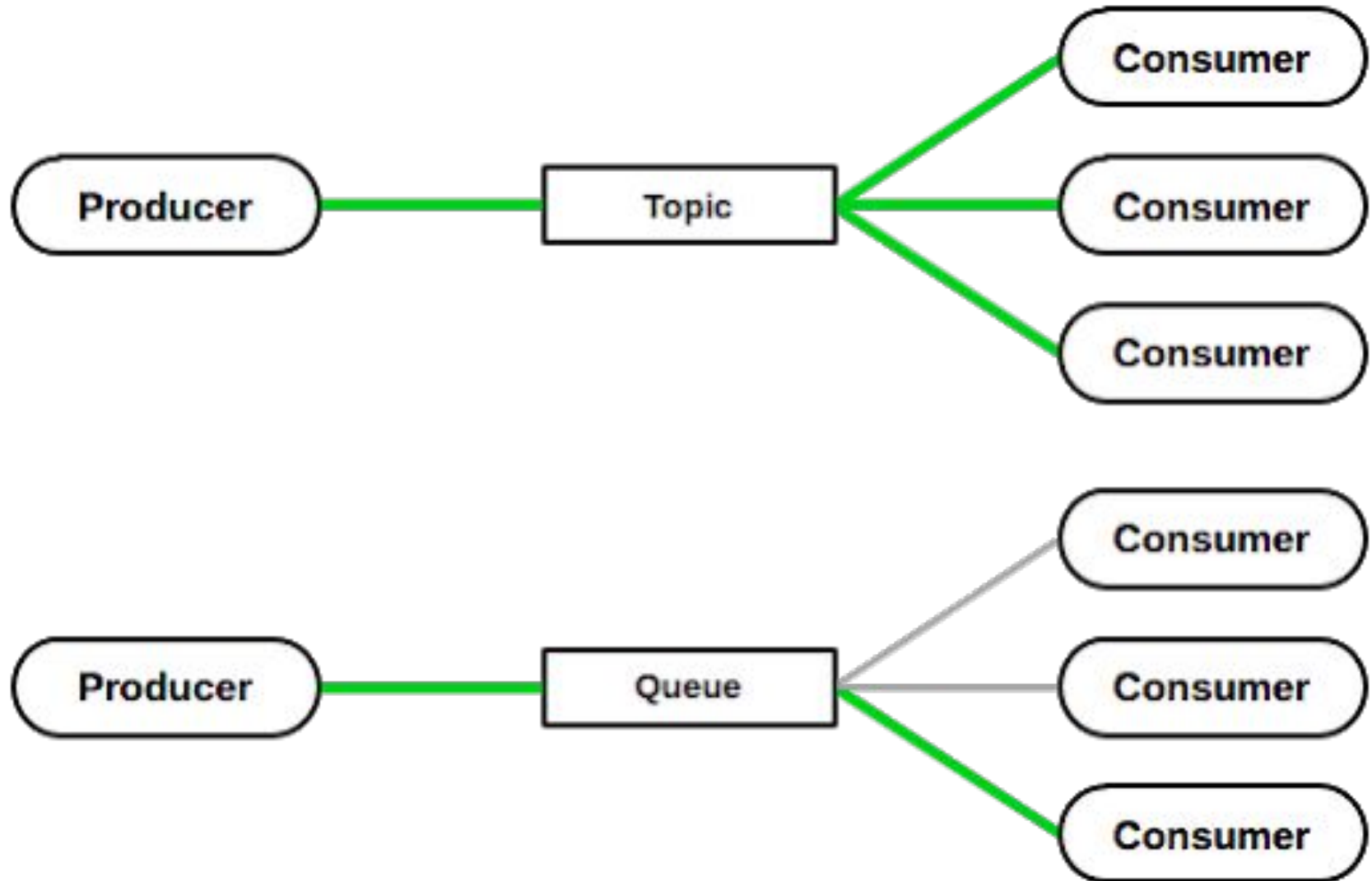
Архитектура JMS



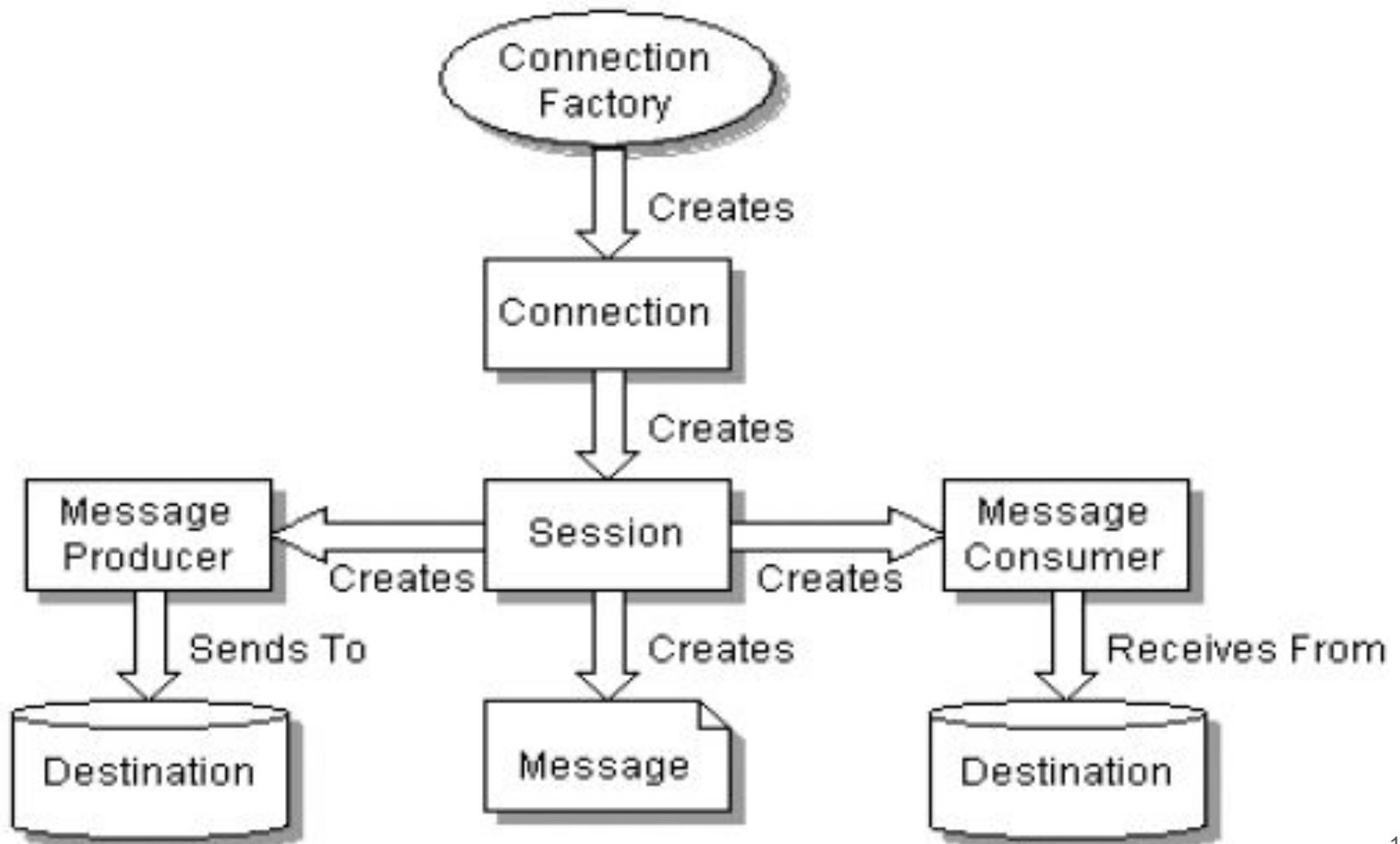
Взаимодействие с провайдером



Topic vs Queue



JMS API





Отправка сообщения

```
Context ctx = new InitialContext();  
ConnectionFactory connFactory =  
    (ConnectionFactory)  
        ctx.lookup("jms/QueueConnectionFactory");  
Connection connection =  
    connFactory.createConnection();  
Session session = connection.createSession(false,  
Session.AUTO_ACKNOWLEDGE);  
Destination dest =  
    (Queue) ctx.lookup("jms/SomeQueue");  
MessageProducer producer =  
    session.createProducer(dest);  
producer.send(message);
```



Получение сообщения

```
Context ctx = new InitialContext();  
ConnectionFactory connFactory =  
    (ConnectionFactory)  
        ctx.lookup("jms/QueueConnectionFactory");  
Connection connection =  
    connFactory.createConnection();  
Session session = connection.createSession(false,  
    Session.AUTO_ACKNOWLEDGE);  
Destination dest =  
    (Queue) ctx.lookup("jms/SomeQueue");  
MessageConsumer consumer =  
    session.createConsumer(dest);  
MessageListener listener = new MyListener();  
consumer.setMessageListener(listener);
```

JMS в Spring

- Поддерживается, есть API и аннотации для отправки и получения сообщений.
- Пример обработчика сообщений с очереди:

```
package hello;

import org.springframework.jms.annotation.JmsListener;
import org.springframework.stereotype.Component;

@Component
public class Receiver {

    @JmsListener(destination = "mailbox", containerFactory = "myFactory")
    public void receiveMessage>Email email) {
        System.out.println("Received <" + email + ">");
    }

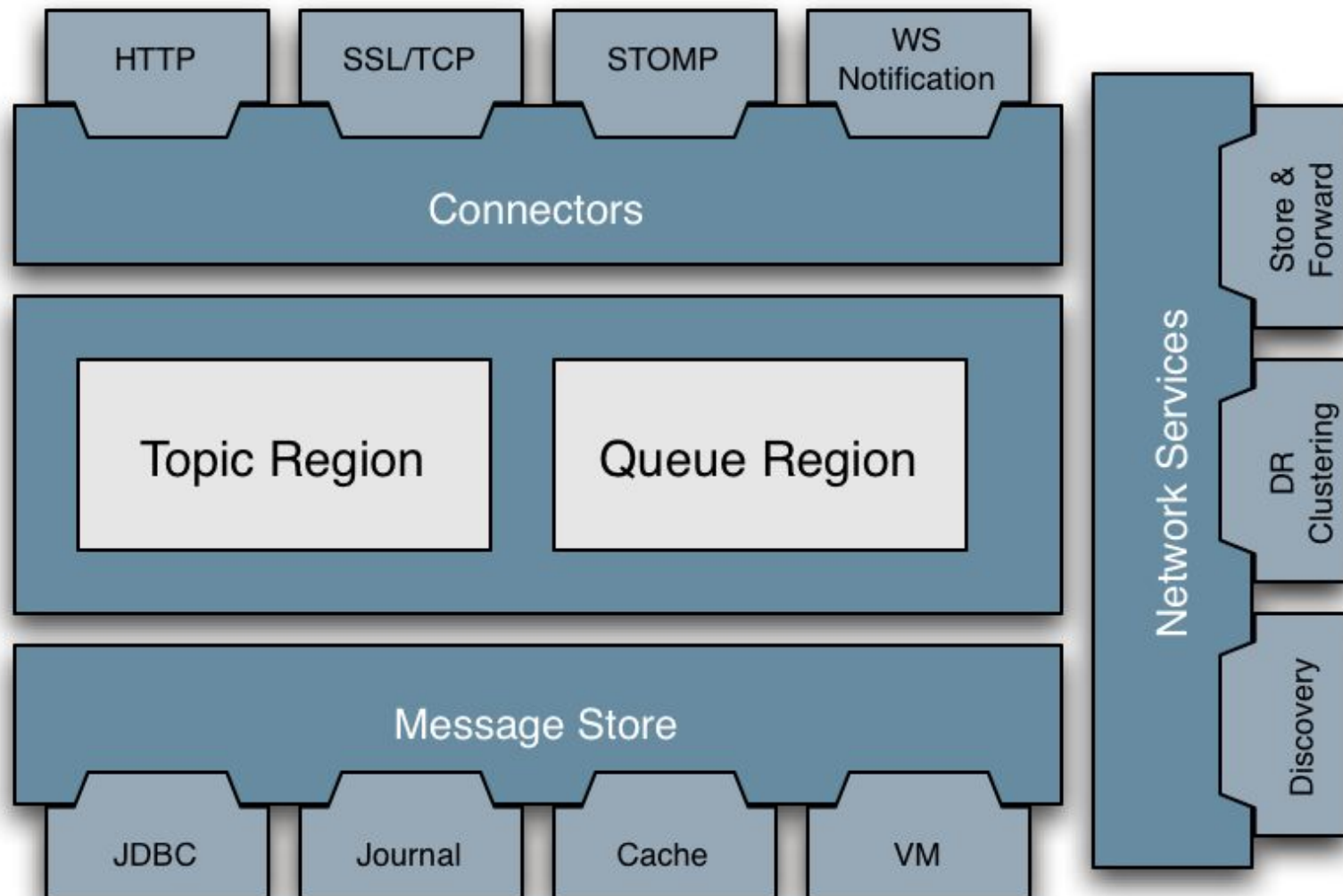
}
```

11. Системы асинхронного выполнения задач

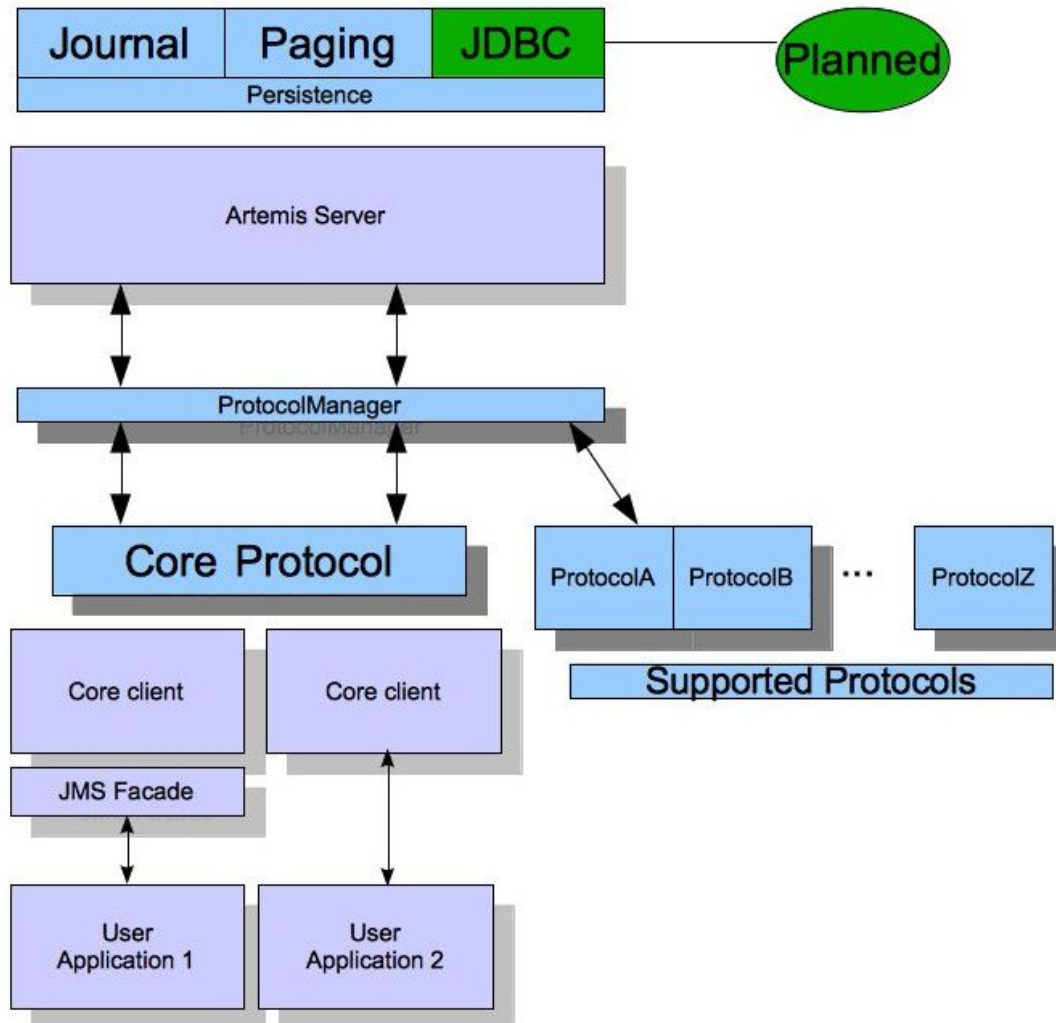
ActiveMQ (Classic & Artemis)

- Брокер сообщений от Apache.
- OpenSource (ASL), написан на Java.
- Первая версия вышла в 2006 г., актуальная (5.17.0) – в марте 2022 г.
- “Един в двух лицах” – есть ещё ActiveMQ Artemis на кодовой базе HornetQ.
- Полностью совместим со спецификацией JMS (1.1 – Classic, 2.0 – Artemis).
- “Родной” протокол – OpenWire.
- Основные поддерживаемые протоколы – AMQP, STOMP и MQTT.

Архитектура ActiveMQ Classic



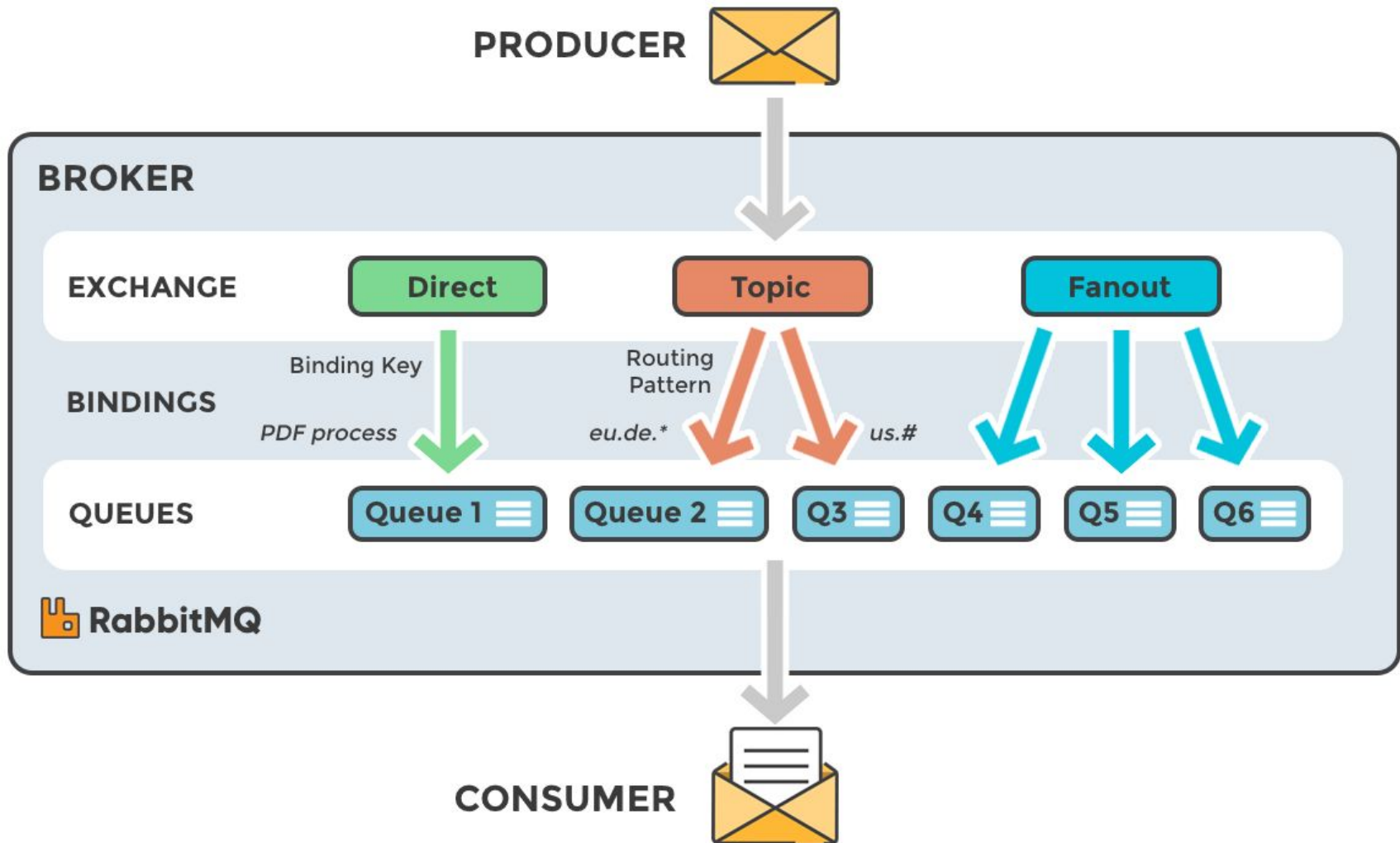
Архитектура ActiveMQ Artemis



RabbitMQ

- Брокер сообщений с открытым исходным кодом (лицензия MPL).
- Написан на Erlang.
- Первая версия вышла в 2007 г., актуальная (3.9.14) – в марте 2022 г.
- “Родной” протокол – AMQP, есть поддержка “из коробки” STOMP, MQTT и HTTP.
- Совместимость с JMS (1.1) обеспечивается с помощью дополнительного плагина.
- Более “легковесный” по сравнению с ActiveMQ, чаще используется при разработке микросервисов.

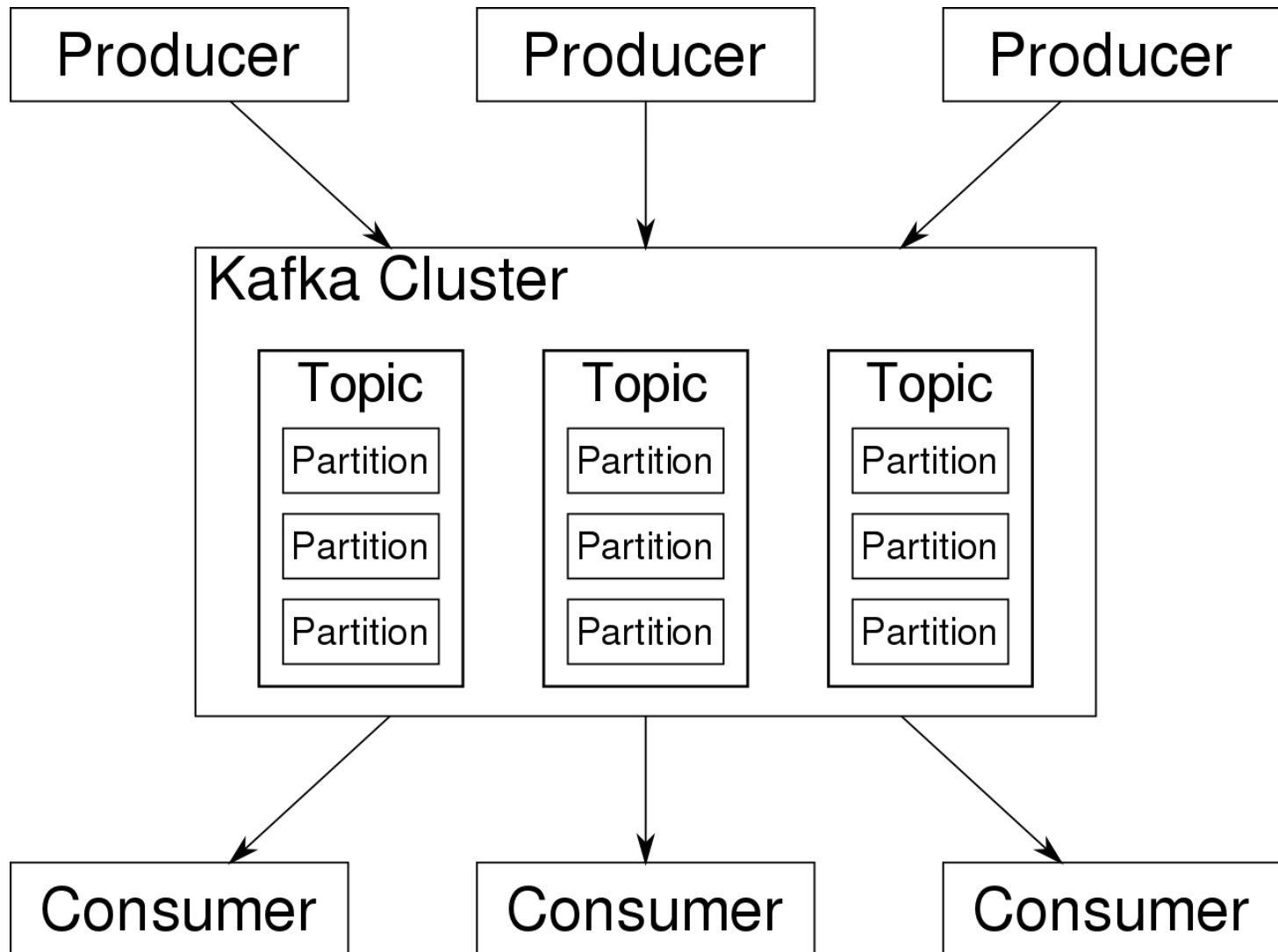
Архитектура RabbitMQ



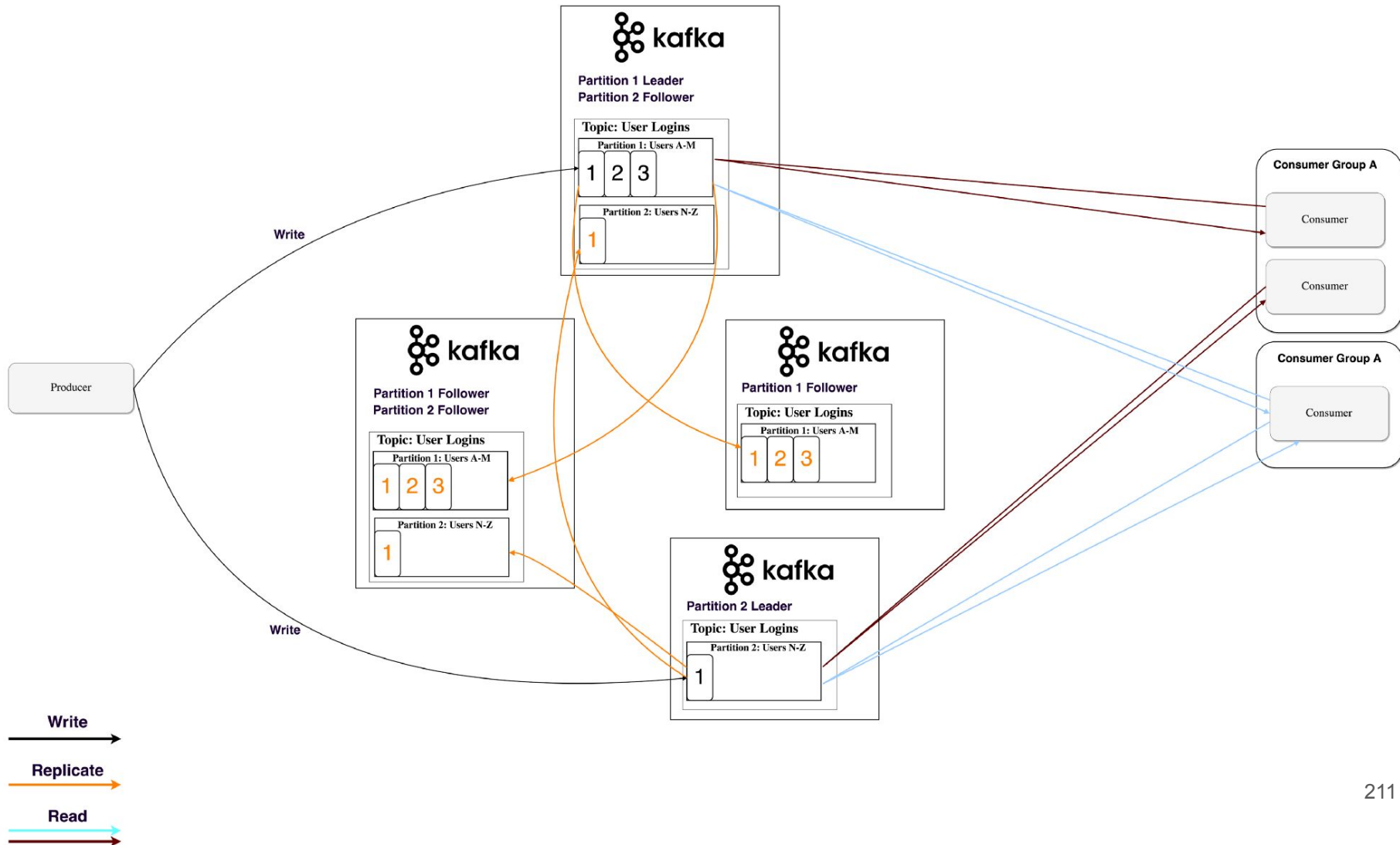
Kafka

- Ещё один брокер сообщений от Apache (изначально разработан LinkedIn).
- OpenSource (ASL 2.0), написан на Java и Scala.
- Первая версия вышла в 2014 г., актуальная (3.1.0) – в январе 2022 г.
- Изначально спроектирован как распределённая и горизонтально масштабируемая система.
- Сообщения хранятся в структуре данных, сходной с журналом транзакций в СУБД.
- Поддерживается только модель доставки “Подписка”!
- Не поддерживает JMS.

Архитектура Kafka



Кластеризация и репликация сообщений в Kafka



12. Планировщики задач

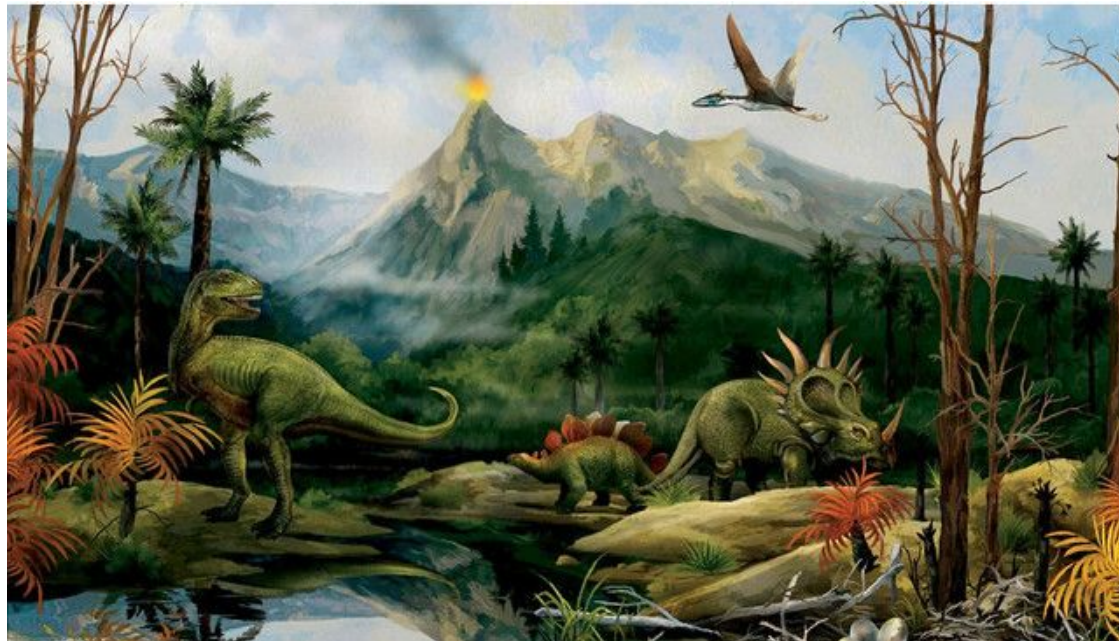
Кто такие и что делают

- Управляют *очередью задач* – определяют порядок исполнения на основе каких-либо критериев.
- Набор критериев может быть любым – приоритеты задач, загрузка системы и т.д.
- Могут управлять очередью задач в масштабе распределённого окружения – например, кластера.
- Примеры решений – Portable Batch System (PBS), Slurm Workload Manager, Platform LSF, Univa Grid Engine, Moab, LoadLeveler, Condor, OpenLava, ProActive Workflows & Scheduling.

Системы управления периодическими задачами

- Частный случай планировщиков задач.
- Нужны, если требуется выполнять задачи не “по требованию”, а “по расписанию”.
- Реализацией может быть специальная программа или модуль в составе инфраструктурного ПО.
- Могут находиться на разных “уровнях” инфраструктуры ПО.
- Правила запуска обычно задаются в конфигурации конкретной задачи или пользователя.

- Классическая программа для периодического выполнения задач в ОС *NIX.
- Первая версия написана в мае 1975 г. Кеном Томпсоном.
- Название происходит от греческого слова *χρόνος* (время).
- В системах на базе `systemd` в настоящее время практически замещён встроенным `systemd.timer`.



Файлы crontab

- Задают правила выполнения периодических задач утилитой cron.
- Основной файл конфигурации – `/etc/crontab`.
- Общесистемные файлы `crontab` находятся в каталоге `/etc/cron.d/`, а также в каталогах `/etc/cron.daily/`, `/etc/cron.weekly/` и `/etc/cron.monthly/`.
- У каждого пользователя есть свой файл, они обычно находятся в каталогах `/usr/spool/cron/crontabs/` и/или `/var/spool/cron/tabs/`.
- Файлы можно редактировать вручную, но рекомендуется вместо этого использовать команду `crontab -e`.

Содержимое файла crontab

- Каждая команда – на отдельной строке.
- В каждой строке 6 колонок, отделённых символами пробела или табуляции.
- Формат строки:

```
# _____ минута (0 - 59)
# | _____ час (0 - 23)
# | | _____ день месяца (1 - 31)
# | | | _____ месяц (1 - 12)
# | | | | _____ день недели (0 - 6) (0 -- воскресенье, 6 -- суббота;
# | | | | | 7 -- тоже воскресенье на части систем)
# | | | | |
# | | | | |
# * * * * * <command to execute>
```

Правила записи условий

- Все условия применяются по логическому “И” (за исключением совместного использования условий “день недели” и “день месяца”).
- Все файлы `crontab` должны заканчиваться пустой строкой.
- Вывод команд отправляется пользователю, от имени которого производился запуск.
- Если в файле `crontab` до команды расположена директива `MAILTO=<new@email>`, то выводимые данные будут отправляться на указанный адрес.

Команды управления содержимым crontab

- `crontab -l` – вывести содержимое текущего файла расписания.
- `crontab -r` – удаление текущего файла расписания.
- `crontab -e` – редактирование текущего файла расписания.

Пример файла crontab



विकिपीडिया
एक मुक्त ज्ञानकोश

```
# как обычно, с символа '#' начинаются комментарии
# в качестве командного интерпретатора использовать /bin/sh
SHELL=/bin/sh
# результаты работы отправлять по этому адресу
MAILTO=paul@example.org
# добавить в PATH домашний каталог пользователя
PATH=/bin:/usr/bin:/home/paul/bin

#### Здесь начинаются задания
# выполнять каждый день в 0 часов 5 минут, результат складывать в log/daily
5 0 * * * $HOME/bin/daily.job >> $HOME/log/daily 2>&1
# выполнять 1 числа каждого месяца в 14 часов 15 минут
15 14 1 * * $HOME/bin/monthly
# каждый рабочий день в 22:00
0 22 * * 1-5 echo "Пора домой" | mail -s "Уже 22:00" john

23 */2 * * * echo "Выполняется в 0:23, 2:23, 4:23 и т. д."
5 4 * * sun echo "Выполняется в 4:05 в воскресенье"
0 0 1 1 * echo "С новым годом!"
15 10,13 * * 1,4 echo "Эта надпись выводится в понедельник и четверг в 10:15 и 13:15"
0-59 * * * * echo "Выполняется каждую минуту"
0-59/2 * * * * echo "Выполняется по чётным минутам"
1-59/2 * * * * echo "Выполняется по нечётным минутам"
# каждые 5 минут
*/5 * * * * echo "Прошло пять минут"
# каждое первое воскресенье каждого месяца. -eq 7 это код дня недели, т. е. 1 -> понедельник, 2 -> вторник и т. д.
0 1 1-7 * * [ "$(date '+%u')" -eq 7 ] && echo "Эта надпись выводится каждое первое воскресенье каждого месяца в 1:00"
```

- Systemd – подсистема управления службами в Linux:
 - Первая версия вышла в 2010 г.
 - Реализована по умолчанию практически во всех современных дистрибутивах.
 - Практически вытеснила службы init.
- Таймеры – файлы служб systemd, запуск которых осуществляется по расписанию.

Возможности таймеров

- Поддерживаются все правила запуска служб, реализованные в ctop.
- Реализовано два вида таймеров:
 - Таймеры реального времени – аналог задач ctop.
 - Монотонные таймеры – запускаются после определённого времени по отношению к какому-либо событию.

Конфигурация таймеров

- Каждому таймеру, помимо файла `.service`, нужно создать файл с суффиксом `.timer`.
- Файл `.timer` определяет правила запуска таймера.
- Файл таймера содержит дополнительный раздел `[Timer]`.
- Список всех запущенных таймеров можно посмотреть с помощью команды `systemctl list-timers`.

Пример файла `.timer` для таймера реального времени

```
/etc/systemd/system/foo.timer
```

```
[Unit]  
Description=Run foo weekly
```

```
[Timer]  
OnCalendar=weekly  
Persistent=true
```

```
[Install]  
WantedBy=timers.target
```

```
OnCalendar=Mon,Tue *-*-01..04 12:00:00
```

Пример файла `.timer` для монотонного таймера

```
/etc/systemd/system/foo.timer
```

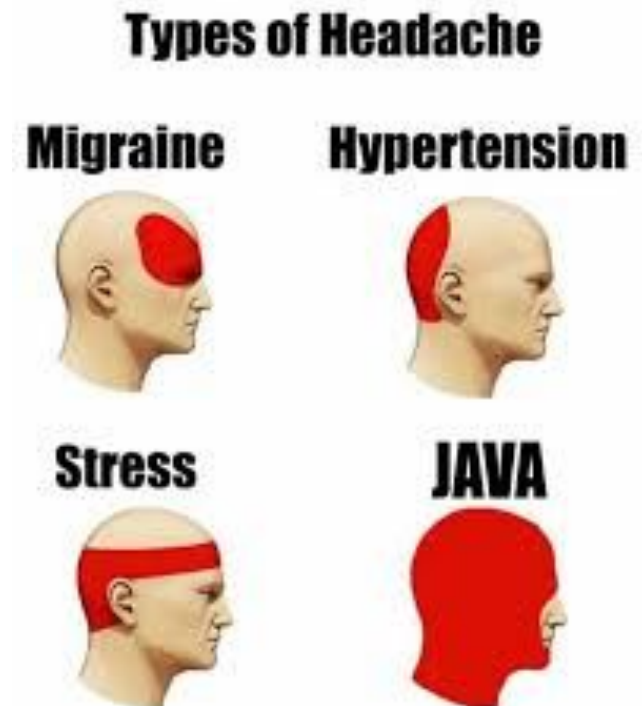
```
[Unit]  
Description=Run foo weekly and on boot
```

```
[Timer]  
OnBootSec=15min  
OnUnitActiveSec=1w
```

```
[Install]  
WantedBy=timers.target
```

Таймеры в Java

- Java SE:
`ScheduledExecutorService`.
- Java EE:
`EJB Timer Services`.
- Spring:
`@Scheduled`.
- Библиотеки –
тысячи их!



ScheduledExecutorService

```
private final ScheduledExecutorService scheduler = Executors
    .newScheduledThreadPool(1);

public void startScheduleTask() {
    /**
     * not using the taskHandle returned here, but it can be used to cancel
     * the task, or check if it done (for recurring tasks, that not
     * going to be very useful)
     */
    final ScheduledFuture<?> taskHandle = scheduler.scheduleAtFixedRate(
        new Runnable() {
            public void run() {
                try {
                    getDataFromDatabase();
                } catch (Exception ex) {
                    ex.printStackTrace(); //or logger would be better
                }
            }
        }, 0, 15, TimeUnit.MINUTES);
}
```

Таймер:

```
@Schedules ({
    @Schedule(dayOfMonth="Last"),
    @Schedule(dayOfWeek="Fri", hour="23")
})
public void doPeriodicCleanup() { ... }
```

Запуск по таймауту:

```
...
    public void setTimer(long intervalDuration) {
        logger.info("Setting a programmatic timeout for " +
            intervalDuration + " milliseconds from now.");
        Timer timer = timerService.createTimer(intervalDuration,
            "Created new programmatic timer");
    }

    @Timeout
    public void programmaticTimeout(Timer timer) {
        this.setLastProgrammaticTimeout(new Date());
        logger.info("Programmatic timeout occurred.");
    }
...

```

Фиксированное время запуска:

```
@Scheduled(fixedRate = 1000)
public void scheduleFixedRateTask() {
    System.out.println(
        "Fixed rate task - " + System.currentTimeMillis() / 1000);
}
```

Запуск по cron:

```
@Scheduled(cron = "0 15 10 15 * ?")
public void scheduleTaskUsingCronExpression() {

    long now = System.currentTimeMillis() / 1000;
    System.out.println(
        "schedule tasks using cron jobs - " + now);
}
```

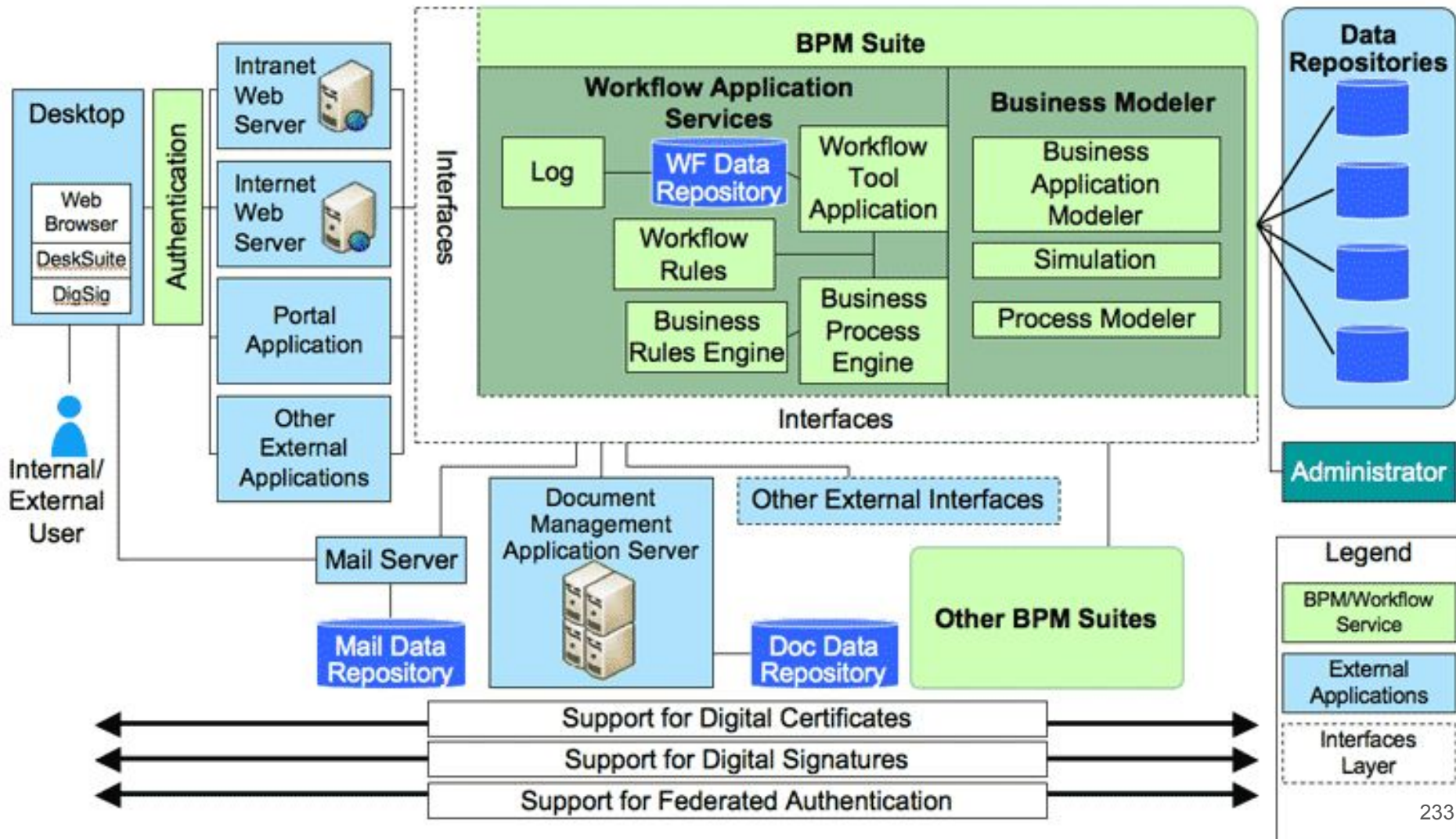
13. Системы управления бизнес-процессами

- Приложения, автоматизирующие управление бизнес-процессами предприятия.
- Позволяют задавать правила управления бизнес-процессом в виде диаграмм (например, BPMN).
- Могут быть отдельными приложениями, могут – библиотеками, встраиваемыми в ИС.
- Могут управлять только бизнес-логикой, могут ещё и генерировать интерфейс.

Использование BPMS: “за” и “против”

- [+] Гибкая адаптация бизнес-процессов под меняющиеся требования.
- [+] Возможна модификация бизнес-процесса без участия специалиста (теоретически).
- [+] Автоматизируемый процесс должен быть специфицирован.
- [-] Сгенерированный интерфейс обычно неудобен.
- [-] Генерируемая программно логика может быть неэффективной в плане быстрогодействия.
- [-] “Внедрение ради внедрения”.

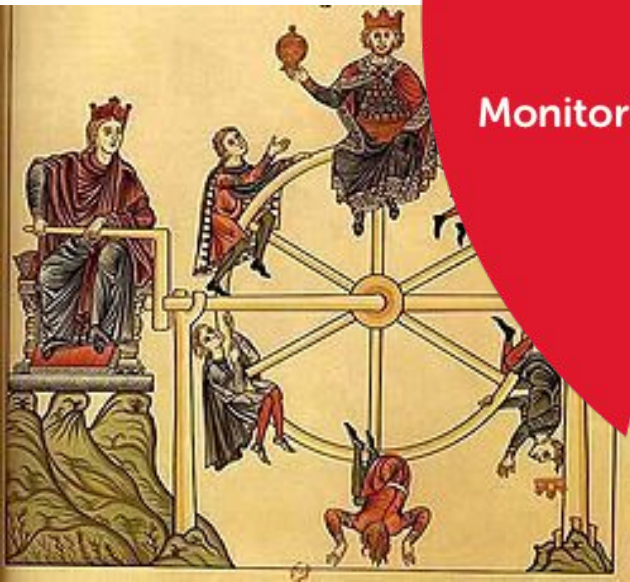
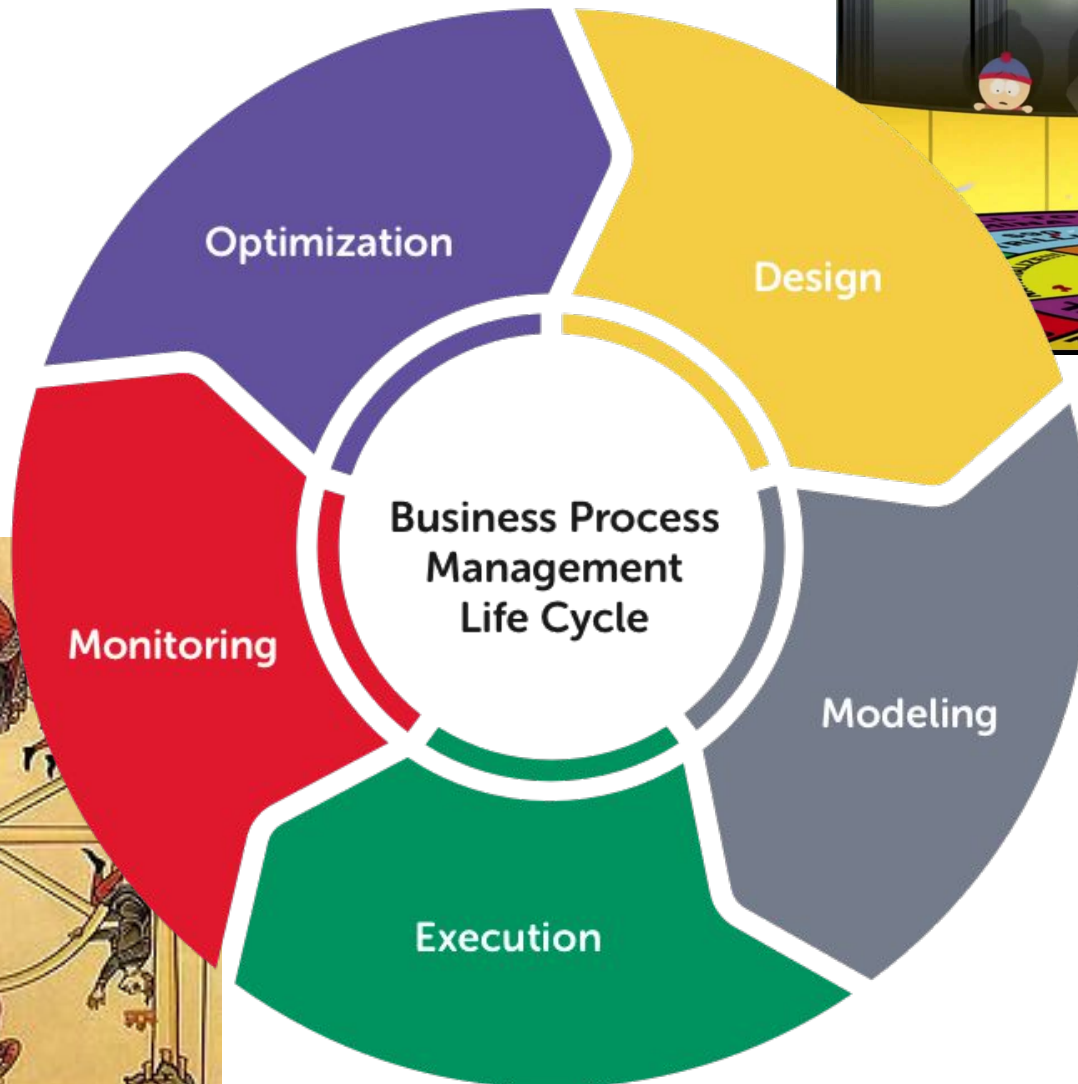
Архитектура BPMS



Ключевые компоненты BPMS

- Process engine – “движок” для моделирования и исполнения компонентов приложения, управляемых бизнес-правилами.
- Business analytics – формирует аналитику по работе бизнес-процесса, отчёты, дашборды и т.д.
- Content management – среда для управления документами, изображениями и прочими файлами.
- Collaboration tools – инструменты интеграции с подсистемами, “внешними” по отношению к BPMS.

Жизненный цикл BPM



Популярные BPMS

- Oracle BPM.
- SAP BPM.
- IBM BPM.
- JBoss BPM (jBPM).
- Camunda Platform.





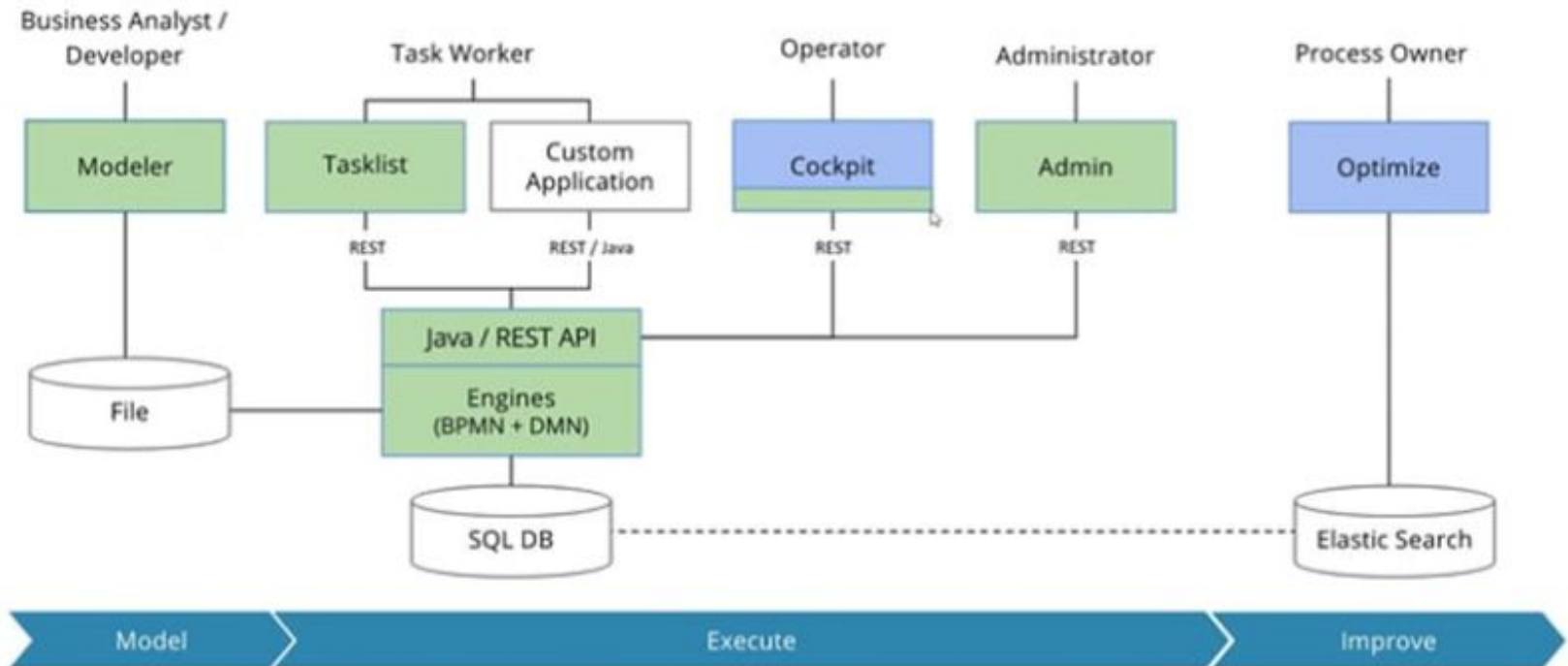
- BPMS с открытым исходным кодом от одноимённой компании.
- Написана на Java, разрабатывается с 2013 г.
- Исходная версия являлась форком BPMS Activiti, сейчас кодовая база существенно переработана.
- Состоит из набора программных продуктов, каждый из которых решает отдельную задачу.
- Распространяется по лицензии Apache, есть проприетарная версия с расширенными возможностями.
- Активно поддерживается, последняя версия – 8.0 от (апрель 2022, **не надо её использовать!**).

- Поддерживаемые форматы описания бизнес-процессов – BPMN 2.0, CMMN 1.1, DMN 1.1.
- Редактор BPMN (Modeler) – отдельное приложение, результат моделирования экспортируется в файл XML.
- Два варианта использования – отдельный сервис, либо модуль в составе информационной системы.
- Есть поддержка параллелизма.
- Есть поддержка транзакций.

Архитектура Camunda



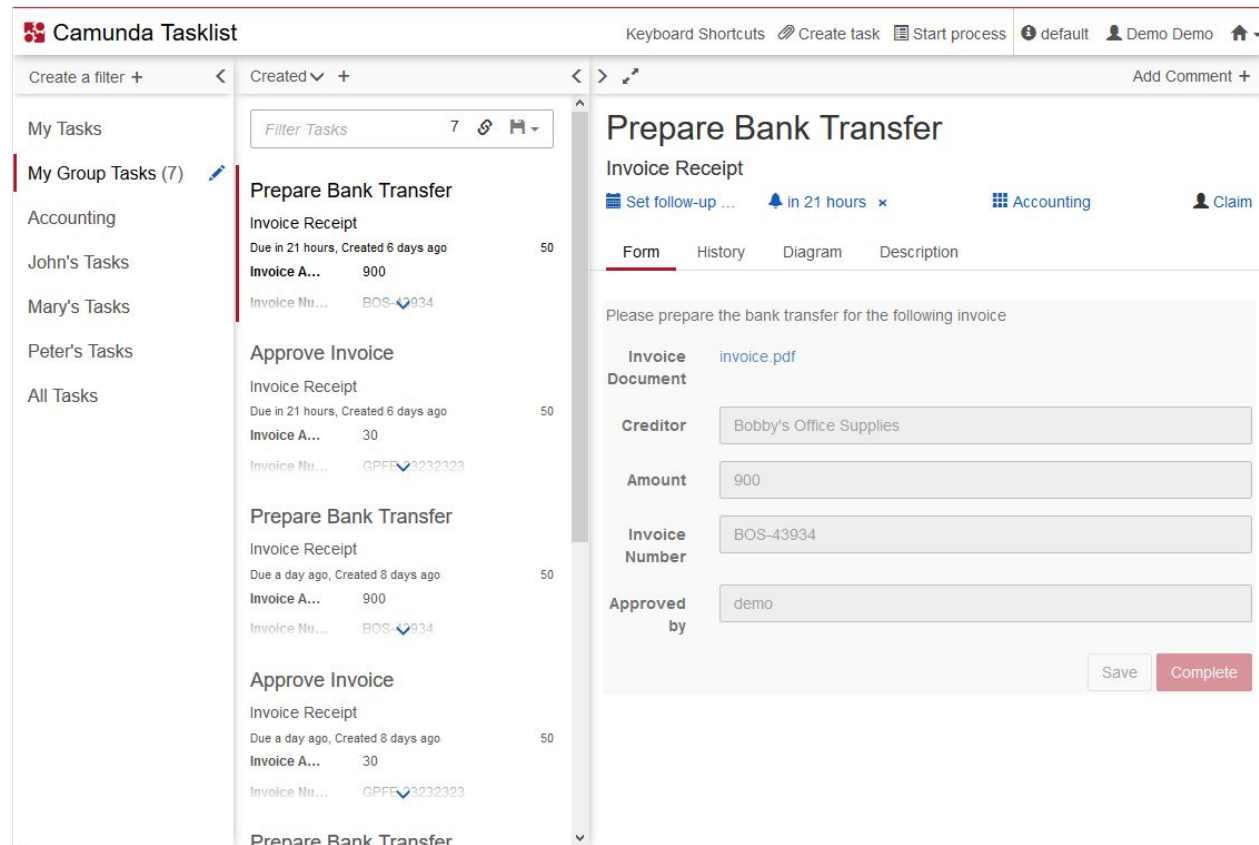
Community vs. Enterprise



- Приложение для создания моделей бизнес-процессов.
- Независим от других подсистем Camunda.
- В теории, может быть заменён другим редактором.
- Существует в веб-версии и в виде standalone-приложения.
- Выходные данные – “скомпилированный” bpmn-проект, который нужно “подложить” рантайму Camunda.

Tasklist

- Веб-приложение для управления *задачами*.
- Задачи формируются на пользователей в ходе выполнения бизнес-процесса.

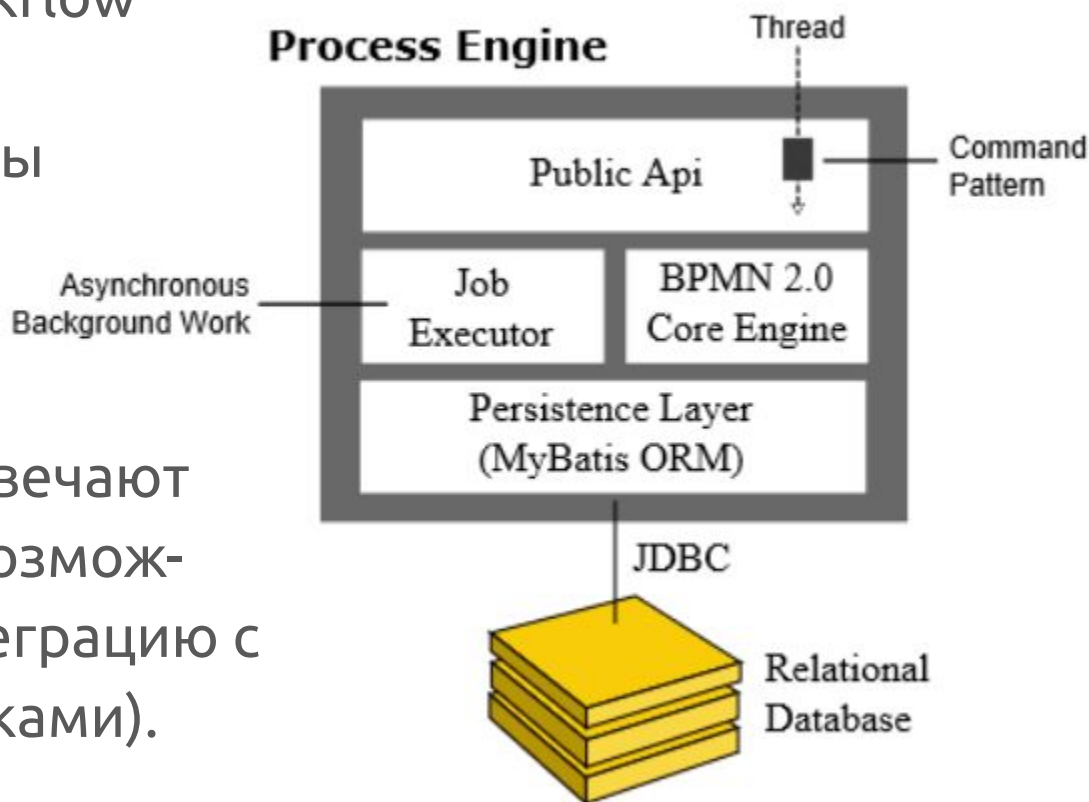


The screenshot displays the Camunda Tasklist web application. The interface is divided into several sections:

- Header:** Includes the Camunda logo, "Camunda Tasklist" title, and navigation links for "Keyboard Shortcuts", "Create task", "Start process", "default", "Demo Demo", and a home icon.
- Left Sidebar:** Contains a "Create a filter +" button and a list of task categories: "My Tasks", "My Group Tasks (7)", "Accounting", "John's Tasks", "Mary's Tasks", "Peter's Tasks", and "All Tasks".
- Task List:** A central list of tasks. The first task is "Prepare Bank Transfer" (Invoice Receipt), due in 21 hours, created 6 days ago, with a value of 50. It includes details like "Invoice A..." (900) and "Invoice Nu..." (BOS-43934). Other tasks include "Approve Invoice" and another "Prepare Bank Transfer".
- Task Detail View:** On the right, a detailed view for the "Prepare Bank Transfer" task is shown. It includes a form with fields for "Invoice Document" (invoice.pdf), "Creditor" (Bobby's Office Supplies), "Amount" (900), "Invoice Number" (BOS-43934), and "Approved by" (demo). Buttons for "Set follow-up...", "in 21 hours", "Accounting", and "Claim" are also present. At the bottom, there are "Save" and "Complete" buttons.

Движки бизнес-процессов (Process Engines)

- “Ядро” Camunda, отвечающее за интерпретацию бизнес-процесса.
- Их несколько – у каждой нотации свой. За BPMN “отвечает” BPMN Workflow Engine.
- Транслируют элементы бизнес-процесса в объекты Java, сохраняют информацию в БД, отвечают за дополнительные возможности (например, интеграцию с внешними фреймворками).



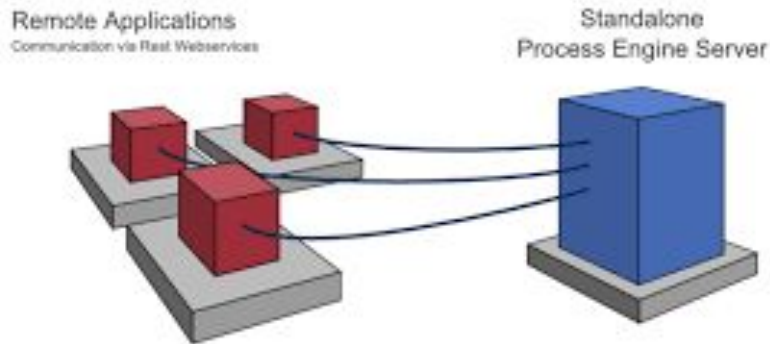
Другие приложения

- Cockpit – веб-приложение для просмотра состояния процессов.
- Admin – веб-приложение для управления пользователями и правами доступа.
- Optimize – (платное) приложение для анализа и оптимизации бизнес-процессов.

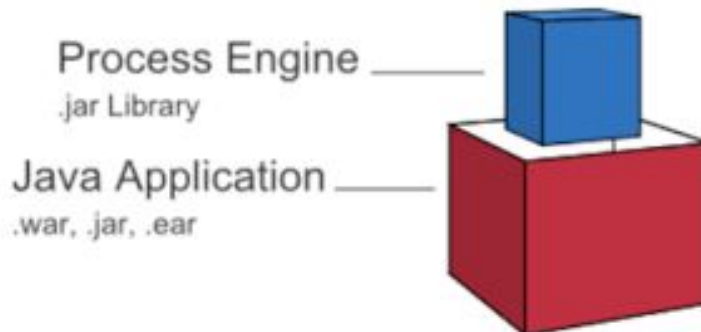


Установка и конфигурация

- Вариант 1: standalone-сервер.



- Вариант 2: встраивание в веб-приложение.



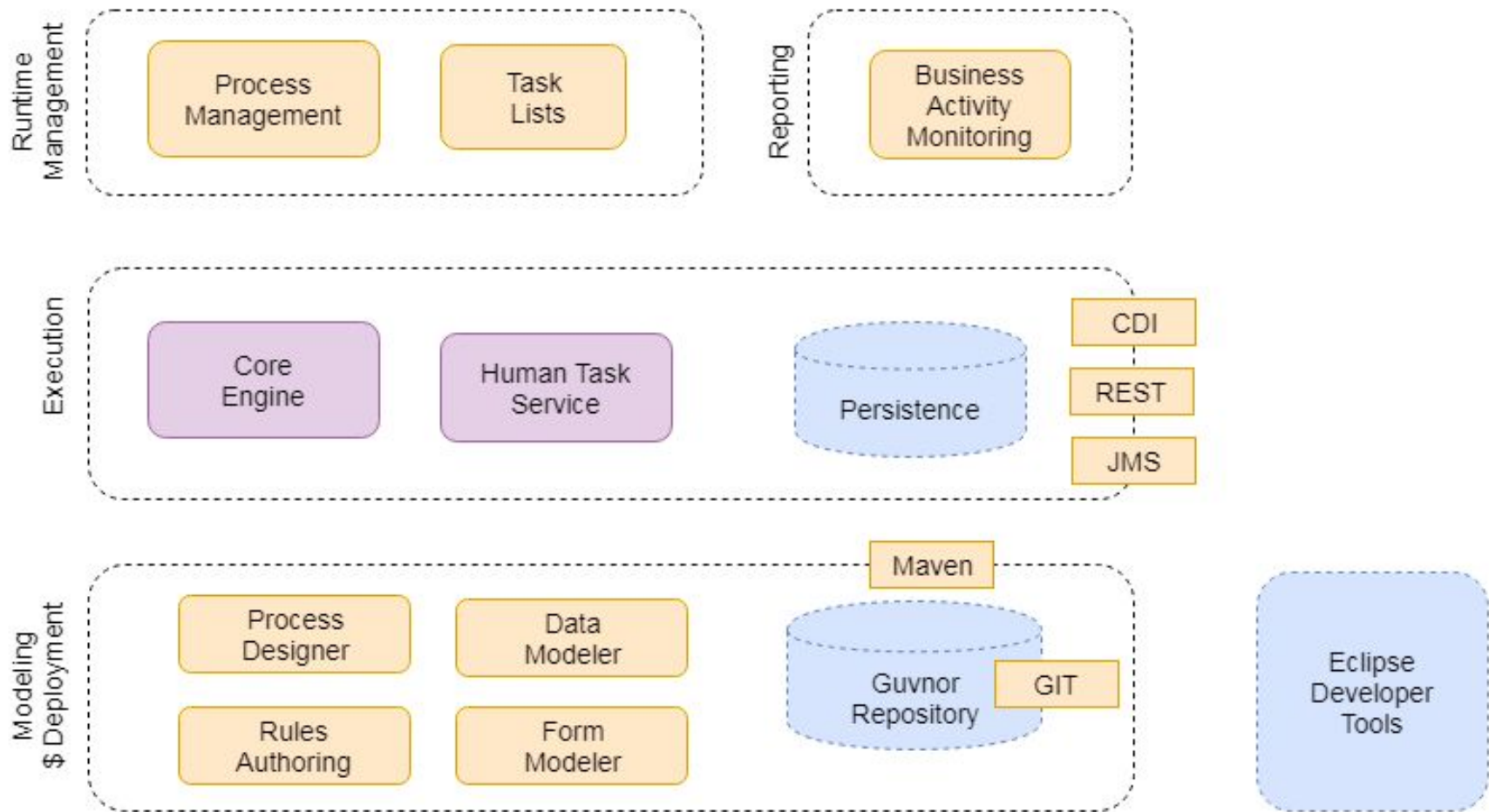
Приложение А. JBoss BPM

- BPMS с открытым исходным кодом от JBoss (спонсируется Red Hat).
- Написана на Java.
- Интегрируется со стеком технологий JBoss: развёртывается на WildFly, среда разработки бизнес-процессов реализована на базе Eclipse и т. д.
- Распространяется по лицензии ASL.
- Активно поддерживается, последняя версия – 7.47.0.Final от 04.12.2020.

Возможности jBPM

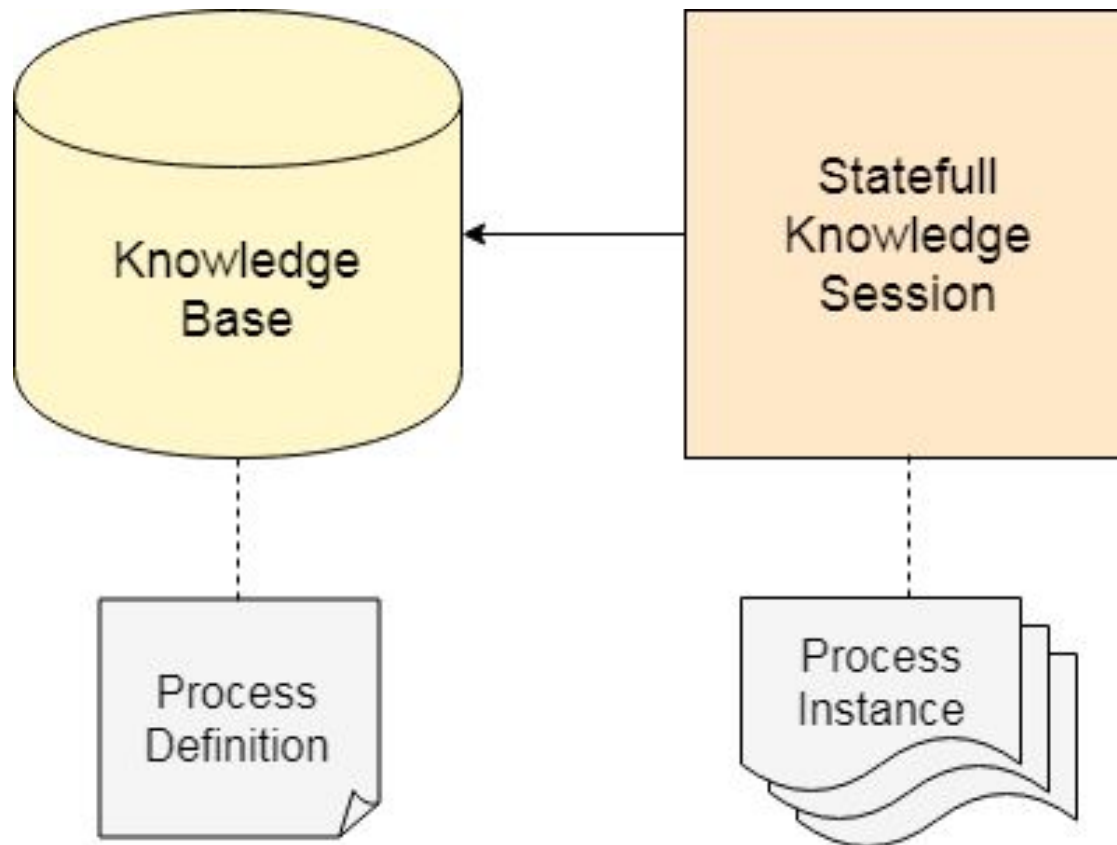
- Формат описания бизнес-процесса – BPMN 2.0.
- Два (на самом деле больше) формата редактора BPMN – на базе Eclipse и веб-приложение.
- Два варианта использования – отдельный сервис, либо модуль в составе информационной системы.
- Состояние процесса можно сохранять в БД, поддерживаются транзакции JTA.
- Есть интеграция “из коробки” с Apache Kafka.
- Есть поддержка параллелизма (“логического” и “технического”).

Архитектура jBPM

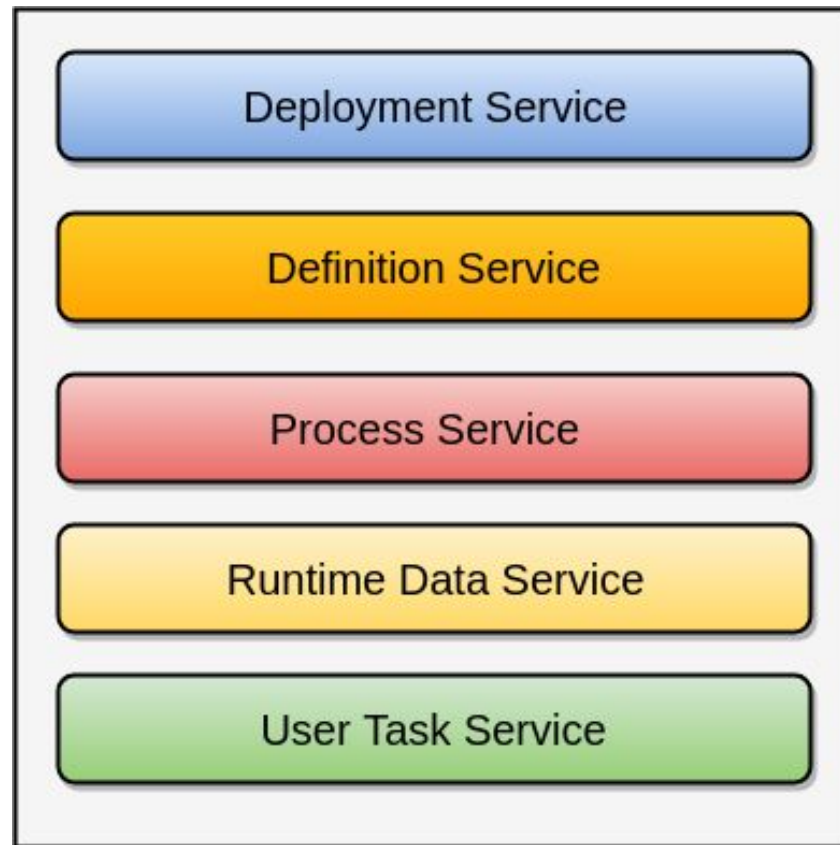


Knowledge Base

- Хранит информацию обо всех бизнес-процессах.
- Доступ осуществляется с помощью сессии.



Высокоуровневые API для взаимодействия с BPMS.



jBPM Services



```

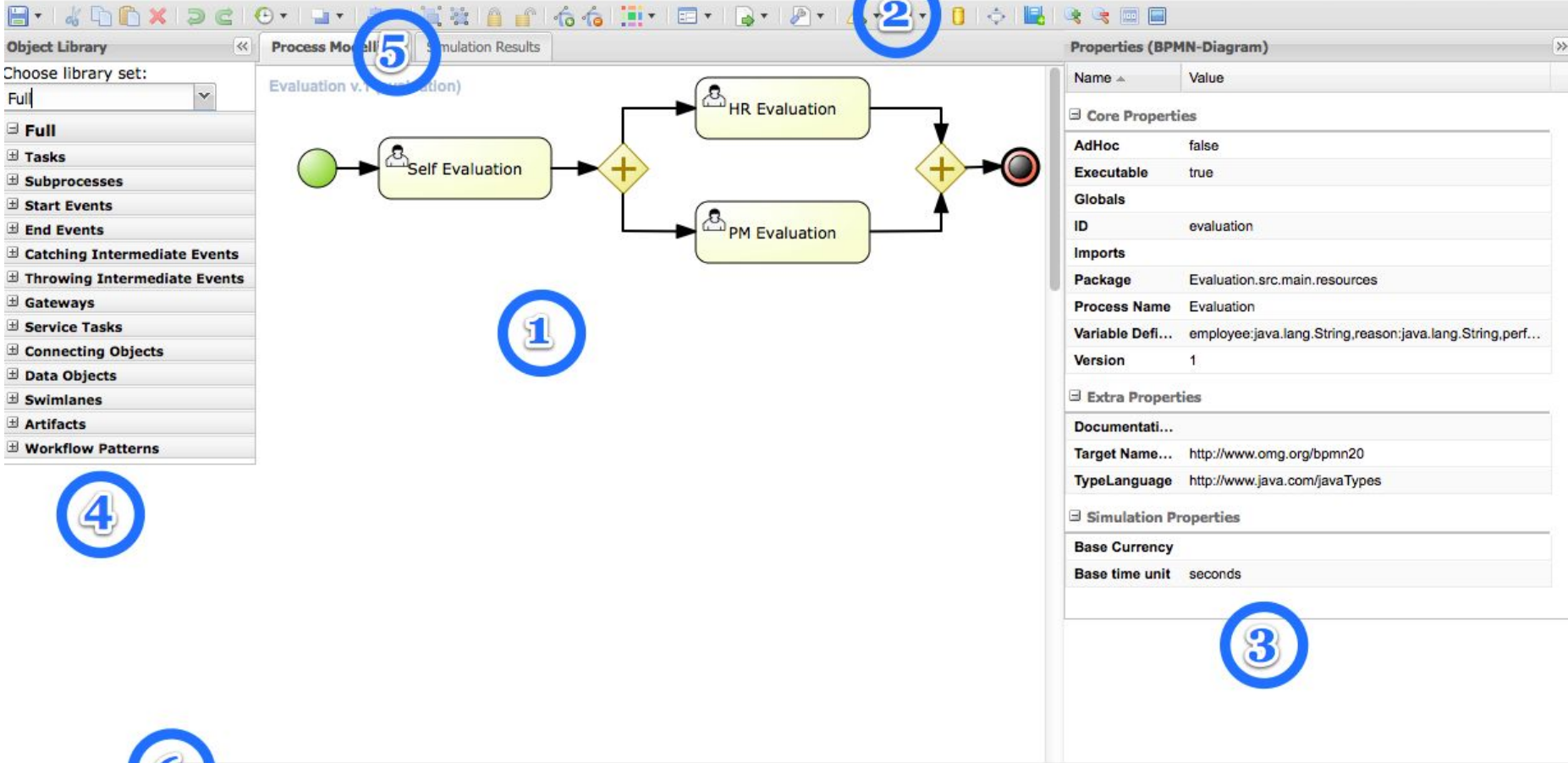
    graph TD
      Inactive((Inactive)) -- "[Task created, coord context obtained]  
Register task with coordinator" --> Created[Created]
      Created -- "(activate || nomination performed) &&  
single potential owner" --> Ready[Ready]
      Created -- "(activate || nomination performed) &&  
(multiple potential owners || work queue)" --> Ready
      Ready -- "suspend" --> Ready
      Ready -- "forward" --> Ready
      Ready -- "resume" --> Ready
      Ready -- "claim || delegate" --> Reserved[Reserved]
      Reserved -- "suspend" --> Reserved
      Reserved -- "resume" --> Reserved
      Reserved -- "delegate" --> Reserved
      Reserved -- "start" --> InProgress[InProgress]
      InProgress -- "suspend" --> InProgress
      InProgress -- "resume" --> InProgress
      InProgress -- "[Completion with response]  
Send result" --> Completed[Completed]
      InProgress -- "[Completion with fault response]  
Send application fault" --> Failed[Failed]
      InProgress -- "[Non-recoverable error]  
Send 'WS-HT fault'" --> Error[Error]
      InProgress -- "[WS-HT exit]  
Exit task" --> Exited[Exited]
      InProgress -- "[Skip && is Skippable]  
Send 'WS-HT skipped'" --> Obsolete[Obsolete]
      Completed --> Closed((Closed))
      Failed --> Closed
      Error --> Closed
      Exited --> Closed
      Obsolete --> Closed
  
```

The diagram illustrates the state transitions of a task lifecycle. The states are: Inactive, Created, Ready, Reserved, InProgress, Completed, Failed, Error, Exited, and Obsolete. A 'Suspended' box groups the Ready, Reserved, and InProgress states. Transitions are labeled with events and actions. The lifecycle starts at Inactive, moves to Created, then to Ready, Reserved, or InProgress. From Ready, it can be suspended, forward, resume, or claim/delegate to Reserved. From Reserved, it can be suspended, resume, delegate, or start to InProgress. From InProgress, it can be suspended, resume, complete with response to Completed, complete with fault response to Failed, or reach Error, Exited, or Obsolete due to non-recoverable error, WS-HT exit, or skip. All final states (Completed, Failed, Error, Exited, Obsolete) lead to a Closed state.

- Отдельный сервис – скачиваем бандл на базе WildFly, распаковываем, конфигурируем, запускаем.
 - Есть бандл в виде Docker-образа.
- Как модуль в составе ИС – качаем нужные `jar`'ы / `war`'ы, добавляем в зависимости приложения, вызываем API.

BPMN2 Designer

Business Process [evaluation]



The screenshot displays the BPMN2 Designer interface for a process named "evaluation". The process diagram shows a start event leading to a "Self Evaluation" task, which then splits into two parallel tasks: "HR Evaluation" and "PM Evaluation". These tasks merge at a join gateway before reaching the end event. The interface includes an Object Library on the left, a Properties (BPMN-Diagram) panel on the right, and a Simulation Results tab at the top.

Object Library:

- Choose library set: Full
- Full
- Tasks
- Subprocesses
- Start Events
- End Events
- Catching Intermediate Events
- Throwing Intermediate Events
- Gateways
- Service Tasks
- Connecting Objects
- Data Objects
- Swimlanes
- Artifacts
- Workflow Patterns

Properties (BPMN-Diagram):

Name	Value
Core Properties	
AdHoc	false
Executable	true
Globals	
ID	evaluation
Imports	
Package	Evaluation.src.main.resources
Process Name	Evaluation
Variable Defi...	employee:java.lang.String,reason:java.lang.String,perf...
Version	1
Extra Properties	
Documentati...	
Target Name...	http://www.omg.org/bpmn20
TypeLanguage	http://www.java.com/javaTypes
Simulation Properties	
Base Currency	
Base time unit	seconds

Business Process Metadata



MTMO BT

Form Modeler

Business Central

Menu

Spaces > MySpace > Mortgage_Process > P master > Application

Components

Model Fields

mortgageamount

errors

Form Controls

HTML

TextBox

TextArea

IntegerBox

DecimalBox

CheckBox

DatePicker

Slider

ListBox

RadioGroup

MultipleSelector

MultipleInput

Document

DocumentCollection

Form Modeler [Application]

Model Overview

Down Payment

Down Payment

Years of amortization

Years of amortization

Applicant

Name

Name

Annual Income

Annual Income

SSN

SSN

Property

Age of property

Age of property

Address of property

Address of property

Locale

Locale

Sale Price

Sale Price

255